



Preface

In the fall of 2014, Stanford ran a class called CS183B: How to Start a Startup. It had no textbook. Twenty lectures, most delivered by a founder or investor who had actually built the thing they were describing, covering everything from how to pick an idea to how to fire someone with dignity. The transcripts of those lectures have circulated online for years — the raw material is freely available, unglamorous, occasionally contradictory, and never organized into anything a reader could sit down with the way you'd sit down with a real book.

This is that book. Not a summary of the course, and not a transcript with the "umms" removed. A synthesis — twenty speakers' worth of hard-won, sometimes conflicting experience, reorganized around the actual arc of building a company instead of the order in which guest speakers happened to be available on a Tuesday, with the gaps filled in, the terms defined from scratch, and the places where two speakers said the same thing without knowing it pointed out explicitly, because that convergence is itself a signal worth noticing.

Who This Book Assumes You Are

You can build software. You understand systems, and you're comfortable with the idea that a complicated thing is usually just several simple things stacked on top of each other. You do not know what a cap table is. You've never read a term sheet. The words "Series A" and "SAFE" and "vesting cliff" are either unfamiliar or half-familiar in a way that's almost worse than not knowing them at all, because you'd nod along in a meeting rather than admit you needed the basics explained.

This book explains the basics. Every term gets defined the first time it's used, the same way you'd want a new library's documentation to define a term before using it three paragraphs later. Nothing is assumed. If a chapter uses a word from a previous chapter, it either re-explains it briefly or points you back to where it was covered — you should never have to already know something to follow what you're reading.

What This Book Is Not About

Chapter 1 draws this line explicitly, but it's worth stating here too, before you've committed to reading further: this book is about one specific, unusual kind of company — one built to grow fast, take outside investment, and attempt to become large. It is not a book about starting a small business, a consultancy, or a lifestyle company that stays deliberately modest. Both paths are legitimate. They require different founders, different capital, and different definitions of success, and conflating them is one of the more common ways people misapply advice meant for one path to a company actually pursuing the other. If you're not yet sure which path you want, Chapter 1 is where that question gets addressed directly, before anything else.

How This Book Was Made

Every chapter in this book traces back to a specific lecture, or combination of lectures, from the original course — that mapping is documented in this book's planning materials and referenced in each chapter's "Further Reading" section. Where the lectures assumed knowledge a reader without a business background wouldn't have — how a SAFE actually converts, what a valuation cap does to real numbers, what a profit and loss statement even is — this book adds that missing layer explicitly, clearly marked as an addition rather than presented as though it came from the original course.

Where lectures given in 2014 made claims later evidence has complicated — that cofounders should never work remotely, for instance, a claim GitLab and Zapier have since tested directly — this book says so, in sidebars marked "What's Changed Since 2014," rather than either pretending the original advice is timeless or quietly updating it without telling you.

Where compressing a lecture's original content risked losing something a reader might actually need — the full text of a mock pitch, worked through line by line; the full historical weight of a single management case study — that material was moved into an appendix rather than cut. Nothing that mattered was thrown away in the name of a tighter narrative; it was relocated to wherever it could still be found.

How to Read It

Linearly, ideally, at least the first time through. The chapters build on each other deliberately: Part One establishes vocabulary the rest of the book uses without re-explaining it; Part Four assumes the product discipline from Part Three; Part Six assumes you already have the team, product, and capital covered in the parts before it. Reading Chapter 15's legal mechanics before

Chapter 1's argument for why you'd want to do any of this at all would be a strange way to spend your time.

After the first pass, treat it as a reference. Each chapter ends with a summary you can scan without rereading the whole argument, and the glossary and index at the back exist specifically so you can find the one paragraph you remember reading, months later, once you actually need it — when a cofounder conversation is getting uncomfortable, when a term sheet lands in your inbox, when a management decision seems small until you notice who else will eventually hear about it.

A Note on What This Book Cannot Do

Every chapter argues, in one form or another, that the tactics inside it only matter in service of something upstream: a real compulsion, a real insight, a real understanding of your own users. No book, including this one, can supply that. What it can do is make sure that once you have it, you're not slowed down by not knowing what a SAFE is, or losing a cofounder relationship because nobody thought to write down the equity split early, or misjudging when to fire someone because no one ever explained what "fire fast" actually looks like in practice. That's the gap this book is trying to close — not the compulsion itself, which Chapter 1 argues has to come from somewhere else entirely, but everything downstream of it.



Chapter 1 — Why Start a Startup

There is a scene near the start of the movie *The Social Network*. Mark Zuckerberg's Harvard dorm room fills up with laptops, cheap liquor, and a coder pouring champagne over someone's keyboard while a crowd cheers. It is a good scene. It is also, according to Dustin Moskovitz, almost entirely fiction. Moskovitz co-founded Facebook with Zuckerberg, so he was actually there. The real version of that photo — the one Moskovitz showed a Stanford class in 2014 — is a picture of Zuckerberg sitting alone at a desk, head down, typing. There are two photos of him in that exact same position, taken on different days, wearing different clothes. That is the whole scene. A guy at a desk, for years.

Moskovitz was not being modest. He was making a point, and this book starts from the same point: almost everything people believe about why they want to start a company is wrong. Believing the wrong reasons is not a small mistake. It is the main reason most people who start companies for those reasons quit, burn out, or build something nobody needed. Before this book teaches you anything about ideas, products, fundraising, or growth, it needs to talk you out of the reasons you probably think you have.

This Book Is About One Kind of Company

First, a boundary. Sam Altman opened the Stanford class this book is built from with a line that the whole rest of the book depends on:

All of the advice in this class is geared towards people starting a business where the goal is hyper growth and eventually building a very large company... if you try to do these things in a lot of big companies or non-startups, it won't work.

There is a good, respectable way to build a small business: a consulting firm, a local service company, a shop, a small business that supports you and a few employees forever without ever trying to get big. Nothing in this book argues against that path. But nothing in this book is *for* it either. This book is about a different, less common kind of company — one built to grow fast,

take money from investors, and try to become large. The psychology, the tactics, and the mechanics in this book are built for that goal only.

A fast-growing, investor-backed company and a steady small business are not the same thing wearing different clothes. They need different founders, different money, and different definitions of success. If you are not sure which one you want, it is worth figuring that out before you read Chapter 2. The two paths call for different kinds of people.

With that boundary set, assume you *do* want the fast-growth kind. Why would you do it?



The Core Idea: Four Reasons That Sound Right, and One That Isn't Really a Reason

Dustin Moskovitz, in that same lecture, listed four reasons people commonly give for wanting to start a company:

1. It's glamorous.
2. You get to be the boss.
3. You get flexibility over your own schedule.
4. You'll make more money and have more impact than you would at an existing company.

All four are wrong, or at least wrong enough to be dangerous if they are your *main* reason. Each one falls apart after about ten minutes of contact with an actual startup. What replaces them is not a fifth item on the list. It's a different kind of test. Moskovitz summed it up as: *you can't not do it*.

The rest of this chapter does three things. First, it shows why each of the four common reasons fails. Second, it explains what "you can't not do it" actually means. Third, it looks at where the edges of this advice are — because "should I do this" has real answers that depend on your situation. It isn't a slogan.

Why the Four Reasons Fail

"It's glamorous"

The glamour comes from movies and culture, not from real data. *The Social Network* cut together parties and big "eureka" moments like a highlight reel, and the highlight reel is what people remember. What doesn't make it into the movie is what Moskovitz actually lived through while running Facebook and later Asana: sitting at a desk, answering support emails, doing sales, working through hard engineering problems, for years, with no editing.

The Economist ran a piece in 2014 called "Entrepreneurs anonymous." It described founders hiding under their desks and talking openly — maybe for the first time in the business press — about founder depression. That is not a made-up story to sell magazines. It describes a real risk of the job, and it comes from a simple fact: you carry the fear of failure not just for yourself, but for everyone who left a stable job to follow you. Sometimes people depend on you financially. Even when they don't, they've given you the best years of their working life, and you are responsible for what that turns out to be worth. Moskovitz said it plainly, describing his own early years at Facebook: "I got really unhealthy, I wasn't exercising, I had a lot of anxiety actually throughout my back, like almost every six months, when I was twenty-one or twenty-two, which is pretty crazy." That is not glamour. That is a 22-year-old with a repeating stress injury.

There's also a problem that's hard to undo, and the glamour story never mentions it. An unhappy employee can quit. An unhappy founder mostly can't — not without real cost to the people who joined because of them. Moskovitz's rough estimate: if a startup is going well, you're committed for around ten years. If it's going badly, more like five — three years to realize it isn't working, then two more years finding a soft landing for the company and its people. Leaving earlier than that doesn't just cost you. It costs everyone who trusted the path you were selling them.

None of this means startups are miserable. It means the glamour is something sold to you from the outside, and if it's your reason for starting a company, the first real year will remove that reason completely. You'll be left needing a better one, and you won't have it.

Sidebar: What's Changed Since 2014

When this course was taught, talking openly about founder depression was still new enough that a major magazine covering it counted as news. That has changed a lot since. Y Combinator itself now runs mental-health-focused support for founders, and it is common today for accelerators, investors, and founder communities to treat burnout and depression as a normal, expected part of the job — not a shameful secret. The underlying reality this

chapter describes has not changed. What has changed is that founders now have more permission to talk about it, and more support built specifically to help them, than they did in 2014.

"You get to be the boss"

This is probably the most tempting of the four reasons, especially if you've ever worked for a manager whose decisions seemed obviously bad. The fantasy goes: I'll start my own thing, and I'll be the one making the decisions, and they'll be right, because I'm the one making them.

Moskovitz quoted Phil Libin, who was CEO of Evernote at the time, on what the job actually is:

People have this vision of being the CEO of a company they started and being on top of the pyramid. Some people are motivated by that, but that's not at all what it's like. What it's really like: everyone else is your boss — all of your employees, customers, partners, users, media are your boss. I've never had more bosses and needed to account for more people today. The life of most CEOs is reporting to everyone else, at least that's what it feels like to me and most CEOs I know. If you want to exercise power and authority over people, join the military or go into politics. Don't be an entrepreneur.

There's a second point here that's easy to miss. The managers you thought were making obviously bad calls at your old job probably weren't idiots. They were making hard tradeoffs with imperfect information, while several people pulled them in different directions — the exact spot you'll be in within your first month as a founder. Most of a CEO's actual time, Moskovitz found, goes to settling conflicts other people bring to them: two customers who want opposite things, two employees who disagree about direction. You don't get to just decide what matters and go do it. You get to decide who you're going to disappoint the least, over and over, all day.

"You get flexibility"

This is really a smaller version of the "be the boss" fantasy — control over your own schedule — and it fails for a related reason. Libin again, on this point:

If you're going to be an entrepreneur, you will actually get some flex time to be honest.

You'll be able to work any 24 hours a day you want!

You are always on call, whether you meant to be or not, because you don't get to choose which part of the day something important happens. You are also, like it or not, the emotional thermostat of your own company: your employees copy your energy level. Slow down for a week because you're tired, and your team notices, and they slow down too. And if you truly care about the problem you're solving, that passion pulls you toward the work anyway, no matter what your calendar says about weekends.

There is one honest exception, and it matters enough to get its own section below: the small "lifestyle business" path, where a four-day week or something close to a Tim Ferriss-style light schedule can genuinely work — but only if you deliberately keep the company small. The moment more than two or three people depend on you, that model stops working, and you're back to full-time commitment whether you planned for it or not.

"You'll make more money and have more impact"

This reason sounds the most rational, so it deserves the closest look. The numbers are less clearly in the founder's favor than most people assume.

Moskovitz built a comparison, using 2014 figures, between joining an already-successful company as an early employee and starting your own company from nothing:

Path	Rough outcome (2014 figures)
Employee #100 at Dropbox (joined a couple of years in)	About \$10M in upside already locked in, with more growth ahead
Employee #100 at Facebook (joined a couple of years in)	About \$200M
Employee #1,000 at Facebook (joined around 2009)	About \$20M
Founder of a made-up "Uber for Pet Sitting," building a \$100M company, ending up owning roughly 10% of it (after selling shares to investors and giving some to early employees along the way)	About \$10M

Path	Rough outcome (2014 figures)
Founder of a made-up "Uber for Space Travel," building a \$2B company	Much larger, but this requires a truly huge idea that doesn't exist yet

This table is not an argument against starting a company. It's an argument against starting one *only for the expected payoff*. The math only favors founding if you are extremely confident you can build something in the \$100M–\$2B range. And you should have more confidence in a company that already exists and is growing fast, like Facebook in 2009 or Dropbox in 2014, than in an idea you haven't even started yet.

More importantly, the "impact" half of this reason doesn't require founding anything at all. Bret Taylor joined Google as roughly its 1,500th employee and built Google Maps there — a product used by hundreds of millions of people, with no company of his own needed. Justin Rosenstein joined Google a bit later, built an early prototype of what became Gchat as a side project, then moved to Facebook as roughly employee 250, where he led the hackathon project that became the Like button — one of the most-used pieces of interface on the internet. That project likely could not have existed as its own startup: it needed Facebook's huge user base to work at all. Both of these are enormous- impact careers, and neither one required starting a company, raising money, or giving up any ownership to get there. If impact is really your goal, joining a company that's already moving fast can get you there faster than starting one from zero.

None of this means money and impact don't matter. It means: if they are your *reason*, an honest look at the numbers usually points somewhere other than founding a company. If you still want to found one after doing that math, your real reason is something else. That "something else" is what the next section is about.

The Test That Actually Works: "You Can't Not Do It"

Moskovitz's own answer, once he gets to it, is short: you start a company when you can't not do it. He means this in two ways, and both need to be true at once.

You're compelled. Not "interested in" — compelled. This matters because you will need that level of pull to survive the real difficulties described above. It also matters because you can't fake it well enough for it to work. Moskovitz's claim: strong job candidates can sense an unconvincing

founder, and there are enough founders out there with the real thing that talented people don't need to take a chance on someone who's only somewhat interested. Your own subconscious, he adds, also tends to know the difference — and that tends to show up in ways that hurt you before you even notice.

The world needs it. This checks that your compulsion isn't just a personal itch. If the problem genuinely doesn't need solving, or doesn't need *you* specifically to solve it, that's a sign your time is better spent somewhere else — there's no shortage of real problems, so there's no reason to invent one. And if it's a real problem but you're not the right person for it, Moskovitz's honest worst case is this: you go do it anyway, you beat the team that actually should have built it, and the world ends up with a worse version of the thing it needed. That's not a hypothetical to feel bad about later. It's a reason to be honest with yourself about fit before you start.

Moskovitz's own example is Asana. He and his cofounder Justin Rosenstein were, in his words, "reluctant entrepreneurs" — they were already working at Facebook on something genuinely important. At night, after a full day at their real jobs, they kept building an internal task-management tool, because they could see it solving something real inside the company, and no one else seemed to be solving it well anywhere. Eventually they had what he called "the hard conversation": what happens if we don't start this company? They had seen the impact inside Facebook. They believed it would matter to the world. They were convinced no one else was about to build the right version of it. That combination — *I can't stop working on this, and if I don't, something valuable doesn't happen* — is what Moskovitz means when he says the idea was "beating itself out of our chests." It's a specific, checkable feeling, not just a stand-in for enthusiasm.

The same pattern shows up outside this course's original 2014 lecture hall, too. Melanie Perkins, cofounder of the design tool Canva, spent about three years pitching investors before one finally said yes — by her own account, more than a hundred said no first. What kept her going wasn't stubbornness for its own sake. Years earlier, teaching university students how to use a clunky, hard-to-learn design program, she'd watched them struggle with software that should have been simple. She built her first product — a much simpler tool for designing school yearbooks — to fix that one specific problem, long before Canva existed. By the time a hundred investors were telling her no, she wasn't chasing an idea that merely sounded good. She had already been living inside the problem for years. Canva later being worth billions of dollars doesn't prove the test right on its own. What proves it is that she kept building through a hundred rejections in a row —

which only makes sense if the "can't not do it" half of the test was already true, well before anyone else agreed with her.

Common Mistakes

Founders talk themselves into starting for the wrong reasons in a few common ways:

- **Treating this as a two-to-three-year plan.** A common pattern, especially with student founders, is assuming they'll do the startup thing for a couple of years and then move on to what they're "really" passionate about. Moskowitz and Altman both call this close to a universal mistake. Real companies — the ones that work — usually take a decade. If you don't love what you're building, you're likely to give up around year three or four, right when the real payoff would have started arriving.
- **Wanting the identity, not the problem.** Wanting to *be a founder* is different from being pulled toward a specific problem. Someone motivated by the identity tends to pick whatever idea is available, rather than the one they can't stop thinking about. That produces exactly the kind of copycat, uninspired company that struggles to hire and struggles to survive year three.
- **Mistaking flexibility for freedom.** Believing the "control your own schedule" pitch, then being surprised — and often ashamed, since people don't talk about this much — when the job turns out to be always-on. That shame makes things worse: founders who feel like they're the only ones struggling this way are less likely to build the support network the burnout answer below says you need.
- **Doing the money math and stopping there.** Running the comparison in the table above and treating the dollar number as the whole decision, instead of using it as a filter. If you're not confident you're building something in the Facebook/Dropbox class, the money argument for founding doesn't hold up on its own — you need a different reason if you're going to go ahead anyway.
- **Ignoring how hard this is to reverse.** Underestimating how committed you become the moment you take money and hire people. Quitting a startup a year in doesn't just cost you — in Moskowitz's words, it hurts your reputation "for the rest of your career," and it costs everyone who joined because of the path you presented to them.



Tradeoffs and Edge Cases

The lifestyle-business exception is real, and it's not a lesser choice — it's a different project. If you want a lifestyle business — something that supports two or three people forever, without needing to grow fast or take outside money — a flexible, Tim Ferriss-style schedule can genuinely work, as long as you deliberately keep it small. That's a real, legitimate choice. But it comes with a different set of tradeoffs than everything else in this book assumes, which is why this chapter opened by drawing the line clearly: this book is written for the fast-growth path. If you're building the other kind of company, treat every "you must give everything to this" claim in the chapters ahead as conditional on the goal you've chosen.

Sidebar: The Rise of the "Calm Company"

*This lecture was given in 2014, near the peak of an era when "hyper-growth or nothing" was close to the only story Silicon Valley told about startups. In the years since, a real counter-movement has grown up around exactly the path this chapter treats as a footnote. Basecamp's founders wrote a whole book, *It Doesn't Have to Be Crazy at Work*, arguing against always-on hustle culture. The "indie hacker" movement — solo or small-team founders building profitable, modest-growth software companies without ever raising outside money — now has its own communities, conferences, and well-known success stories. Investor Aileen Lee coined the term "unicorn" in 2013 for a \$1B+ startup; a group of founders later coined a deliberate counter-term, "zebra," for companies built for sustainable profit instead of hypergrowth, arguing those deserve just as much respect. None of this contradicts what this book teaches — a fast-growing, investor-backed company is still a real and valuable thing to build. But it is worth knowing, before you commit to the path this book describes, that "hyper-growth or you're not a real founder" is itself a choice this book is making on purpose. It is not the only option that exists.*

Should you do this in college? Paul Graham, teaching the same course, said no — firmly. His reasoning: starting a company "changes your life irrevocably" (in a way you can't take back), the same way having kids does. There are things you can only do easily before that happens: travel with no deadline, follow a project that seems to have no payoff, meet

dozens of potential future cofounders in a place built exactly for that. Once a startup is working, those options close. Graham's view: you're not a student who happens to also run a company. If the startup succeeds, "you're not a student anymore," no matter what your enrollment status says. His advice, aimed at an audience of college students: "Be a real student and not start a startup or start a real startup and not be a student." He argues twenty usually isn't the best age to do this — not because young founders can't succeed (some clearly do), but because the serendipity you give up by succeeding early is a real cost. And it's a cost you never get to see, because if you succeed at twenty, you never find out what the other path would have given you. Larry Page, in Graham's telling, started running as fast as he could at twenty-five and hasn't stopped to catch his breath since — every day something comes up inside Google that only he can deal with, vacation or not. Mark Zuckerberg's version of the same cost: he can't backpack around a foreign country the way an ordinary twenty-something might, because any trip he takes becomes either a de facto state visit or an exercise in hiding from attention. Graham's line about YC's own successful founders: "it never gets any easier." The problems change — construction delays on a new London office instead of a broken air conditioner in a studio apartment — but the total amount of worry doesn't go down. It may go up.

You genuinely can't tell in advance whether you're capable of this. Graham makes this point directly, and it's worth taking seriously rather than dismissing as false modesty: your life so far has probably given you a decent read on whether you could become, say, a professional athlete or a mathematician. It has probably given you almost no read on whether you could become a startup founder — because starting a company changes you. What you're really trying to guess isn't what you are now, but what you could become, and that's close to unknowable from the outside, and often from the inside too. Graham's report, after nine years of picking founders at Y Combinator: confidence going in doesn't predict outcomes. The confident applicants who expected to do great were no more likely to succeed than the ones who arrived wondering if Y Combinator had made a mistake letting them in. His practical rule: if you are *completely* terrified of the idea, you probably shouldn't do it, unless you're someone who's specifically energized by doing scary things. If you're merely unsure — which is far more common — the only way to really find out is to try it. Graham's addition, aimed at the college-age reader specifically: try it, "just not now."

Burnout is not something you can plan your way around. It's a recurring cost you manage, not solve. A student in the same course asked how to handle burnout while

staying productive. Sam Altman's answer was deliberately unglamorous: "it sucks and you keep going." The advice that works for employees — take a vacation, recharge, come back — mostly doesn't work for founders, because a founder's problems don't pause while they're away. They pile up. What actually helps: a support network you build *before* you need it, and a willingness to deal with the real problem directly instead of waiting for the feeling to pass on its own. Founder depression, Altman is direct about this, is serious and real, not a minor inconvenience — which is exactly why "it's glamorous" and "you'll have flexibility" are such dangerous reasons to start a company. They set you up to be blindsided by a cost that nobody advertises.

Applying the Test

Think about two people, both with an idea for the same kind of product.

The first has spent two years frustrated by one specific, narrow problem in her own work. She has already built three rough versions of a fix for herself, because she couldn't stop thinking about it. She describes the market not in terms of how big it could eventually get, but in terms of who is underserved right now. Ask her why she'd do this instead of joining an existing company working on something similar, and she doesn't really have an answer beyond: *I'm already doing this, at night and on weekends, and I don't seem to be able to stop.*

The second has read about the funding rounds and the acquisition news. He likes the idea of being a founder. He picked this particular idea three weeks ago because it sounded like a defensible space with a large potential market. Ask him why this instead of joining a fast-growing company already working nearby, and the honest answer is closer to: *because I want to be the one who gets the credit.*

You don't need to read anyone's mind to tell these two apart. The first person passes "you can't not do it" on both halves: she is compelled, and she already has evidence the problem is real, because she has been living inside it. The second person hasn't shown either half — he's justifying a decision he made for reasons this chapter just spent several thousand words explaining don't hold up. Neither movie-style glamour, nor the fantasy of being the boss, nor the promise of flexibility, nor the raw money math got the first founder

to her decision. All four of them are exactly what got the second founder to his. That's the test, and you can run it on yourself before you ever run it on anyone else.

Chapter Summary

- This book is about venture-backed, fast-growth-oriented companies specifically — not lifestyle businesses, which are a legitimate but different path with different rules.
- Four common reasons for starting a company — it's glamorous, you get to be the boss, you get flexibility, you'll make more money and have more impact — all fall apart under a closer look, and are dangerous as your main motivation.
- The glamour story hides a real cost: founder depression is well documented, and leaving a struggling company early has lasting professional and financial costs for you and everyone who joined you.
- "Being the boss" flips in practice — you end up reporting to everyone: employees, customers, partners, media.
- "Flexibility" flips the same way — you are on call at all hours, and your energy sets the tone for the whole team.
- The money-and-impact argument, done honestly, often favors joining an already-successful company over founding a new one. Bret Taylor and Justin Rosenstein built huge-impact products without ever starting a company.
- The real test is "you can't not do it": you're personally compelled, and the world (not just you) needs the thing built.
- You cannot reliably know in advance whether you're capable of this — confidence going in does not predict outcomes. So the honest answer to "am I ready" is usually: try it, if you're not terrified, and not right now if you're still in college.



Exercises

1. **Recall:** In your own words, state the two conditions in Moskowitz's "you can't not do it" test, and explain why both are needed. Why would satisfying only one of them not be enough?
2. **Apply:** Take an idea you've thought about building a company around. Write down, as honestly as you can, which of the four tempting-but-wrong reasons (glamour, being the boss, flexibility, money/impact) is actually driving your motivation right now. Then write down what would have to be true for you to pass the "you can't not do it" test instead.
3. **Apply:** Using the comparison table in this chapter as a model, make your own rough guess: what would you likely earn, and what impact would you likely have, by joining the fastest-growing company you could realistically get hired at today — compared to founding the company you're considering? Where does the honest math point?
4. **Confront a counterexample:** Mark Zuckerberg started Facebook at nineteen, in college — directly against Paul Graham's advice in this chapter. Graham would likely say Zuckerberg is exactly the "extremely unlikely" case he mentioned: a side project that took off before its founder ever decided to run a company. Does that answer fully resolve the tension, or does it just relabel the exception? What would you need to see in your own situation to know whether you're a real outlier, versus someone rationalizing an exception that doesn't apply to you?



Further Reading

- Dustin Moskowitz's original "Why to Start a Startup" talk, delivered as part of Stanford's CS183B, is the main source for most of this chapter and is worth reading or watching directly.
- Paul Graham's essays on startup timing and founder psychology, mentioned throughout his CS183B lecture, go deeper into the "you can't tell" argument than the lecture format allowed.
- *The Economist's* 2014 feature on founder mental health, "Entrepreneurs anonymous," is the piece Moskowitz points to on founder depression as a documented, not anecdotal, problem.

- Jason Fried and David Heinemeier Hansson's *It Doesn't Have to Be Crazy at Work* makes the fullest case for the "calm company" alternative mentioned in this chapter's sidebar — useful if you suspect hyper-growth is not actually the goal you want.





Chapter 2 — The Anatomy of a Great Idea

The thirteenth search engine. Nobody wanted that. Search was already a solved problem in the late 1990s, or so everyone believed, and the real money was supposed to be in "portals" — big sites packed with news, email, weather, and shopping. A plain page with one text box and two buttons sounded like a step backward. That plain page was Google.

The tenth social network. Also a bad idea, on paper — MySpace had already won, and this new one only let in college students, which meant it had no money and a tiny audience by design. That was Facebook.

A website where strangers sleep on your couch and pay you for it. That one sounds bad by any measure — unsafe, small, a little strange. That was Airbnb.

None of these ideas sounded good. That is not a coincidence. It is close to a rule. Sam Altman, opening the same Stanford course this book is built from, put it directly: "the best ideas often look terrible at the beginning." This chapter is about why that's true, what actually separates a good idea from a bad one once you look past how it sounds, and how to get better at finding ideas like these instead of the ones that only sound good.



The Core Idea: What "Idea" Actually Means, and Why It Comes First

Altman opens with a rough formula for how a startup's outcome gets decided:

$$\text{Outcome} \approx \text{Idea} \times \text{Product} \times \text{Execution} \times \text{Team} \times \text{Luck}$$

(where luck is treated as a random number between zero and ten thousand)

Four of those five factors are things you can control. The fifth you can't, and that's the point of writing it into the formula at all — you may do everything right and still fail,

because luck really is that large a multiplier. But if you get the other four right, you have a real shot. This chapter is only about the first one: the idea.

It has become fashionable to say the idea doesn't matter — that you should just start building something, throw it at users, and pivot until something sticks. There's a grain of truth in this. Things do evolve in ways you can't fully predict ahead of time. And Altman says great execution is roughly ten times more important than a great idea, and about a hundred times harder to pull off. But the fashionable version of this advice has swung too far. A bad idea, executed brilliantly, still gets you nowhere. Most great companies start from a real idea, not a random pivot. Even the pivots that do work tend to prove the point rather than disprove it: Airbnb happened because Brian Chesky couldn't make rent and had a spare air mattress, which is to say, it wasn't a random pivot at all — it was the founders' own problem, staring them in the face. Look at the track record of pivots that came from nowhere, disconnected from anything the founders actually cared about or understood, and you'll find they rarely become big companies.

So "idea" in this book means something broader than "the product." It includes:

- the size of the market today, and — far more important — how fast that market is growing
- your strategy for getting more users once you have some
- how hard it will be for someone else to copy you once you've succeeded

That third point is the subject of the entire next chapter, so this one won't try to cover it in full — just note for now that a good idea usually points toward a business that gets harder to copy over time, not easier. If you're evaluating an idea and can't say anything about that yet, that's worth sitting with before you go further.

You're likely to spend ten years on whatever idea you pick. That's worth real up-front thought before you commit. Not because the plan you write down will survive contact with reality — it mostly won't. The exercise of thinking it through is valuable on its own, and most founders skip it entirely. The idea itself doesn't need to be fully worked out on day one. It needs to be a good starting kernel: something that can grow and become more ambitious as you learn more, not something you've already mapped out to its final form.



Why It's True: Good Ideas Are Supposed to Sound Bad

The market you can't get later

Here's a fact that catches a lot of first-time founders off guard: you can change nearly everything about a startup except the market. The team can grow. The product can change completely. The whole business model can flip. But if you picked a market that was never going to grow, no amount of execution fixes that. This is why so much of evaluating an idea is really about evaluating a market, not a product.

Most investors, in Altman's view, make a specific and common mistake here: they focus on how big a market is *today*. What matters far more is how fast it's *growing*, and whether there's a real reason it will keep growing rather than topping out. A small market growing fast beats a huge market growing slowly, because the huge slow market is probably already full of competitors, and the small fast one usually isn't yet.

Investors sometimes use the term **TAM** — total addressable market — as shorthand for the ceiling on how big a market could eventually get. It's a useful term, but by itself it answers the wrong question. A market with a large TAM that's flat or shrinking is a worse bet than a market with a smaller TAM that's compounding every year. You'll get more practice sizing markets properly once this book reaches fundraising in Part Five — for now, the point is just this: growth rate matters more than the number you'd put on a slide today.

There's a well-known question, credited to the venture firm Sequoia, that captures this well: *why now?* Why is this the right moment for this specific idea and this specific company — and why couldn't it have been done two years ago, and why will it be too late two years from now? The startups Altman has seen succeed almost always have a sharp answer to this question. If you don't have one, that's a reason to be suspicious of your own idea, not proud of how original it sounds.

One reason there are more good "why now" answers today than in earlier decades: as Marc Andreessen has put it, software is eating the world — meaning the number of industries being rebuilt on top of new technology keeps growing, which means the supply of real "why now" moments keeps growing too. You don't need to invent a market from nothing. You need to notice one that's already starting to move, and get there early.

Sidebar: A Modern "Why Now"

The clearest recent example of a "why now" moment is the wave of companies built around large language models starting in the early 2020s. None of the underlying idea — software that can write, summarize, or answer questions in natural language — was new. What changed was that the cost of running these models dropped and their quality crossed a threshold where products built on top of them actually worked well enough to use. The founders who moved early weren't smarter than the people who'd had similar ideas years earlier. They were answering "why now" correctly: the technology had just become cheap and good enough, and it wouldn't stay this uncrowded for long. That's the same pattern Altman describes with cheaper cloud storage and faster networks enabling Dropbox, a decade earlier — a real, external shift creates a short window, and the founders who notice the window early get the advantage.

Altman has a memorable way of describing what a genuinely growing market feels like from the inside: surfing someone else's wave, stepping into an up elevator, being part of a movement. All three images point at the same thing. You want a market that's already being pulled upward by something larger than you, not one you have to push uphill by yourself.

This is also one of the few places in this book where being young and technical is a real advantage, not something to work around. Altman's advice to students specifically: trust your own instincts here more than you'd trust an older, more experienced person's. You are already living inside the behavior that's about to become mainstream — what you and your friends are doing with technology right now is a better predictor of where a market is headed than market research an outsider could buy.

Small markets, not big ones

If good ideas look bad, one specific reason they look bad is that they usually start in a market that sounds too small to matter. That's not an accident — it's close to a requirement. Altman's advice: find a small market you can dominate completely, then expand outward from it in stages.

Figure 2.1 — Expand outward, don't start big


```
Ring 1 (start here):  a small market you can dominate completely
                      |
                      v  expand
Ring 2:                the adjacent market next door
                      |
                      v  expand
Ring 3:                the large market you were really after all along
```

The mistake most first-time founders make is starting at the outer ring — going after a giant market on day one because it sounds more impressive, or more fundable. That's usually a sign the idea hasn't been thought through carefully enough, because a market that large and that obvious is already full of competitors. The founders who succeed usually start at the center: a market so small that it looks unattractive or even a bit silly, dominate it completely, and only then work outward.

This is also why it's fine — even good — if the first version of your idea doesn't sound big. A common mistake, especially with first-time founders, is believing the pitch has to sound huge from day one. It doesn't. It needs to take over one small, specific market first. "Unpopular but right" is the actual goal: an idea that sounds bad enough that nobody else is working on it, but is right enough that it works.

None of this contradicts the "why now" argument above — it sharpens it. The best version of a "why now" answer usually explains why a market that looks small today is about to grow quickly, not why a market that's already huge is worth entering.

Good ideas fit in one sentence

There's one more test worth applying before you go further: can you explain the idea in a single, simple sentence? Altman calls this one of the more counterintuitive things about good startup ideas — they are almost always very easy to explain. If it takes more than a sentence to say what you're doing, that's usually a sign the idea itself is too complicated, not that you haven't found the right words yet.

A modern example: Stripe's own pitch has stayed close to "payment infrastructure for the internet" for well over a decade. That's simple enough that a first-year engineering student could repeat it back correctly after hearing it once. Compare that to a pitch that needs three sentences and a diagram before anyone understands what the company

actually does. The extra length is rarely a sign of a richer idea. It's usually a sign the founders haven't yet figured out what's essential and what isn't.

The gap between the jargon version and the real version is often bigger than founders realize. None of the following companies actually pitched themselves the "jargon-heavy" way in the left column below — this is an illustrative comparison, not a quote from any lecture — but the exercise of writing both versions out side by side makes the point concrete:

Company	A jargon-heavy version nobody actually used	The one-sentence version that actually worked
Uber	"A peer-to-peer transportation network leveraging mobile technology to disrupt urban mobility"	"Tap a button, get a ride."
Airbnb	"A trust-and-verification marketplace platform for peer-to-peer accommodation sharing"	"Stay in someone's home instead of a hotel."
Stripe	"A unified API-driven payments infrastructure layer for internet-native commerce"	"Payment infrastructure for the internet."

Notice that the one-sentence version isn't a dumbed-down summary of the jargon-heavy one — it *is* the idea. The jargon-heavy version doesn't add information; it hides the idea behind words that sound more like a pitch. If you find yourself reaching for language like the left column to explain your own idea, that's worth treating as a warning sign, not a sign of sophistication.

Build what you need, or get uncomfortably close to who does

The strongest position to build a product from is needing it yourself. You'll understand the problem at a level no amount of user interviews can fully replace. If that's not your situation — if you're building something for someone else's problem — the honest takeaway is that you start at a real disadvantage, and the fix is to close the distance as

much as you possibly can: work in your customers' office if they'll let you, and if not, talk to them multiple times a day.

Paul Graham, teaching later in the same course, made a related and sharper point about what "expertise" actually means here. Mark Zuckerberg, he pointed out, didn't succeed at Facebook because he was an expert on startups — Facebook was originally incorporated as a Florida LLC, which is not what an experienced startup lawyer would recommend. He succeeded because he understood the people he was building for. Graham's conclusion: what you need is not expertise in startup mechanics. What you need is expertise in your own users. This is worth sitting with, because it cuts against a natural instinct — the instinct to go read about term sheets and incorporation and pitch decks before you've spent any real time with the people you're trying to build for.

There is no trick — the market decides, not a judge

This same idea shows up in a blunter form when Graham talks about why founders keep looking for shortcuts. Most of a founder's life up to the point they start a company, he argues, has trained them to look for the trick — the equivalent of figuring out what will be on the exam instead of learning the material, which is a skill plenty of people (himself included, by his own account) got good at in school. Startups are the first part of life where that skill stops working, because there's no examiner to trick:

There is no boss to trick, how can you trick people, when there is nobody to trick? There are only users and all users care about is whether your software does what they want, right? They're like sharks, sharks are too stupid to fool, you can't wave a red flag and fool it, it's like meat or no meat.

This is also, in Graham's view, exactly what's wrong with the phrase "growth hacks":

Whenever you hear somebody talk about Growth Hacks, just mentally translate it in your mind to "bullshit," because what we tell them is the way to make your startup grow is to make something that users really love, and then tell them about it. So that's what you have to do: that's Growth Hacks right there.

There's an older, more colorful version of the same warning, delivered by Sam Altman by way of the rapper 50 Cent, describing how he thought about the Vitamin Water business he was involved in:

Most people think first of what they want to express or make, then find the audience for their idea. You must work the opposite angle, thinking first of the public. You need to keep your^[^correction] focus on their changing needs, the trends that are washing through them. Beginning with their demand, you create the appropriate supply.

[^correction]: The original transcript reads "you focus," almost certainly a transcription slip for "your focus" — corrected here rather than reproduced as an error.

Altman's own read on why this matters: this is the single most common thing missing from Y Combinator applications. Founders build the thing they want to build, then go looking for people to want it — the opposite of the order that actually works. Business school doesn't fix this either, in Graham's blunt assessment: it teaches management, which is a problem you only get to have once you're already successful. What you need before that is product and market instinct, and the only reliable way to get that is direct exposure to the people you're building for — a theme this book returns to at length in Part Three.

Turning your brain into the kind that has ideas

If deliberately hunting for a startup idea tends to produce bad-but-plausible-sounding ideas — ideas that sound reasonable enough to fool you and everyone around you before you waste a year on them — the alternative Graham proposes sounds almost too simple: don't consciously try. Instead, change what your brain pays attention to, so that ideas show up as a side effect of things you were doing anyway. His formula:

1. Learn about a lot of things that genuinely matter.
2. Work on problems that interest you.
3. Do it with people you like and respect.

That third point solves two problems in one motion, since it's also how you find cofounders — a subject Chapter 4 covers in full.

The track record backs this up in an odd way: Yahoo, Google, Facebook, and Apple all started as side projects that weren't meant to become companies. None of their founders set out that morning to start a business. Graham's explanation: the very best ideas are such outliers that a conscious, deliberate search for "a startup idea" would talk you out of them before you'd even tried them. Airbnb is a good example of how broad "learn about things that matter" can be — Brian Chesky and Joe Gebbia weren't technology experts. They went to art school. What they were good at was design and getting projects organized and done. The domain doesn't have to be programming, as long as it genuinely stretches you.

Graham has an image for what this actually looks like in practice: think of technology as a fractal stain, spreading outward in every direction at once. Every point on its edge is an interesting problem, waiting for someone to notice it. Move yourself to that edge, and problems that look strange or premature to everyone else start to look obvious to you.

Graham's phrase for the same state, borrowed from Paul Buchheit, is to "live in the future" — get yourself to wherever technology is moving fastest, and ideas that look strangely obvious to you will look prescient to everyone else. His example, told to him secondhand by two of his own friends, Robert and Trevor: a fellow graduate student of theirs at Harvard in the mid-1990s wrote his own voice-over-IP software, not to start a company, but because he wanted to talk to his girlfriend in Taiwan without paying long-distance rates, and he happened to already understand networks well enough that turning voice into packets and sending them over the internet seemed like the obvious thing to do. He never tried to commercialize it. That's what Graham means when he says the best startups tend to happen this way. If you're looking for a two-word summary of his whole argument, he gives one: *just learn*.

Common Mistakes

- **Believing the pitch has to sound big from day one.** The opposite is usually true — ideas that sound modest or even a little embarrassing at first are a good sign, not a bad one.

- **Chasing a market because of its size today, not its growth rate.** A large, slow market is usually already crowded. A small, fast-growing one usually isn't yet.
- **Copying an existing company with a small, made-up differentiator.** "Like X, but with better design" or "like Y, for people who like red wine instead" is a pattern that tends to fail — the best ideas differ from what exists in one important way (Google: a search engine that actually worked well, with none of the extra portal features) or are something genuinely new (SpaceX).
- **Trying to consciously invent a "startup idea" instead of noticing one.** This produces plausible-sounding bad ideas, because your conscious mind is optimizing for "sounds like a good pitch" rather than "is actually true."
- **Keeping the idea secret out of fear someone will steal it.** If the idea is genuinely good, it will sound bad enough that most people won't want to steal it. Secrecy is usually a symptom of an idea that only sounds good, which is itself a warning sign.
- **Not thinking about the customer's demand before building the product.** This is, in Altman's own account, the most common problem in weak Y Combinator applications: founders who built what they wanted to build first, and are only now looking for people who might want it.
- **Needing several sentences and a diagram to explain the idea.** If the explanation is long, that's usually a sign the idea itself is too complicated — not that you haven't found the right words yet.



Tradeoffs and Edge Cases

Doesn't "build what you need yourself" rule out entire categories of good businesses? Not entirely, but it does mean you should be honest about the disadvantage you're taking on if you build for someone else's problem. Paul Graham makes an important exception in the same lecture: a non-technical founder who is a genuine domain expert can be just as valuable as the person needing the product themselves. His example is Uber — if one founder is an expert in the limo business and the other is technical, the non-technical founder would actually end up doing most of the work: recruiting drivers and handling everything else specific to running that kind of operation, while the technical

founder writes what amounts to less than half of what the company actually needs, the app itself. The domain expert isn't a weaker version of a technical cofounder. In a business this specific, they may be the one carrying more of the load. The rule isn't "you must need it yourself." It's "someone on the founding team must deeply understand the people you're building for, one way or another" — and in a sufficiently specialized business, that person's expertise can matter more than anyone's ability to write code.

When does a side project actually become a startup? This matters because Chapter 1's whole test — "you can't not do it" — is hard to apply if you can't tell whether what you're doing counts yet. Graham's answer, when a student asked exactly this question: you will know, because it will start taking over an alarming percentage of your life. If you're neglecting your classes or your job because you can't stop working on the thing, that's usually the signal, not some external milestone like a launch date or a user count.

Is market size ever the right thing to optimize for early? Occasionally, yes — some categories (heavy infrastructure, certain hardware and biotech businesses) genuinely do require significant scale before they work at all, and Part Five of this book covers how capital-intensive businesses need to plan differently. But for the large majority of software startups this book focuses on, starting in the center of Figure 2.1 rather than the outer ring is the right default, and treating your own idea as an exception to it is worth being suspicious of.



Applying the Test: Why Now?

Take an idea and try to answer Sequoia's question honestly: *why now, and why not two years ago, and why not in two more years?*

A weak answer sounds like this: "There's a huge market for X, and nobody's built a good version of it yet." That's not an answer — it doesn't explain why the market has stayed unaddressed until this exact moment, which should make you suspicious that maybe there's a reason nobody's built it (regulation, a missing piece of technology, a cost that hasn't come down yet).

A strong answer names a specific, external, dated shift: a cost that dropped below a threshold, a law that changed, a platform that only just released the API you need, a behavior shift that only just became common enough to build on. Dropbox's version: storage and bandwidth had gotten cheap enough, and broadband use common enough, that syncing files across devices finally made sense as a mass-market product rather than an enterprise IT product. The AI-native companies in this chapter's earlier sidebar have their own version: models had just crossed a usability threshold, and inference had just gotten cheap enough, to make products viable that would have been too expensive or too unreliable three years earlier.

If you can't point to a specific, dated, external reason your idea works now and didn't work before, that's not necessarily fatal — but it's a gap worth closing before you spend a decade on the answer.

Chapter Summary

- The outcome of a startup is roughly idea × product × execution × team × luck — the idea is only one factor, but it's the one this chapter and the next one cover.
- "Idea," in the broad sense this book uses, includes market size and growth, your growth strategy, and how hard the business will be to copy — not just the product itself.
- Good ideas are supposed to sound bad at first and be explainable in one plain sentence. If everyone already agreed it was a good idea, it would already be crowded — and if it takes three sentences to explain, that's usually a sign the idea is too complicated, not that you haven't found the right words.
- You can change almost everything about a startup except the market — which is why evaluating the market's growth rate matters more than its current size, and why students especially should trust their own instincts about which markets are about to move.
- Sequoia's "why now?" question is a useful filter: a strong idea usually has a specific, dated, external reason it works now and didn't work two years ago.

- Start in a small market you can dominate completely, and expand outward in stages — don't start with the largest version of the market on day one.
- The strongest founders build what they need themselves, or — failing that — get as close as possible to the people who do, including through deep domain expertise rather than personal need.
- There is no trick to finding a good idea. The advice that works is blunt: learn about things that matter, work on what interests you, and do it with people you respect — "just learn."



Exercises

1. **Recall:** In your own words, explain why a good startup idea is expected to sound bad at first, and why that isn't just a coincidence.
2. **Apply:** Take an idea you've considered. Write down your honest answer to "why now" — a specific, dated, external reason this idea works today and didn't work two years ago. If you can't find one, what would you need to learn to find one?
3. **Apply:** Using Figure 2.1, describe the smallest possible market you could dominate completely with your idea, and the two expansion rings you'd move through after that.
4. **Confront a counterexample:** Paul Graham argues the best ideas start as side projects nobody meant to turn into a company. But plenty of successful startups — including several covered later in this book — started with founders deliberately setting out to solve a specific business problem from day one, not stumbling into it. Does that contradict Graham's claim, or is there a way to reconcile deliberate, research-driven idea generation with his "just learn" advice?



Further Reading

- Paul Graham's full essay "How to Get Startup Ideas" goes well beyond what his CS183B lecture had time to cover, and is the direct source for the "live in the future"

and "don't consciously search" arguments in this chapter.

- Peter Thiel's *Zero to One* expands on the idea that good businesses are hard to copy — the subject of the next chapter — and is worth reading alongside it.
- Marc Andreessen's essay "Why Software Is Eating the World" gives the fuller argument behind the one-line quote used in this chapter's "why now" discussion.



Chapter 3 — Monopoly, Not Competition

In 2012, U.S. airlines brought in about \$195 billion in domestic revenue. Google brought in a little over \$50 billion worldwide. By revenue alone, airlines look nearly four times more important than Google. But look at profit instead of revenue, and the picture flips completely. Airlines have been flying passengers for a hundred years, and across that entire century, their cumulative profit is close to zero. Companies in the industry make money for a while, go bankrupt, get bought or restructured, and the cycle repeats. Google, over a much shorter life, has piled up enormous profit margins. The airline industry's total stock market value is roughly a quarter of Google's alone — one company, not even one industry.

Peter Thiel, who co-founded PayPal and Palantir and built one of Silicon Valley's most influential venture firms, uses this comparison to make a single point that this chapter is built around: **being big is not the same as being valuable**. What separates airlines from Google isn't the size of the market they're in. It's whether they have a monopoly. This chapter is about what that word actually means, why nearly every successful startup is trying — quietly — to build one, and why the businesses that admit this out loud are rare.



The Core Idea: Two Numbers, and Only Two Kinds of Business

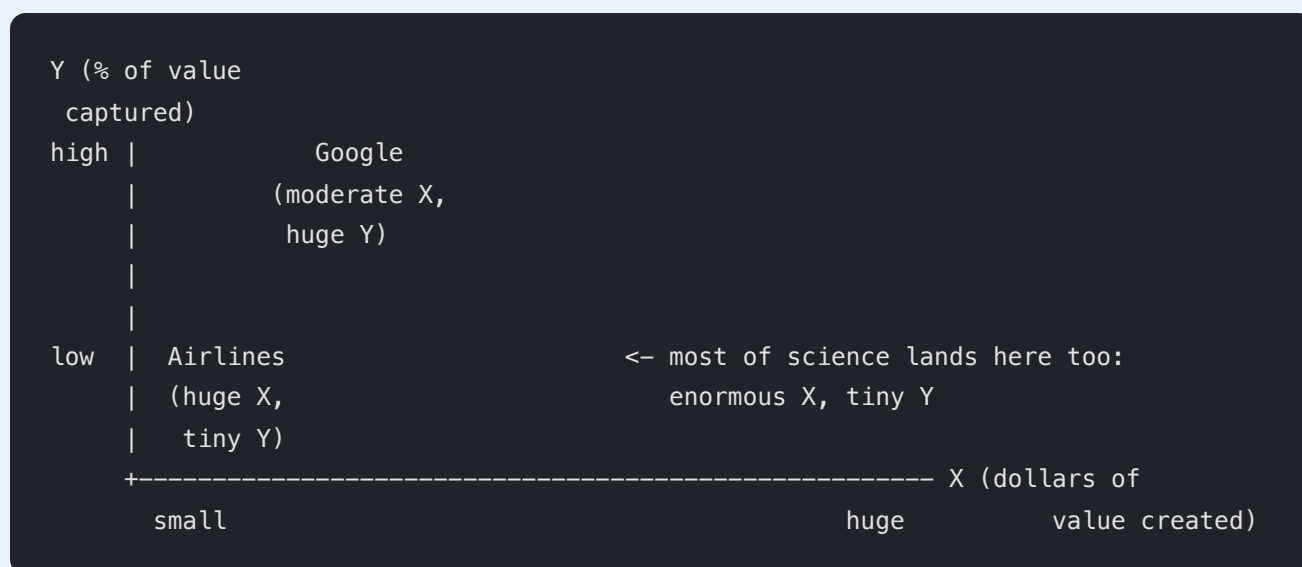
Thiel starts with a simple formula for how valuable a company is. Two things have to be true:

*A company creates **X** dollars of value for the world, and captures **Y** percent of that value for itself.*

The critical point is that X and Y are **independent**. A company can create an enormous amount of value (a large X) and still capture almost none of it (a small Y) — that's the airline industry, and as this chapter will get to later, it's also most of science. Or a

company can create a comparatively modest amount of value and still capture a large slice of it, which is enough to make it worth more than an industry many times its size. Google does both: it created a lot of value, and it captured an unusually large share of it. Value creation alone tells you almost nothing about a company's worth. Capture is the other half of the equation, and it's the half most people forget to ask about.

Figure 3.1 — Value creation and value capture are independent axes



A company in the bottom-right (huge X, tiny Y) can matter enormously to the world and still be a terrible business to own. A company in the top-left (modest X, high Y) can look small by comparison and still be worth more. The two axes have to be reasoned about separately — neither one predicts the other.

Thiel's own title for this argument, delivered with a straight face to a room of ambitious Stanford students, was blunter than anything in this chapter so far: **"competition is for losers."**

Thiel's next claim is more provocative: there are really only two kinds of businesses. There's **perfect competition**, where many companies sell similar things, nobody can charge much above cost, and — this is the part people miss — almost nobody makes money. And there's **monopoly**, a business with no serious competitor doing what it does, which is stable, can plan years ahead, and usually makes real money. "There is shockingly little that is in between," in Thiel's words. Most businesses aren't a blend of the two. They're much closer to one end of the spectrum than people assume, and the reason that's hard to see is the subject of the next section.

Terminology note: *the qualities that make a monopoly durable — the things that make it hard for a competitor to attack — are often bundled today under the word "moat," a term popularized by Warren Buffett and widely used across Silicon Valley. This chapter will get to what actually builds a moat, but it's worth knowing the word, since you'll see it used everywhere from here on in the book.*



Why It's True: The Lies, the Small Start, and the Characteristics That Last

Everyone lies about which one they are, in opposite directions

If monopoly and competition were easy to tell apart, this chapter would be much shorter. They're hard to tell apart because of a specific, structural reason: **companies have an incentive to lie about which one they are, and the lie runs in opposite directions depending on which one they actually are.**

A real monopoly has every reason to hide it. Admitting you dominate a market invites regulators, journalists, and antitrust lawyers. So a monopoly describes its market as enormous and crowded with competition — the opposite of the truth.

A company stuck in real competition has the opposite incentive. Nobody wants to invest in a business with no way to make money, so a struggling, undifferentiated company describes its market as narrow and unique — again, the opposite of the truth.

Thiel gives a blunt example of the second kind of lie. Restaurants are famously bad businesses — too much competition, thin margins, nobody keeps their capital for long — so a new restaurant owner looking for investors won't say "I'm opening a restaurant." They say something like, "We're the only British restaurant in Palo Alto." That sounds like a real, defensible niche. It's actually a meaningless one: nothing stops a customer from driving to Mountain View, and there probably isn't a large population of people who eat only British food. Thiel's phrase for this kind of manufactured niche: "the something of

somewhere is really mostly just the nothing of nowhere — it's like the Stanford of North Dakota." One of a kind, maybe. Not actually Stanford.

Hollywood pitches work the same way. Thiel's parody of the genre: a movie about "a college football star who joins an elite group of hackers to catch the shark that killed his friend." No such movie exists, which is the point — the pitch is built by stacking together enough narrow, specific-sounding categories that it feels like its own unique thing, when the honest question is simpler: is this actually a new category, or is it just another movie, competing in an industry so brutally competitive that almost nobody ever makes real money making films?

The same move happens with startup pitches: sharing, mobile, and social, mashed together into a single business, describing the *intersection* of several huge categories as if that intersection were both real and valuable. Sometimes it is. Often it's a warning sign disguised as a differentiator.

Now look at the opposite lie, from an actual monopoly. Google, which had roughly two-thirds of the search market at the time Thiel gave this lecture, almost never describes itself as a search engine. Depending on which regulator or reporter is asking, it describes itself as:

- an **advertising company** — in which case search advertising, a real but modest \$17 billion, is a small fraction (roughly 3.5%) of the ~\$500 billion global advertising market
- a **technology company** — in which case it's "competing" with car companies on self-driving, Apple on phones, Facebook on social, Microsoft on office software, and Amazon on cloud computing, all at once

Both descriptions describe the *union* of many huge markets rather than the actual market Google dominates. The real number — a company with roughly two-thirds share of search — sounds like exactly what it is: a much bigger, more durable monopoly than Microsoft ever had on operating systems in the 1990s. That's precisely why Google avoids saying it.

The pattern to remember: **a non-monopoly describes its market as an intersection, to sound uniquely differentiated. A monopoly describes its market as a union, to sound like just one competitor among many.** Once you know to look for this, you'll notice it in almost every pitch and every regulatory filing you read.

One piece of outside evidence for how real this pattern is: Apple, Google, Microsoft, and Amazon have spent years accumulating cash faster than they know what to do with it. That kind of profit margin, sustained over years, is close to impossible in real competition. It's a signature of monopoly-like economics, whatever the company's public description of its market says.

Build small, take over completely, then expand

If monopolies are valuable and competition mostly isn't, the obvious next question is how a startup — which starts with nothing — ever gets to be one. Thiel's answer is the same one Sam Altman reaches independently in Chapter 2, which is worth pausing on: two people teaching different parts of the same course, coming from different backgrounds (Altman as an investor and founder, Thiel as an investor and founder from an earlier generation of companies), land on an identical structure without citing each other. That's a decent sign the structure is actually true rather than just a fashionable opinion.

The structure: start in a market small enough that you can take almost all of it, then expand outward in the same concentric rings from Figure 2.1. Thiel's examples put real numbers on this pattern:

- **Amazon** started as an online bookstore — a better bookstore than any physical competitor, since it could offer far more titles — before expanding into every other category of e-commerce.
- **eBay** started with people trading Pez dispensers, moved to Beanie Babies, and only later became the platform for practically any category of good sold at auction.
- **PayPal** started with eBay's "power sellers," a group of only about 20,000 people. Thiel and his cofounders reached roughly 25–30% penetration of that tiny market within two to three months of launch, and used that foothold to expand from there.
- **Facebook** started with about 10,000 people at Harvard, and reached roughly 60% of that market within ten days.

In every case, a business-school-style market-size analysis, done on day one, would have concluded the company was worthless: a market of 20,000 eBay sellers, or 10,000 students at a single university, looks too small to matter. What that analysis misses is the concentric expansion — the fact that dominating a tiny market completely is what makes the next, larger ring reachable at all.

The clean tech boom of 2005–2008 is Thiel's cautionary example of doing the opposite. Nearly every clean tech pitch deck from that period opened with the total size of the global energy market — trillions of dollars. Starting there put every one of those companies in the position of "a minnow in a vast ocean," competing against more rivals than they could even count, with no way to carve out the kind of dominant foothold that Amazon or PayPal built early on. Bigger, in this case, was worse, not better — it's the same mistake Chapter 2 already warned against (starting at the outer ring of Figure 2.1 instead of the center), just at a larger scale and with more money burned proving it. Thiel's blunter way of putting the same warning: you don't want to be the fourth online pet food company, or the tenth solar panel company, or the hundredth restaurant in Palo Alto. The restaurant industry, taken as a whole, is a trillion-dollar market — which means a market-size slide alone would make restaurants look like a fantastic business to get into, and any restaurant owner who has actually tried to make money running one could tell you exactly why that conclusion is wrong.

What actually makes a monopoly last

Getting small and dominating it is only half the problem. The other half is durability — staying dominant once you've gotten there. Thiel is explicit that there's no single formula for this — the closest he comes to one is an inversion of the opening line of *Anna Karenina*: Tolstoy's line is that all happy families are alike, and each unhappy family is unhappy in its own way. Thiel's business version runs the other direction — every happy (durable, monopoly) company is different, because each one is doing something genuinely unique; every unhappy company looks the same, because they all failed to escape the sameness that defines real competition. With that said, Thiel does name four characteristics that build a durable monopoly, and it's worth being honest that no single one of them is required; the best companies usually have more than one.

Characteristic	What it means	Example
Proprietary technology	Something meaningfully better than the next-best alternative — Thiel's rough rule of thumb is at least 10 times better on some important dimension	PayPal moved money roughly 10x faster than the checks it replaced; the iPhone worked as a smartphone in a way nothing before it really did

Characteristic	What it means	Example
Network effects	The product gets more valuable to each user as more people join	Hard to start — there has to be a reason the first users show up before the network exists — but once it takes off, it tends to get stronger, not weaker, over time
Economies of scale	High fixed costs, very low costs to serve each additional customer	Software is unusually good at this: the marginal cost of one more user is close to zero, so a company that gets ahead can pull away fast
Branding	A perception that gets lodged in people's minds	Real, but Thiel is candid that he doesn't fully understand how it works, and avoids investing in a company on branding alone

Two of these deserve a further point. **Proprietary technology** needs to keep winning, not just win once — disk-drive manufacturers in the 1980s got overtaken by the next better drive roughly every two years, in a cycle that ran for the better part of fifteen years straight. Fantastic for consumers, who got a dramatically better product every couple of years. Terrible for the manufacturers themselves, since none of them ever got to hold the advantage long enough to become durably profitable from it. **Economies of scale** is where software has a genuine structural edge over almost every other kind of business: zero marginal cost combined with the ability to reach users instantly, worldwide, means a software company that gets a real lead can turn that lead into a durable monopoly faster than almost any other kind of company in history.

Applying the four characteristics to real companies

Asked directly, after his lecture, which of the four characteristics actually applies to Google, Thiel's answer touches all four at once: network effects through its advertising network, proprietary technology through the original PageRank algorithm (though he considers this edge weaker today than it once was), economies of scale from the sheer

infrastructure needed to index the web, and by now, real brand strength too — in his own words, "maybe three and a half out of four."

A second, smaller example makes the small-market strategy concrete in a different industry. Palantir, the data-analysis company Thiel co-founded, didn't start by trying to serve every possible customer. It started with a single, deliberately narrow submarket — the intelligence community — combined with a genuinely different proprietary approach: building tools around human-computer collaboration rather than the more common paradigm of trying to fully automate analysis and remove the human from the loop entirely. Small market, real proprietary angle — the same two-part recipe as PayPal's eBay power sellers or Amazon's original bookstore, just in a completely different industry.

| Last mover, not first mover

Here's the part of Thiel's argument that inverts a piece of standard startup wisdom. Everyone talks about the value of being a "first mover." Thiel argues you should be aiming to be the **last** mover instead — the final company to enter a category and the one that's still standing at the end of it. Microsoft was the last operating system that mattered, for decades. Google was the last search engine that mattered. Whether Facebook turns out to be the last major social network is, in Thiel's framing, the real test of whether it was ever a monopoly at all.

There's a chess analogy Thiel likes for this: the player with white pieces (the first mover) gets a real but small edge — something like a third of a pawn's worth of advantage. It's real, but it's not decisive. World chess champion José Raúl Capablanca's advice was the opposite of "move first and don't look back": "You must begin by studying the endgame." The question that matters for a monopoly isn't who got there first. It's who's still there in fifteen years.

This connects to something concrete and checkable: where a company's *value* actually comes from. Thiel did this exercise for PayPal in March 2001, when the company was 27 months old, growing at 100% a year, and using a 30% discount rate on future cash flows. The result: roughly three-quarters of PayPal's value at that moment came from cash flows the model expected in 2011 and beyond — a full decade out. Run the same kind of analysis on Airbnb, Twitter, Facebook, or nearly any other fast-growing company to come out of Y Combinator, Thiel argues, and the math tells the same story at an even more extreme ratio: something like 85% of the value sits in cash flows expected in 2024 and

beyond — more than a decade past the year he gave this lecture. The overwhelming majority of the value sits far in the future, not in this year's growth number. That has an uncomfortable implication for how outsiders judge these companies: **growth is the thing everyone can measure right now, so it gets over-weighted. Whether the company will still be around and still be winning in ten years is what actually determines the value, and it's much harder to measure — which is exactly why it gets under-weighted.**

The history of value that got captured, and value that didn't

Zoom out further, and the X/Y framing from the start of this chapter explains a genuine pattern across roughly 300 years of technological progress — steam engines, railways, the telephone, refrigeration, household appliances, the computer, aviation — and a closely related 250-year pattern in science: most of it created enormous value and captured almost none.

Science, as an institution, is close to a hundred-percent example of this. The smartest physicist of the twentieth century works out special and general relativity — foundational to enormous amounts of value created afterward — and doesn't become a millionaire, let alone a billionaire, from either discovery. The railroads created enormous value for the societies they connected and mostly went bankrupt doing it, because there were too many competing railroads for any one of them to keep the profit. The Wright brothers flew the first airplane and made essentially no money from having done it. Even the first Industrial Revolution's textile mills, which delivered something like 5–7% efficiency gains *every single year* for sixty or seventy years — a staggering, compounding run of real technological progress — didn't make the manufacturers who ran them rich either. There were too many of them, competing the gains away as fast as they appeared. Even the landed aristocracy, not the industrialists, still held most of Britain's wealth by 1850.

By Thiel's account, there are really only two categories in that long stretch of history where people captured serious value from genuine innovation, rather than just creating it for everyone else. The first is **complex, vertically integrated monopolies** — Ford, Standard Oil — where the advantage came not from one single breakthrough but from coordinating an enormous number of moving pieces better than anyone else could. Thiel's modern example is Tesla and SpaceX: neither company's edge, in his account, comes from one single 10x technical breakthrough. It comes from vertical integration done more

thoroughly than competitors — Tesla controlling its own distribution instead of ceding margin to dealers the way the rest of the auto industry does, or SpaceX bringing subcontracting in-house instead of leaving itself exposed to suppliers who could otherwise charge monopoly prices on critical parts.

The second category is **software**, for the structural reasons already covered above: extreme economies of scale, and the ability to reach a global market almost instantly, which lets a small starting market get dominated and expanded from before competitors can catch up. This is, in Thiel's telling, the actual explanation for why Silicon Valley has been so financially successful: not that software people are smarter, but that software as a category is unusually well-suited to building the kind of monopoly this whole chapter has been describing.

Why we're drawn to competition anyway

If avoiding competition is clearly the better outcome, why do so many founders — and whole industries — end up competing instead of monopolizing? Thiel's answer isn't just practical. It's psychological, and he argues it runs deeper than most people are willing to admit.

The word "ape" has meant both "primate" and "to imitate" since at least Shakespeare's time. That's not a coincidence of English vocabulary. It's a hint about human nature: we are, at some level, deeply imitative, and we tend to treat other people wanting the same thing as validation rather than as a warning sign. Thiel's example: roughly twenty thousand people move to Los Angeles every year hoping to become a movie star. About twenty of them make it. Ask any one of those twenty thousand whether so many others chasing the identical goal makes the goal feel more attractive or less, and most say more — proof, in their minds, that the goal must be worth having, since so many people agree.

This gets built into people well before they ever start a company, through an education system that runs almost entirely as a tournament: competing for grades, for college admission, for the right internship, for the right first job. Henry Kissinger's description of his own Harvard faculty is the sharpest version of the same point Thiel has found: "The battles were so ferocious because the stakes were so small." That sounds, on the surface, like a description of people behaving irrationally. Thiel's reading is more specific — when the real differences between competitors are genuinely small, people fight *harder*, not less hard, to manufacture a difference that feels real.

Thiel offers his own life as the personal version of the same trap. He describes himself as "hypertracked" from early adolescence: a classmate wrote in his eighth-grade yearbook that she knew he'd get into Stanford. Four years later, he did. From there he went to Stanford Law School, and from Stanford Law into a top law firm in New York — exactly the job everyone on the outside of it wanted, and exactly the job everyone already inside it wanted out of. He left after seven months and three days. A colleague told him on his way out, "I had no idea it was possible to escape from Alcatraz" — and then pointed out that the door had been unlocked the whole time. All it had ever taken was walking out the front door and not coming back. So much of that colleague's own identity, and the identities of the people who stayed, had gotten wrapped up in winning a competition that nobody had stopped to ask whether the prize was actually worth having.

None of this means competition doesn't make you better at whatever you're competing at — Thiel is explicit that it does. Comparing yourself against the people around you is a real way to improve. The cost is that it also crowds out a harder, more important question: whether the thing you're getting better at is worth getting better at in the first place. Thiel's closing image is worth carrying past this chapter: don't always go through the tiny door that everyone else is rushing to get through. Go around the corner, and look for the vast gate that nobody else is taking.

Common Mistakes

- **Sizing your market by its current, obvious total instead of the smallest slice you can dominate first.** This is the same mistake as Chapter 2's "starting at the outer ring," restated here with the clean tech bubble as its most expensive historical example.
- **Describing your market as an "intersection" to sound unique, when it's actually small and fake.** ("The only British restaurant in Palo Alto.") If your market description only makes sense with several specific adjectives stacked together, be suspicious of it.
- **Chasing "first mover" status instead of asking whether you'll be the last one standing.** Being first is a small, real advantage. Being durable is the one that actually determines value.

- **Treating growth and durability as the same thing.** A company can be growing fast and still be in a business that's fundamentally competitive, with no lasting advantage — which means the growth won't hold once competitors catch up.
- **Investing in — or building toward — a single 10x technical breakthrough as the whole strategy, with no plan for what happens once someone else matches it.** Disk-drive manufacturers in the 1980s did this repeatedly, improved the world enormously, and made almost no money doing it.



Tradeoffs and Edge Cases

Doesn't "last mover" contradict itself — Google and Facebook weren't literally first, so how are they "last"? This is worth resolving directly, because it's a real tension in Thiel's own language. Google wasn't the first search engine — there were several before it. Facebook wasn't the first social network; Reid Hoffman had already started one, called SocialNet, back in 1997. What made each of them "last" wasn't literal chronological order. It was being the first to nail one specific, important dimension well enough that no one after them displaced it: Google was the first to use an automated, algorithmic approach (PageRank) that was an order of magnitude better than manually-curated search. Facebook was the first social network built around real identity rather than invented personas. "Last mover" describes durability along the dimension that actually mattered, not who launched a website first.

Does this chapter's skepticism of "lean startup" methodology contradict the rest of this book's emphasis on talking to users? It looks that way, and it's worth resolving directly rather than leaving the contradiction sitting unaddressed, since Chapters 7, 8, and 16 all lean heavily on user interviews and rapid iteration as core practices. Thiel's actual claim is narrower than "don't talk to users." His own words on this, from the same lecture: founders should spend more effort trying to have "a virtual ESP link with the public" — a direct, almost intuitive read on what people need — and less effort treating iteration itself as the answer. His argument is that the *degree* of iteration and customer-survey-driven development some founders rely on can become a substitute for genuine insight and differentiation, rather than a source of it. Notice that PayPal and Palantir — his own companies — both started in a deliberately narrow market and both had a specific

technical or strategic insight going in; they didn't arrive at their strategy by surveying users into it. The reconciliation: talking to users (the subject of Part Three of this book) is how you refine and validate an idea you already have real conviction about. It's not a substitute for having that conviction in the first place. A founder with no differentiated insight who iterates rapidly on user feedback can still end up with a well-executed, undifferentiated product — competing, not monopolizing.

Is Thiel's four-characteristics model the only way to build a durable business? No — Thiel is explicit that there's no single formula, and that the vertically-integrated model (Ford, Tesla, SpaceX) doesn't fit neatly into "proprietary technology, network effects, economies of scale, branding" at all. It's really a fifth path: winning through superior coordination of many ordinary pieces, rather than through one exceptional piece. This matters if your business involves physical operations, logistics, or manufacturing, where the software-style economies of scale don't automatically apply.



Applying the Test: Is This a Monopoly, or a Competitive Business Wearing a Costume?

Take an idea, and try describing its market honestly in both directions.

First, describe it the way a struggling competitor would describe theirs: as narrowly and specifically as you can, stacking on adjectives until it sounds unique. If that description only sounds meaningful because of how many qualifiers you had to add, you've probably found an "intersection" lie rather than a real, defensible position.

Second, describe it the way an actual monopoly downplays itself: as the union of several much larger categories, so the real dominant position gets lost in a crowd of imagined competitors. If your honest internal sense of the business — the one you'd only say out loud to a cofounder — sounds a lot closer to the second description than the first, you may actually be closer to a real monopoly than your public pitch admits.

The test that matters, per Thiel's own answer to this exact question in a Q&A after his lecture: ignore the narrative in both directions, and ask what the *actual* market is — not the story that makes you sound uniquely interesting, and not the story that makes you

sound reassuringly unthreatening. People, in his words, "have incentives to powerfully distort these things," in both directions at once, and the only way through is to keep asking what's real underneath the story.

Chapter Summary

- A company's value depends on two independent things: how much value it creates (X) and what percentage of that value it captures (Y). Being big (a large X) doesn't automatically mean being valuable (a large X times Y).
- There are really only two kinds of businesses: monopolies, which are rare, stable, and profitable, and competitive businesses, where almost nobody makes money. There is very little in between.
- Monopolies downplay themselves to avoid regulators; competitive businesses inflate themselves to attract investors. The lies run in opposite directions, which is why the difference is hard to see from the outside.
- The path to monopoly starts small — a market so tiny it looks worthless on a spreadsheet — and expands outward in concentric rings, the same structure Chapter 2 introduced from a completely different lecture.
- Durable monopolies usually have at least one of: proprietary technology roughly 10x better than the alternative, network effects, extreme economies of scale, or branding — and software is unusually well-suited to several of these at once.
- Aim to be the *last* mover in a category, not the first — durability, not speed to market, is what determines whether most of a company's value (often concentrated a decade or more in the future) actually gets realized.
- Across 250 years of innovation, most value created was never captured by the people who created it — science, railroads, and early industrial manufacturing all show this. The rare exceptions are complex vertically-integrated businesses and software.



Exercises

1. **Recall:** In your own words, explain why a monopoly and a struggling competitive business would describe their markets in exactly opposite ways, even though both are lying.
2. **Apply:** Take an idea you're evaluating. Write its market description twice — once the way a monopoly would understate it, and once the way a weak competitor would overstate it. Which one is closer to true?
3. **Apply:** Using the characteristics table in this chapter, identify which of proprietary technology, network effects, economies of scale, or branding your idea could plausibly have. If the honest answer is "none yet," what would need to be true in a year for that to change?
4. **Confront a counterexample:** This chapter argues that heavy reliance on user surveys and rapid iteration can substitute for genuine differentiated insight rather than produce it. But Part Three of this book (particularly Chapters 7 and 8) treats direct user feedback as close to essential. Reconcile these two claims in your own words — what is each one actually arguing, and where exactly is the line between them?



Further Reading

- Peter Thiel's book *Zero to One*, co-written with Blake Masters, expands the entire argument of this chapter — especially the X/Y framing and the last-mover argument — well beyond what a single lecture could cover.
- Warren Buffett's shareholder letters are the original source of the "moat" terminology this chapter introduces, applied originally to public-market investing rather than startups.

- Chapter 2 of this book covers the concentric-expansion structure from Sam Altman's independent version of the same argument — worth rereading side by side with this chapter's version from Thiel.



Chapter 4 — Choosing a Cofounder

One Y Combinator batch, Sam Altman recalls, had about seventy-five companies. Nine of them added a new cofounder sometime between their interview and the actual start of the program — someone who joined the team late, after the selection process was already done. All nine of those nine companies fell apart within the following year.

Not most of them. All of them.

That number should reframe how you think about this decision. Most first-time founders treat choosing a cofounder as something close to a social accident — you meet someone who also wants to start a company, you both seem into it, you start a company together. Altman's blunt assessment of this pattern: "You would never hire someone like this, and yet people are willing to choose their business partners this way." This chapter is about why that casualness is dangerous, and what to actually look for instead.



The Core Idea: You Can Trust Your Instincts About People

Chapter 1 made a point that's worth restating precisely, because this chapter is about to say something that sounds like its opposite: you cannot reliably know in advance whether *you* are capable of building a startup. Confidence going in doesn't predict outcomes.

Paul Graham makes a related but distinct claim about *other people*, and it points the opposite direction: your instincts about people are actually trustworthy, more trustworthy than most founders give them credit for. His observation, from watching founders make this mistake repeatedly: they meet someone who seems impressive, feel some private unease about them anyway, and override that unease because the person's credentials or confidence are hard to argue with. Then, months later, when the relationship falls apart, they say some version of: "I knew there was something wrong about that guy, but I ignored it because he seemed so impressive."

Graham's advice is almost embarrassingly simple: pick a cofounder the way you'd pick a friend. Not a colleague, not a business contact — a friend. Work with someone you would generally like and respect, and whom you've known long enough to actually be sure of, because plenty of people are extremely good at seeming likable for a while. The real test only arrives once your interests are in conflict, which is exactly the situation a new startup manufactures constantly.

These two claims — don't trust your self-assessment (Chapter 1), do trust your read on other people (this chapter) — aren't in tension. They're about different kinds of uncertainty. What you're capable of becoming is genuinely unknowable in advance, because starting a company changes you in ways you can't simulate ahead of time. Who someone already *is* — whether they're trustworthy, whether they follow through, whether you'd actually want to be around them during a crisis — is much more visible, if you're willing to look honestly instead of being dazzled.



Why It's True: The Data, the Metaphor, and the Wrong Way to Meet Someone

The choice matters more than almost any other you'll make

Altman is direct about the stakes: cofounder relationships are among the most important in the entire company, and cofounder blowups are Y Combinator's single most common cause of early startup death. That's a strong claim to make about one relationship, in a book full of chapters about products, growth, and fundraising — but the data backs it up. Looking across the most valuable companies YC has funded, almost all of them have at least two founders. Y Combinator funds solo founders at roughly a tenth the rate of teams. The ranking, in Altman's own words, goes: a cofounder you already know well is best. A solo founder is acceptable — worse, but survivable. A cofounder you barely know, chosen the way many students choose one, is the worst option on the list, worse than having no cofounder at all.

That last point is worth sitting with, because it cuts against instinct. It feels safer to have a partner than to go alone. The data says a bad partner is more dangerous than no partner.

Reid Hoffman, who co-founded LinkedIn and has invested in dozens of startups since, frames the reason teams beat solo founders in a way worth stating directly: it explodes the myth of the "super founder." Popular imagination pictures a founder — the Steve Jobs, Bill Gates, Elon Musk, Mark Zuckerberg, Jeff Bezos image — as someone who can do everything — product, fundraising, sales, management, strategy — at a world-class level, alone: a kind of panopticon of skills, watching and excelling at every function of the company at once. Hoffman is candid that he once bought into this image himself, recalling an article early in his own career that called Bill Gates "smarter than Einstein" — a claim, he points out, that Gates himself probably wouldn't want standing next to his name, and a symptom of exactly this fantasy: the founder as someone who can jump over tall buildings in a single bound. In reality, nobody has that full range of skills, and a company that depends on one person having it is fragile in a way that isn't obvious until something goes wrong in the exact area that person is weak. Two or three founders, chosen well, let you cover each other's gaps and split a genuinely superhuman set of demands across real, ordinary humans. This is also, not coincidentally, the same reason Chapter 1 spent so much time debunking the myth of the founder as a lone, glamorous decision-maker — this chapter's version of that myth is simply the same fantasy applied to a team of one instead of a team of several.

The James Bond test

Y Combinator has an internal phrase for the kind of cofounder you want: "relentlessly resourceful." Paul Graham popularized a more memorable, more specific version of the same idea at YC's own orientation sessions, and Altman has kept using it since: you're looking for someone who behaves like James Bond.

Not because you need an expert marksman. Because the archetype captures a specific cluster of traits better than a checklist would: unflappable under pressure, tough, knows what to do in almost any situation, decisive, creative, ready for anything. Altman's framing: you need someone who behaves this way *more* than you need someone who happens to be a deep expert in one particular domain. Domain expertise can be learned or hired in. Being unflappable under real pressure mostly can't.

Alongside "relentlessly resourceful," a few more specific traits matter: tough, and calm — especially if you know you aren't naturally either of those things yourself. Smart is assumed; almost everyone already screens for that and undervalues the rest. And if

you're not a technical person, you want a technical cofounder, particularly if you're building a software company. Altman is skeptical of the increasingly popular idea that a non-technical founding team can just "hire the technical people" and focus on being great managers instead: "Software people should really be starting software companies."

How many, and how close

Y Combinator's experience points to two or three cofounders as close to the ideal number. One founder, working alone, is workable but harder. Five is close to unworkable — too many voices, too much coordination overhead before the company has any process to manage it. Four sometimes works. Two or three is the real target.

Getting to know a potential cofounder takes time — ideally years, not weeks. This is part of why "cofounder dating" events, where strangers meet and agree to start a company within the same afternoon, are close to the worst possible way to find one. Altman's advice, matter-of-factly: take advantage of whatever environment already puts you around smart, ambitious people for an extended period — college is one version of this, working at a company full of people like Facebook or Google's early employees is another. Being in either environment for years is, in a very literal sense, cofounder-rich in a way that a one-off networking event never will be.

A modern illustration of what "already knowing someone" looks like at its most extreme, outside anything covered in the original lectures: Patrick and John Collison, who founded Stripe, are brothers. That's obviously not a formula anyone can copy — you can't manufacture a sibling — but the underlying mechanism is exactly the one this section has been describing. By the time they started a company together, there was no discovery process left to do. They already knew, in complete and tested detail, how the other person behaved under real pressure, what they were actually good at, and whether they could trust each other completely. Most founders have to build that knowledge deliberately, over years of working or studying next to someone. The Collisons simply arrived at the starting line already having it.

Being in the same place

One of Altman's more absolute claims, delivered without much hedging: don't have cofounders working from different locations. His reasoning is that in the early days of a startup, speed and constant, informal communication outweigh nearly everything else,

and video calls — even good ones — don't replicate that. His supporting evidence: look at a list of the most successful software companies of all time and try to find one where the founders were based in different cities from day one. He couldn't find a clean example, and treated the absence as close to conclusive.

Sidebar: What's Changed Since 2014

This is one of the few claims in this book that later evidence has genuinely complicated. GitLab was built from the start as an entirely remote company and went public in 2021 with no central office at all. Zapier's founders were geographically distributed from early on and built a large, durable business that way. Neither company disproves Altman's underlying point about the value of fast, informal, high-bandwidth communication — both invested heavily and deliberately in written culture, asynchronous processes, and tooling specifically to compensate for not sharing a room. What's changed since 2014 is less "remote cofounders can work" (a few examples always existed) and more that an entire toolkit for making it work — better video infrastructure, a cultural expectation of async-first communication, a labor market used to it — has matured enough that the deliberate substitution is easier to pull off than it was when this lecture was given. The underlying warning still holds: if you go remote, it has to be a deliberate design choice with real infrastructure behind it, not a default you fell into because it seemed easier to start that way.

Reid Hoffman, teaching a related lecture in the same course, makes a companion point about location that's easy to conflate with Altman's but is actually distinct: it's not that Silicon Valley is unconditionally the right place to start every company. It's that different businesses need different networks, and Silicon Valley is unusually good at supplying the network some specific kinds of businesses need — software, mobile, marketplaces — and much less good at supplying what other businesses need. His example is Groupon, which required an enormous, old-fashioned sales force as a core part of its early growth strategy. That kind of plan — rent a large office, staff multiple floors with salespeople — tends to get criticized rather than funded in Silicon Valley, where investors and talent are oriented around capital efficiency and network effects instead. Groupon started in Chicago, where that model wasn't a strange or unfundable idea. Hoffman's broader point: ask what kind of company you're actually building, and go to wherever the right network

for *that* kind of company already exists — which is usually Silicon Valley for the categories this book focuses on, but isn't automatically true for every category of business.

Decide the equity split early, and make it near-equal

There's one conversation founders reliably put off, for no good reason: how to split the equity. Altman has watched founders delay this for months, sometimes structuring their own incorporation paperwork in strange ways specifically to avoid having the conversation. His observation: this isn't a discussion that gets easier with time. It gets harder, because the longer you wait, the more each person's sense of what they've already contributed hardens into a position. The right time to have it is near the very start — ideally within the first few weeks of deciding to work together, before the company has gotten far enough along for the conversation to feel like it's litigating the past.

The split itself should be close to equal. This isn't just a fairness norm — it's a diagnostic. If you find yourself unwilling to give a prospective cofounder a genuinely equal share, Altman's advice is to treat that reluctance as information: it's a sign you should think hard about whether you actually want them as a cofounder at all, rather than a sign you should negotiate them down. A cofounder relationship you're already structuring to protect yourself from is a relationship with a problem in it from day one.

None of this replaces the legal mechanism that actually protects the company if the relationship falls apart later — vesting, which Chapter 15 covers in full. This section is about the decision that vesting later locks in, not the paperwork itself: decide early, decide close to equal, and treat any resistance to doing either as a signal worth listening to.



Common Mistakes

- **Treating cofounder selection with less rigor than hiring an employee.** You would never hire someone you'd known for an afternoon into your first ten employees. Doing the equivalent with a cofounder — someone who will own a much larger share of the company and be much harder to remove — is a worse version of the same mistake.

- **Meeting a cofounder through a "cofounder dating" event or a cold introduction and starting immediately.** The track record for teams that don't already know each other well is poor, and it doesn't improve much even when both people seem genuinely talented individually.
- **Ignoring a private sense of unease about someone because they seem impressive.** This is exactly the mistake Paul Graham describes founders making again and again — and by the time it's confirmed, the equity and the company structure are already built around the person.
- **Building a solo-founder company by default, rather than by genuine necessity.** A solo founder can succeed, but the data suggests it's a real, measurable handicap — worth accepting only if the alternative is a bad cofounder, not as a first choice.
- **Choosing a location based on where you happen to be, rather than where the network your specific business needs actually exists.**



Tradeoffs and Edge Cases

Doesn't this chapter's "trust your instincts about people" contradict Chapter 1's "you can't tell if you're capable"? No, but it's worth stating clearly why not, since they sound similar on the surface. Chapter 1's claim is about self-knowledge under a kind of change you haven't experienced yet — you can't know who you'll become once a startup takes over your life, because nothing in most people's prior experience simulates that. This chapter's claim is about reading someone else's existing character: whether they follow through, whether they're honest under pressure, whether you'd trust them when it's hard. That's a different kind of judgment, built from ordinary life experience picking friends and colleagues, and Graham's argument is that this ordinary judgment transfers just fine — the mistake founders make isn't lacking the instinct, it's overriding the instinct they already have.

Are there real exceptions to "never have remote cofounders"? Yes, and the sidebar above covers the clearest ones (GitLab, Zapier). The honest synthesis: the underlying reason for Altman's advice — fast, high-bandwidth, informal communication matters more in the first year than almost anything else — hasn't gone away. What's changed is that it's

now possible to build that bandwidth deliberately without sharing a room, if you invest in it as seriously as those companies did. Choosing to be remote by default because it's convenient, without that investment, is still exactly the trap Altman is warning against.

Is a solo founder ever the right call? Occasionally — some of the highest-valued YC companies have had solo founders, and Altman doesn't claim it's impossible, only that the data shows it's harder. The honest framing from this chapter and the last one combined: a solo founder who is genuinely compelled (Chapter 1's test) and genuinely capable of covering both the technical and non-technical demands of the business is a real, survivable path. It's a worse bet than a great cofounder team, and a much better bet than a mediocre one.

Applying the Test

Think about a specific person you've considered starting a company with. Run Graham's test directly: if you strip away their resume, their confidence, and how impressive they seemed the first time you met them, would you still describe them the way you'd describe a genuine friend — someone you like, respect, and have actually seen operate under some kind of pressure? Not "someone impressive I'd enjoy being associated with." Someone whose character you'd vouch for even if their startup failed and they had nothing to show for it.

Then run the reference-check version of the same test, the way you would for a serious hire rather than a friend: ask someone who has actually worked with them, closely, under stress, whether they were still the same person once things got hard. If the honest answer to either question is uncertain, the data in this chapter says that uncertainty is itself the answer — not a detail to resolve after you've already started the company together.

Chapter Summary

- Cofounder blowups are Y Combinator's most common cause of early startup death — in one batch, all nine companies that added a late, unfamiliar cofounder failed within a year.
- You can't reliably judge your own future capability (Chapter 1), but you can and should trust your instincts about another person's character — the mistake is overriding that instinct because someone seems impressive.
- The ranking that matters: a cofounder you already know well, then a solo founder, then — worst of all — a cofounder you barely know.
- "Relentlessly resourceful," embodied by the James Bond archetype, matters more than deep domain expertise in a cofounder; domain expertise can be hired in later.
- Two or three cofounders is the practical target. Getting there usually takes years of actually knowing someone, not a single event designed to pair up strangers.
- Being in the same physical location remains the default advice, though remote-first companies since 2014 (GitLab, Zapier) show it can work with serious, deliberate investment in communication — not as a default.
- The right location for a company depends on which network that specific business needs, not just where Silicon Valley happens to be strong.
- Decide the equity split early and keep it near-equal. Reluctance to give a cofounder an equal share is itself useful information about whether you actually want them as a cofounder.



Exercises

1. **Recall:** Explain, in your own words, why "trust your instincts about people" isn't a contradiction of Chapter 1's claim that you can't reliably judge your own future capability.
2. **Apply:** Think of someone you've considered as a cofounder. Run the reference-check test from this chapter's "Applying the Test" section on them honestly. What did you learn that you hadn't consciously admitted to yourself before?

3. **Apply:** If you don't currently have a cofounder, identify one environment in your life right now (school, work, a community) that's genuinely cofounder-rich in the way this chapter describes. What would it take to actually spend real time there, rather than treating cofounder-finding as a one-off event?
4. **Confront a counterexample:** GitLab and Zapier both succeeded as deliberately remote companies, directly against Altman's advice in this chapter. What specifically would you need to see in your own founding team's plan to know whether you're doing what they did — building real infrastructure for remote collaboration — versus just defaulting to remote because it's easier to start that way?



Further Reading

- Paul Graham's own CS183B lecture is the direct source of the "trust your instincts about people" argument this chapter builds on.
- Reid Hoffman's own CS183B lecture, only briefly drawn on in this chapter for the team-composition and location material, covers a different set of topics at greater length in Chapter 18 of this book (the investment thesis, and navigating a founder's recurring paradoxes) — not LinkedIn's early hiring, which isn't repeated there.
- Chapter 15 covers the mechanics of cofounder vesting in full — the legal structure that protects a company if a cofounder relationship does fall apart.



Chapter 5 — The First Ten Hires

Airbnb spent five months interviewing its first employee. In the company's entire first year, it hired two people. Before any of that, Brian Chesky and his cofounders wrote down the culture values they wanted every future Airbnb employee to hold — before there was a product worth using, before there was any revenue, before there was anyone to apply the values to yet. One of those values was that you had to "bleed Airbnb." Chesky used to ask candidates whether they'd take the job if they'd just been told they had one year left to live. He later decided that was a little too intense and relaxed it to ten years. As far as anyone at Y Combinator has heard, he still asks it.

This is not how most first-time founders hire. Most first-time founders hire the way they'd hire for anything else that felt urgent: post a listing, take some interviews, extend an offer to the strongest candidate within a reasonable amount of time. This chapter is about why that instinct, applied to a startup's first ten employees, is one of the more expensive mistakes a founder can make — and what the alternative actually looks like.



The Core Idea: Try Not To Hire, Then Hire Completely Differently

The starting stance, before any hiring tactics matter at all, is this: try not to hire. Sam Altman's observation about the psychology of headcount is worth stating plainly, because it cuts against a natural instinct — people treat a company's employee count as a proxy for how real and impressive it is. Say a high number, and people are impressed. Say a low number, and it can feel like an admission that the company isn't serious yet. The actual relationship runs the other way. A large headcount usually means a high burn rate, more complexity, and slower decisions. The startups that stay smallest longest, for their size and stage, are frequently the ones doing the best — proud of how much they can get done with how few people, not embarrassed by it.

That said, at some point you do have to hire — and once you're in hiring mode, the rule changes completely. Altman's answer to how much time hiring should take: zero percent, or twenty-five percent. There's no comfortable middle. Either you're not hiring at all right now, or it is close to the single largest block of your time as a founder. Most management books recommend something like fifty percent, but Altman notes that the people who give that advice rarely spend even ten percent of their own time on it themselves. Twenty-five percent, done for real, is already an enormous commitment — and it's what's actually required to get this right.

The reason this matters so much comes down to one blunt fact: the cost of getting an early hire wrong is far higher than the cost of getting a later one wrong. A mediocre hire at a company with a thousand employees causes real but contained damage — that person is one of a thousand, and the company's culture and standards absorb the mistake. A mediocre hire among a company's first five or ten people sets the tone for everyone who joins after them, and Altman has seen it kill companies outright. A friend of his keeps a sign in his conference room, angled so interview candidates can read it during the interview: *mediocre engineers do not build great companies*. It's blunt, and it's true, and it's exactly the standard a big company can survive violating in a way a ten-person startup can't.



Why It's True: Culture Before Hiring, Sourcing Beyond LinkedIn, and What to Actually Test For

Write the culture down before you hire anyone

Airbnb's approach — writing culture values down before making a single hire — sounds like a strange amount of process for a company with no employees yet. Patrick Collison of Stripe, teaching later in the same course, offers the underlying logic for why it isn't overkill: when you hire your first ten people, you're not really hiring ten people. You're hiring something closer to a hundred, because each of those first ten will go on to bring in roughly ten more people who resemble them culturally. Deciding, deliberately, what those first ten should be like — before you're under pressure to just fill a role — is one of the

highest-leverage decisions a founder makes, precisely because its effects compound for years afterward.

At Airbnb, this showed up concretely: every one of the company's first forty or so employees reports feeling like they were part of founding the company, not like they joined after the fact. That feeling doesn't happen by accident. It happens because the company set an extremely high bar — Airbnb spent five months on its first hire alone — and everyone who cleared that bar knew, going in, exactly what they were signing up to believe in.

There is no wrong place to find people, but there is a best one

Personal referrals — people you already know, and people your existing employees already know — are, by a wide margin, the best source of early hires. Most founders feel awkward calling every talented person they've ever met and asking their own employees to do the same. Do it anyway: at a large company like Facebook or Google, one of the first things that happens to a new hire is an HR conversation designed specifically to extract every talented person they know, for exactly this reason. A useful, underused variant: look outside the immediate geographic area you're hiring in. Competition for engineers in a place like Silicon Valley is brutal; the person you already know from somewhere else in the world may be just as good and much easier to bring on.

Ben Silbermann, Pinterest's cofounder, makes a complementary point from the other direction: there's no wrong *place* to look, even if referrals are the best source on average. His own early hiring included posting ads on Craigslist, showing up to random tech talks, and running weekly bring-your-own-food-and-drink barbecues at the office just to get people talking. His candid observation about where a lot of his best people actually turned up: any time he went to get coffee at a nearby café, there'd be another founder there doing exactly the same kind of recruiting. Genuinely good people are almost always doing something else already — you have to go find them, especially when nobody has heard of your company yet.

That last point — nobody's heard of you yet — is worth sitting with, because it's the central obstacle to hiring anyone at all in the first few months. John Collison describes what this actually feels like: you're two people working on something strange that nobody recognizes, trying to convince someone to leave a stable situation for it. For Stripe's second hire, his own friends staged what Collison calls "a full-on assault" the night before

he accepted, trying to talk him out of joining. He joined anyway. One of those same friends who tried to talk him out of it later joined Stripe himself.

Given that friction, a specific and slightly counterintuitive sourcing insight from Patrick Collison is worth naming directly: look for people who are early in their careers or currently undervalued by the broader market, rather than trying to poach someone already recognized as excellent. Someone the market has already correctly identified as a "known spectacular quantity" is, by definition, already happy somewhere and hard to move. Stripe's early team included a designer who was eighteen and still in high school in Sweden, and a future CTO who was still in college. Neither looked, on paper, like an obvious hire for a payments company. Both turned out to be exactly the kind of person the company needed — because the company was betting on who they'd become, not just who they already were on a resume.

What to actually test for, and how

Once you have candidates, the traits that matter are more specific than "smart" — smart is close to a given, and most founders already screen for it without being told to. Altman's own three-question test, applied consistently across every hire he's ever made and rarely regretted: **is this person smart? Do they get things done? Do I want to spend a lot of time around them?** Answer yes to all three, and it's almost always worked out.

Patrick and John Collison arrived independently at a closely related, slightly more specific set of three traits after examining what their own first ten hires had in common: they were **genuine** — people others trusted and wanted to work with; they **cared a great deal**, to a degree Collison himself now considers a little extreme in hindsight (every API error at Stripe was, for a period, emailed directly to the founders' personal inboxes, and early employees would copy each other on outgoing emails just to flag small grammar mistakes); and they were people who **finished things**, as opposed to people who were excited about many projects but rarely completed any of them.

The best way to actually assess any of this isn't a standard interview. It's working together on something real, even briefly, before formally hiring. If you've already worked with someone, in many cases you don't need a traditional interview at all — you already have the evidence. If you haven't, a short paid project, ideally spanning a couple of days, teaches both sides more than any conversation could. Where an interview is unavoidable, ask about specific past projects in detail rather than reaching for brainteasers — a pattern

Altman notes young technical cofounders fall into surprisingly often, apparently because it feels more rigorous than it actually is.

References are the step first-time founders skip most often, and the step that matters most when it isn't skipped. The wrong way to reference-check is to ask "how was so-and-so?" and accept a pleasant, generic answer. The right way, borrowed from Ben Silbermann's own practice: force the reference to be specific by asking a version of "if I asked [a mutual contact] what you're best at, or what you're working to improve, what would they say?" — which builds in social accountability the reference can't easily dodge. Then push further: is this person in the top five percent of everyone you've worked with? The top ten? Would you hire them again? If not, why not? A generic answer to "were they good?" tells you almost nothing. A forced ranking against everyone else that person has worked with tells you a great deal.

One more correlated signal worth tracking, per Altman's own observation across many YC companies: communication skills predict whether a hire works out more reliably than founders initially expect. Someone who struggles to explain themselves clearly in an interview tends to struggle the same way once they're actually on the team, under real pressure. One last practical heuristic, from Mark Zuckerberg by way of Altman: would you be comfortable hanging out with this person socially, and would you be comfortable reporting to them if your roles were reversed? You don't need to be friends with everyone you hire, but if the honest answer to both is no, that's worth taking seriously.

Give employees real equity — more than feels natural

Founders consistently make an equity mistake in a specific, predictable direction: generous with investors, stingy with employees. Altman's advice is to reverse that instinct. His own rough benchmark: aim to give about ten percent of the company, combined, to your first ten employees. They'll earn it over the standard four-year vesting period, and if they're genuinely good, they'll go on to create far more value than that ten percent represents. Investors write a check once and, despite the promises that often come with a term sheet, frequently contribute less ongoing value than founders expect. Employees are the ones who build the company, year after year. The YC companies that have been unusually generous with early equity are, in Altman's account, disproportionately represented among the ones that went on to be the most successful.

Keep the people you just spent months hiring

Hiring well and then managing badly wastes the whole effort. A specific, recurring failure mode among first-time founders: telling people when they're doing something wrong constantly, because it feels obviously correct to point out mistakes as you see them, without realizing that a team told they're failing every day will simply leave. The founders who get this right tend to follow a few unglamorous habits: let the team take public credit for what goes well, and take the blame yourself for what goes badly. Don't micromanage — hand people real, if initially small, areas of responsibility, and let that responsibility grow. Researcher Dan Pink's framework for what actually motivates people to do their best work — autonomy, mastery, and purpose — is worth taking seriously even though it's rarely top of mind for a first-time founder buried in product work. Regular one-on-ones and direct, specific feedback round out the list. None of this is complicated. All of it is easy to skip when you're moving fast, which is exactly why it gets skipped.

Fire fast when it isn't working

The flip side of generous hiring is fast firing, and first-time founders reliably get this wrong in the same direction: they wait too long, hoping the person will turn it around. The better rule, painful as it is in the moment, is to fire quickly once it's clear something isn't working — it's better for the company, and it's usually better for the person too, who's stuck in a role they're failing at. Two specific patterns are worth watching for beyond simple underperformance: people who create office politics, and people who are persistently negative, since both are corrosive to a small team in a way a large company can better absorb. The test for knowing it's genuinely time to act, rather than just a rough patch: everyone makes mistakes occasionally, and that's normal — the signal to actually act is when someone is getting nearly every decision wrong, consistently, over weeks or a month, to the point where it's obvious to everyone on the team, not just to you.



Common Mistakes

- **Hiring to look more legitimate, rather than because the work genuinely requires it.** A low headcount is a sign of efficiency, not a sign of being unserious — treat it that

way.

- **Interviewing with brainteasers instead of asking about real past projects.** Past work is a far better predictor of future work than a clever puzzle solved under artificial pressure.
- **Accepting a vague, pleasant answer from a reference instead of forcing specificity.** "They were great to work with" is not a data point. "Top five percent of everyone you've worked with, and here's specifically why" is.
- **Compromising on a hire because of a pressing deadline.** Every founder does this eventually as the company scales, and it's regretted almost every time it happens among the first five to ten hires specifically.
- **Only sourcing from people who already look impressive on paper.** The most valuable early hires are frequently undervalued by the broader market — early in their careers, or overlooked for reasons that have nothing to do with how good they actually are.
- **Skipping the culture-values conversation until the company already has employees.** By the time you need the values to resolve a real disagreement, it's too late to invent them from scratch.
- **Being generous with equity to investors and stingy with employees.** It's usually backwards — employees create most of a company's long-term value.
- **Telling people they're doing something wrong constantly, without also giving them credit and real ownership.** This is one of the fastest ways to cause otherwise-good hires to quit.
- **Waiting too long to fire someone who is clearly, consistently failing.** It's painful in the moment and usually better for both the company and the person once it finally happens.



Tradeoffs and Edge Cases

Does "try not to hire" ever stop applying? Yes — this chapter's advice is specifically about the earliest hires, where the cost of a mistake is highest and the company has the least infrastructure to absorb one. Later in a company's life, the opposite skill becomes

necessary: hiring fast and scaling a team deliberately. Chapter 21 covers what changes once a company crosses roughly twenty-five employees and needs real management structure for the first time — the "try not to hire" instinct from this chapter is explicitly not the right instinct once you're there.

Does experience matter, or doesn't it? Both, depending on the role. For a role running a large, established part of the organization, real experience usually matters. For most early hires at a very young startup, Altman's own experience is that aptitude and genuine belief in the mission matter more than having done the exact job before — some of the best hires he's ever been involved in had never done the specific role they were hired into. The judgment call is knowing which kind of role you're filling, and not defaulting to "more experience is always better" without asking that question first.

What happens when someone who was a great early hire stops being able to grow with the company? This is a real, common problem — someone hired to lead engineering at five people may not be the right person to lead a team of fifty. Chapter 19 covers a more precise framework (task-relevant maturity) for reasoning about when someone can grow into a bigger role versus when a role needs to change around them. The short version for this chapter: genuinely smart, adaptable people can usually find *some* valuable role as the company scales, even if it isn't the one they started in — which is a reason to hire for underlying aptitude in the first place, not just for the specific job description in front of you today.

Applying the Test

Take someone you're considering hiring, or someone already on your team whose fit you're unsure about. Don't ask yourself the vague version of the question — "are they good?" — because it's too easy to answer yes by default. Ask the forced version instead: if you called three people who've worked closely with this person, would each of them independently rank them in the top five percent of everyone they've worked with? Would each of them tell a specific story about something this person finished, under pressure, without being managed closely? If you're not confident what the honest answer would be,

that uncertainty is itself the signal — the same way it was in Chapter 4's cofounder test. The fix isn't to hope it resolves itself. It's to go make the calls before you decide, not after.

Chapter Summary

- Try not to hire until you genuinely need to — a small headcount is a sign of efficiency, not a sign of being unserious.
- Once you are hiring, it should take zero percent of your time or roughly twenty-five percent — there's no comfortable middle, because getting it right requires real, sustained attention.
- A bad hire among your first five to ten people can kill the company outright, in a way a bad hire at a thousand-person company can't.
- Write your culture values down before you hire anyone, the way Airbnb did — deciding this deliberately, rather than under pressure to fill a role, compounds for years.
- Personal referrals are the best source of candidates, but there's no wrong place to look — Ben Silbermann hired through Craigslist, tech talks, and office barbecues.
- Test for specific, concrete traits — smart, gets things done, someone you want to spend time with; or Stripe's version: genuine, cares a great deal, finishes things — rather than generic intelligence alone.
- Work together on something real before formally hiring, whenever possible. When you can't, ask about specific past projects instead of brainteasers, and force specificity out of every reference check.
- The most valuable early hires are frequently undervalued by the broader market — early in their careers, not yet recognized as exceptional by anyone else.
- Be generous with equity toward employees, not just investors — roughly ten percent of the company to the first ten hires is a reasonable benchmark.
- Retention is a management habit, not a one-time decision: give credit publicly, take blame yourself, avoid micromanaging, and don't tell people they're failing every day.

- Fire fast once it's clearly not working — waiting is worse for the company and usually worse for the person, too.



Exercises

1. **Recall:** Explain why the cost of a bad hire is so much higher in a company's first ten employees than in its thousandth.
2. **Apply:** Write down your own company's (real or hypothetical) culture values, the way Airbnb did before hiring anyone. If you already have employees, would each of your first few hires have clearly passed those values, or were the values invented after the fact to describe people you'd already hired?
3. **Apply:** Pick someone currently on a team you're part of, or someone you're considering hiring. Draft the specific reference-check questions from this chapter you'd actually ask about them, and predict honestly what the answers would be.
4. **Confront a counterexample:** This chapter argues aptitude and mission-belief matter more than experience for most early hires. But some of the companies covered elsewhere in this book (enterprise software, capital-intensive businesses) clearly do need specific deep expertise from day one. How would you tell, for your own company, which category you're in?



Further Reading

- Patrick and John Collison's account of Stripe's early hiring, referenced throughout this chapter, is covered from the culture-building side in Chapter 6.
- Chapter 19 covers task-relevant maturity — the framework for deciding how much a hire can grow into a larger role as the company scales.

- Chapter 21 covers what changes about hiring once a company crosses roughly twenty-five employees and "try not to hire" stops being the right default.



Chapter 6 — Designing Company Culture

In the fall of 2008, Airbnb had almost no revenue, no real traction, and founders who were paying for the company on a stack of credit cards. Around that time, Paul Graham asked Brian Chesky whether people were actually doing this — renting out rooms in their own homes to strangers over the internet. Chesky said yes. Graham asked what was wrong with them. It wasn't a promising interview.

That same fall, during the Democratic and Republican National Conventions, the founders needed money and came up with something almost nobody would have predicted from a "travel" company: breakfast cereal. Obama-O's — "the breakfast of change" — and Cap'n McCain's — "a maverick in every bite." General Mills wouldn't return their calls. A local printer made them a thousand boxes. The story hit national news within a week, and the founders sold \$40,000 worth of cereal — more than the \$5,000 the actual Airbnb website had made that entire year.

Chesky doesn't tell this story because it's funny, though it is. He tells it because it's the origin of one of Airbnb's actual, permanent core values: **be a cereal entrepreneur** — meaning, be resourceful and creative under real constraint, because constraint is what forces creativity in the first place. That's the subject of this chapter: not culture as a poster on a wall, but culture as the set of values a company's founding moments actually produce, deliberately written down, and then protected for as long as the company exists.



The Core Idea: A Fill-in-the-Blank Definition, and Why It's Not Optional

Alfred Lin, an early investor in Zappos and Airbnb, offers a useful skeleton for a definition that otherwise tends to stay vague: company culture is the **core values** and the **actions** of every team member, every day, in pursuit of the company's **mission**. Fill in those three blanks honestly for any company, and you have a working definition of its actual culture — not the one printed in the employee handbook, the one people actually operate by.

Why this matters enough to spend a whole chapter on: culture is what a team falls back on to make decisions when there's no time to ask the founder directly. It's what tells people what *not* to do — often the more valuable half of the instruction. It's what lets you tell, cleanly, who should stay and who shouldn't, when the honest answer would otherwise be complicated and political. And it's not just a soft, feel-good asset: looking at the Fortune 100 Best Companies to Work For against the S&P 500 and the Russell 3000 from 1994 to 2013, the "best places to work" index returned roughly 11.8% — close to double the other two. Trust and strong culture, in other words, show up in stock returns, not just in employee satisfaction surveys.

Patrick Collison, teaching a related lecture in the same course, offers the sharpest explanation of *why* culture becomes so important as a company grows, rather than staying a nice-to-have: culture is a solution to a bandwidth problem. In an ideally small company, a founder could be involved in every decision. That stops being physically possible almost immediately — by somewhere around 150 people, in Collison's estimate, it's completely hopeless. Culture is the thing you deliberately design to stay constant as the fraction of decisions you can personally touch keeps shrinking toward zero. Get it right early, and the company keeps making decisions the way you would have, long after you've stopped being in the room.



Why It's True: How Real Companies Actually Wrote Theirs Down

Zappos: a year, thirty-seven values, and a whittling process

When Zappos went through the exercise of defining its own values, employees proposed thirty-seven candidates. The company spent roughly a year narrowing that list down to about ten. That length of time is itself a lesson: a value like "honesty" sounds appealingly simple, but it's nearly meaningless as written, because no company would ever claim the opposite. Nobody sets out to build a culture of dishonesty. A real value has to say something that could plausibly *not* be true of some other company — which is why it takes real time and real specificity to get right.

Zappos landed on specifics like "deliver wow through service" — not just "great customer service," but a claim about the *degree* and *quality* of service, tied directly to the company's actual mission. Alongside it: "be humble," which came from the opposite direction Alfred Lin recommends founders use when defining values — think about the people you've disliked working with in the past, and write down the opposite of what made them difficult. Zappos's founders didn't like working with arrogant people, so humility became one of their core values, credible precisely because it was a specific reaction to something real.

One more Zappos habit worth naming directly: prioritizing company, then department, then team, then self, when smart people disagree. The point isn't hierarchy for its own sake — it's steering disagreement toward "how do we make this idea better together" rather than toward "who's right," which matters because a room full of talented people arguing to win is a very different room from a room full of talented people building on each other's ideas.

That last point connects to a specific, checkable model of what actually breaks inside teams, borrowed from Patrick Lencioni's *The Five Dysfunctions of a Team*. The dysfunctions stack on top of each other, each one causing the next:

1. **No trust** among team members leads to —
2. **No real conflict or debate.** Without trust, people avoid disagreement rather than working through it, which means —
3. **No genuine commitment.** If ideas were never actually debated, nobody is sure the final decision is right, so nobody fully commits to it, which means —
4. **No accountability.** People who never truly committed don't hold each other, or themselves, to the outcome, which means —
5. **No results.**

The practical use of this stack: if a team isn't producing results, the fix usually isn't "try harder" or "communicate more." It's diagnosing which layer actually broke first, since accountability and results are downstream problems that can't be fixed directly — they're symptoms of a trust or conflict problem further up the stack.

Airbnb: writing the values before hiring anyone at all

Zappos wrote its values down at around a hundred employees — successful, but later than its own founders wished in hindsight. Airbnb's founders, after visiting Zappos and learning this, decided to do it earlier: before hiring a single person. As far as Sam Altman has been able to tell, Airbnb may be the only company in Y Combinator's history to have written its core values down before making any hire at all.

Chesky's framing for why this mattered so much: your first engineering hire isn't really one hire. It's closer to a DNA sample for the entire company, because if the company succeeds, there will eventually be a hundred thousand more people like that first one. The real question isn't "can this person do the job." It's "do I want to build an organization out of a hundred thousand people who think and behave the way this person does." Airbnb's first engineer took four to five months to hire — a number that echoes Chapter 5's account of Airbnb's five-month search for its very first employee overall, because it's describing the same hire.

Chesky distinguishes two layers inside "culture," and the distinction matters: there are **behaviors** — the specific rituals and practices a company follows — and there are **core values** underneath them. Behaviors can and should evolve over decades. Core values are supposed to be the three to six things that never change, the things that would still describe the company fifty years from now even if almost everything else about how it operates looks completely different. Steve Jobs's own version of this distinction, quoted by Chesky: "Our products change but our values never have." Chesky's broader observation is that this isn't unique to companies — nations do the same thing with founding declarations, precisely so they can endure past any single leader.

Missionaries versus mercenaries

The clearest test of whether a company's stated values are real or decorative is what happens when following them costs something. Airbnb's version of this test arrived in 2011: a company founded by the Samwer brothers, known for aggressively cloning successful American startups and building them out at extreme speed, copied Airbnb across Europe. Within thirty days, the clone had hired four hundred people. The pragmatic case for acquiring them was strong — Airbnb, still only forty employees and seven million dollars raised at the time, had no real path to matching that speed on its own, and a travel company that isn't in Europe is missing a critical piece of the map.

Chesky chose not to buy them. His own explanation: "I felt like we were missionaries and they were mercenaries," and he believed — as a bet, not a certainty — that missionaries outlast mercenaries in the long run, and that the best response to being cloned was simply to keep building the real thing better than the clone ever could. This was, by his own account, a genuinely controversial call inside the company at the time. It's worth being honest that it wasn't an obviously correct decision in the moment — reasonable people on Airbnb's own team argued for buying the clone. What makes it a useful example for this chapter isn't that it was risk-free. It's that Airbnb had actually written down, in advance, what kind of company it wanted to be — which meant that when a real, costly decision arrived, there was already a values-based answer to reach for instead of improvising one under pressure. More than half of Airbnb's revenue eventually came from Europe anyway.

Culture and brand are the same coin, seen from two sides

A recurring theme across the founders in this chapter: culture (what you believe internally) and brand (what customers believe about you) aren't separate projects. Zappos's own framing is that they're two sides of one coin — whatever a company actually believes, internally, eventually leaks out into how customers experience it, whether or not that was intended. Google's internal hiring shorthand — asking whether a candidate "is Google" — captures the same idea from the inside. There's no such thing, in this framing, as a good culture or a bad culture in the abstract. There's only a strong culture or a weak one, and a strong culture that fits one company might be entirely wrong for another.

Airbnb learned this the hard way with its own external messaging. Its early tagline — "forget hotels, save money with Airbnb" — described the company as a cheaper substitute for something else, which is a fundamentally weak position: a company selling on price alone is, in Chesky's words, "a utility," and utilities get commoditized and priced down over time. The company's later language — "travel like a human," built around belonging rather than savings — matched what Airbnb's founders actually believed the company was for. Steve Jobs made the identical pivot at Apple with the 1997 "Think Different" campaign: not a word about processor speed or specifications, entirely about what the company believed. Chesky's summary of a founder's actual communications job: you articulate the vision, then keep repeating it — in recruiting conversations, investor

meetings, press interviews, customer emails — until it's said itself so many times that it's simply what people believe about the company, largely because it's true.

Culture has to extend past your own employees, to the people your product depends on

Chesky is candid that Airbnb got this one wrong for years before correcting it. In the company's earliest days, he took what he calls "the Craig Newmark school of thought" — named for Craigslist's founder — that anyone should be able to use Airbnb: if you wanted to rent out your place, you simply could, with no cultural gate at all. Most hosts, it turned out, genuinely believed in the same values the company did. But a meaningful minority didn't — they'd found a way to make money renting out a home, full stop, with no particular attachment to what Airbnb was actually trying to build — and Chesky is direct that this group caused a disproportionate share of the company's real problems. The lesson landed late: hosts aren't just users of the product, they're partners in it, and partners need to actually believe in the same culture employees are hired against, not merely comply with a terms-of-service agreement. Airbnb's fix was the Superhost program — a status hosts have to actively earn by demonstrating the company's values in practice, in exchange for priority support and better placement in search results. The broader point generalizes past Airbnb specifically: a culture that only governs the people on your payroll, while leaving the people your product actually depends on entirely ungoverned, has simply moved its weakest link outside the building instead of fixing it.

Transparency scales, but not automatically

Culture at ten people and culture at two hundred people require different tools, even if the underlying values stay fixed. Stripe's founders made an unusually aggressive bet on one specific value — internal transparency — as the company grew, and their experience shows both the upside and the real cost of taking a value that seriously. Early on, nearly every email at Stripe went to every employee by default, on the theory that shared context eliminates a large share of the meetings and misunderstandings that otherwise accumulate as a company grows. Ben Silbermann of Pinterest, comparing notes with Stripe's founders, describes the underlying design philosophy well: culture is more like gardening than architecture. You plant values early, and then you spend ongoing effort pulling weeds and helping the right things grow — it's not a structure you build once and leave alone.

The costs of Stripe's transparency bet were real, not hypothetical. At roughly fifty employees, new hires reported being overwhelmed rather than informed — too much email to find what actually mattered to them. At one point, internal email volume grew large enough to break Gmail itself. The fix wasn't abandoning the value; it was building purpose-made tooling (mailing-list structures, a company-wide shared-documents system) and establishing social norms about what should and shouldn't be shared as openly, once open sharing itself was happening in front of a much larger and more anonymous audience than the first ten people in one room. The lesson generalizes past transparency specifically: a value that worked effortlessly at ten people usually needs real, deliberate infrastructure to keep working at two hundred — it doesn't scale for free just because it was true from day one.

Common Mistakes

- **Treating culture as a one-time document instead of a daily habit.** Alfred Lin's comparison: culture is like fitness. You can't do a single hard session and then coast — skip it for a while, and it deteriorates the same way physical fitness does.
- **Writing values so generic that no company would ever claim the opposite.** "Honesty" and "integrity" sound safe, but they don't tell anyone how to actually behave differently from a competitor. A real value should be specific enough to be falsifiable.
- **Interviewing only for skill, not for whether someone actually believes in the mission.** A brilliant hire who doesn't believe in what the company is doing tends not to put in the sustained effort the mission actually requires.
- **Assuming a value that worked at ten people will keep working at two hundred without any new investment.** Stripe's own transparency bet needed real tooling and new norms once it stopped being ten people in one room.
- **Only discovering what the company's values mean under pressure, instead of before it.** Airbnb had already decided, before the Samwer brothers ever appeared,

that it wanted to be a company of "missionaries" — which is what made a genuinely hard decision resolvable instead of just another argument.



Tradeoffs and Edge Cases

Was the missionaries-versus-mercenaries decision actually correct? Worth being honest here: it's not possible to prove, even in hindsight, that acquiring the Samwer brothers' clone would have gone worse for Airbnb. The pragmatic case for buying speed in a new, unfamiliar continent was real, and reasonable people inside the company argued for it. What this chapter is actually claiming is narrower and more defensible: having a written, real set of values in advance turned an agonizing, ambiguous decision into one Airbnb's founders could reason through quickly and stand behind, rather than making it up under pressure with no anchor at all. A company without that anchor doesn't necessarily make the wrong call — it just has no faster way to make the right one.

Isn't a company this focused on mission and values really just a marketing exercise? This is a real question people asked Chesky directly, and it's worth taking seriously rather than dismissing. Sequoia's Doug Leone once told Chesky, half-joking, that his job was the worst in his entire portfolio: Airbnb is a technology company facing every challenge a normal software startup faces, plus operating in effectively every country in the world, plus being a payments company handling billions of dollars with real fraud and regulatory exposure, plus a trust-and-safety operation on the scale of governing a mid-size city (Leone's comparison: imagine being responsible, every single night, for everything that happens among the roughly 425,000 people staying in strangers' homes on a typical peak night, across 190 countries), plus navigating distinct regulations in thirty thousand different cities, plus a genuinely hard search-and-matching problem, since there's no single "best" listing in a city the way there might be a single best search result for a query. None of that is marketing. The point of this chapter isn't that culture *replaces* solving those problems — it's that a real, specific culture is what makes an organization capable of solving all of them at once, coherently, without a founder personally supervising every decision.

Does writing values down early guarantee anything? No — Zappos wrote its values at a hundred employees and still built one of the most-cited culture examples in business. Earlier is better, mainly because it means fewer bad habits have already calcified by the time you do it. But "we haven't done this yet and we already have fifty employees" is a reason to do it now, not a reason to conclude it's too late to matter.

Applying the Test

Try Alfred Lin's actual exercise, the one Zappos used to generate its own list. First, write down the people you've most enjoyed working with, and name specifically what made them good to work with — not vague traits, but things you could point to. Then write down people you've actively disliked working with, and name the opposite. Somewhere in that contrast is the beginning of a real values list — not one copied from another company's careers page, but one built from your own specific, lived experience of what good and bad collaboration actually look like.

Then apply Chesky's harder test to it: if you had to describe your company in three to six words that would still be true in fifty years, regardless of what the product looks like by then, what would they be? If the honest answer is closer to "we're not sure yet," that's useful information — it means the exercise above is worth doing now, before the company is large enough to make doing it much harder.

Chapter Summary

- Culture is the core values and daily actions of every team member, in pursuit of the company's mission — not a poster, a real, checkable pattern of behavior.
- Culture matters because it's how decisions get made when a founder can't be involved in every one — Patrick Collison's framing: culture is a solution to a bandwidth problem that gets worse as the company grows.

- The best-places-to-work index outperformed the S&P 500 and Russell 3000 by nearly 2x from 1994–2013 — strong culture shows up in financial returns, not just morale.
- Write values from lived experience — who you loved and hated working with — not from generic, unfalsifiable words like "honesty" that no company would ever claim the opposite of.
- Separate behaviors (which can evolve for decades) from core values (which shouldn't change) — Airbnb wrote its values before hiring anyone; Zappos wrote its own at a hundred employees and still succeeded.
- The real test of a stated value is what it costs you to follow it under pressure — Airbnb's choice not to acquire a well-funded clone is the clearest example, and it's worth being honest that it wasn't a risk-free, obviously correct call.
- Culture and brand are two sides of the same coin — what you actually believe internally eventually shows up in how customers experience the company, intentionally or not.
- A value that works at ten people doesn't automatically keep working at two hundred — Stripe's transparency bet needed real tooling and new norms as the company grew.



Exercises

1. **Recall:** Explain, in your own words, why "honesty" is a weak core value even though it sounds important, and what would make a value strong instead.
2. **Apply:** Run Alfred Lin's exercise from this chapter's "Applying the Test" section: list people you've loved and hated working with, and derive three to six specific values from the contrast.
3. **Apply:** Describe a real or hypothetical situation where following your own stated values would cost your company something concrete — money, speed, or a deal. If you can't think of one, what does that suggest about whether the values are specific enough yet?
4. **Confront a counterexample:** Airbnb's decision not to acquire its European clone is presented as a values-driven success story, but this chapter admits it wasn't provably the correct call. Under what circumstances would a values-driven decision actually be

the *wrong* one for a company to make, and how would you tell the difference in the moment rather than only in hindsight?



Further Reading

- Patrick Lencioni's *The Five Dysfunctions of a Team* is the source of the trust-conflict-commitment-accountability-results stack covered in this chapter, and goes into far more depth than the lecture format allowed.
- Chapter 5 covers the hiring side of this same story — Airbnb's five-month search for its first employee, which this chapter revisits from the culture-definition angle.
- Chapter 21 returns to the transparency and communication challenges introduced here, at the larger scale of an entire company past its first hundred employees.



Chapter 7 — From Idea to MVP: Talking to Users

Adora Cheung's first real company wasn't Homejoy. It was Pathjoy, founded with her brother around 2009, built on a genuinely appealing mission: make people happier. From that mission, they reasoned their way to a product — a platform for life coaches and therapists, since those are, on paper, the people whose job is making other people happier. They spent almost a year building it.

There was one problem, and it only became obvious once the product existed: neither founder would ever actually use a life coach or a therapist themselves. It wasn't a problem they had. It was a problem they'd reasoned their way into believing they should care about. Homejoy — a real company, a real business, eventually a real acquisition target — came later, and only after this failure taught Cheung something she carried into every product decision afterward: an idea that sounds meaningful in the abstract is worthless if you can't picture yourself actually being a customer for it.

This chapter is about the space between "I have an idea" and "I have something worth building a company around" — the part of the process where a founder turns a hunch into a real minimum viable product, and learns, through actual conversations with actual users, whether that hunch was ever true.



The Core Idea: Storyboard It, Then Build the Smallest Real Version

Before writing any code, Cheung's advice is to storyboard the entire experience a customer will have — not just the product screen, but everything around it: how they first hear about you, what the copy says when they land on your site, what happens when they sign up, what they get from actually using the product, and what happens afterward — do they leave a review, tell a friend, come back next week. Walking through that whole arc on

paper, before any of it is built, tends to reveal what the smallest real version of the product actually needs to contain.

That smallest real version is the **MVP** — minimum viable product — and the emphasis belongs on *viable*, not *minimum*. Cheung's specific warning: a lot of founders build the "minimum" and quietly drop the "viable," shipping something so stripped down it can't actually solve anyone's problem, and then wonder why nobody sticks around. An MVP has to be genuinely small — but it also has to genuinely work for the specific problem it claims to solve.

Once you have something worth testing, Emmett Shear, who built Twitch, adds a discipline that's easy to skip: **who** you talk to about it matters as much as **what** you ask them. Get the wrong people's feedback, and you'll build confidently in the wrong direction no matter how carefully you listen to them.



Why It's True: Becoming an Insider, Storyboarding, and Asking the Wrong Question on Purpose

Become a cog in the industry, even though "disruption" thinking says not to

The popular version of startup wisdom says outsiders disrupt industries precisely because they aren't weighed down by how insiders already think. Cheung's own experience argues for something close to the opposite, at least at the very beginning: spend real time becoming a genuine insider in the industry you're about to build in, before you try to change it.

When she and her brother started what became Homejoy, they didn't just research the cleaning industry — they became cleaners themselves, and discovered quickly that they were bad at it. So one of them went and got an actual job at an existing local cleaning company. That job taught two different things at once: how to actually clean a house well, and — more valuable in the long run — exactly why an existing local cleaning business could never grow into something like Homejoy. The inefficiencies weren't visible from

outside. Booking customers and scheduling cleaners, done the old way, were so manual and ad hoc that nobody inside the business had any reason to imagine them done differently. Cheung's analogy: you can read every book about basketball ever written, but you don't get better at basketball until you actually throw the ball.

This extends to genuine obsessiveness about the competitive landscape: reading every competitor's public filings, sitting in on earnings calls, working through search results well past the first page. Most of it won't teach you anything. Occasionally you'll find a detail that changes how you think about the whole market, and there's no way to know in advance which search result that will be — which is the actual argument for doing the tedious version of this research rather than skipping straight to building.

Positioning in one sentence, users recruited one at a time

Once there's something to show people, the same "one sentence" discipline from Chapter 2 applies to how you describe it to a potential customer. Homejoy's original pitch was long and abstract — an online platform for home services, with cleaning as the first category, explained across several sentences. It lost people before they finished reading. The version that actually worked was blunt and concrete: get your place cleaned for \$20 an hour. Once the offer was that specific, people understood immediately what they'd get, and signups followed.

Getting the very first users rarely looks dignified. Cheung and her brother tried the obvious channels first — using it themselves, asking parents (who lived too far away), asking friends (mostly in the wrong city) — and each one dead-ended. What actually worked was standing at a street fair on a hot day, handing out free ice-cold water bottles, and using the goodwill from that to talk people into booking a cleaning on the spot. Cheung's own description: they'd essentially guilt-tripped strangers into becoming customers. The important detail isn't the tactic itself — it's that most of those guilt-tripped customers didn't cancel afterward, which was the real signal that the underlying product was solving something real, however unglamorous the first ten customers' path to signing up had been.

Narrow the customer segment before you try to serve everyone

A related discipline, easy to skip past because it feels like giving up on the eventual goal: Cheung's advice is to deliberately corner off a specific customer segment early, rather

than building for "everyone" from day one. The eventual product might reasonably serve nearly anyone, but optimizing for a genuinely broad audience at the start means optimizing for no one in particular. Whether the chosen slice is teenage girls or soccer moms, picking one narrow group on purpose lets a founder focus hard enough on that group's specific needs to actually build something that works for them, rather than a diffuse compromise that half-works for everyone.

Once real users exist, they need a real way to reach back. Cheung's specific recommendation is a phone number with voicemail attached, so a founder isn't forced to answer every call live — but some inbound channel has to exist. That said, an open inbound channel is a supplement to talking with users directly, not a substitute for it: the real signal comes from founders going out and initiating those conversations themselves, not waiting for complaints to arrive. For a faster read on whether things are working than the weeks or months a full retention curve takes to materialize, Cheung recommends leading indicators that arrive immediately — star ratings and reviews, or a **Net Promoter Score (NPS)**: a simple 0–10 rating of how likely a user is to recommend the product to a friend.

Ask what people do, not what they'd like

Emmett Shear's sharpest methodological point, drawn from building Twitch, is about a mistake that feels reasonable in the moment and quietly ruins most early user research: asking people directly whether they'd use a specific feature. The honest-sounding answer you get back is almost worthless, because people are bad at predicting their own future behavior and good at being polite about ideas that sound plausible. The fix is to ask about current behavior and existing pain instead — what do you actually do today, what's frustrating about it, what have you tried — and let the underlying opportunity surface on its own, rather than asking a user to design your roadmap for you.

This is also why *who* gets interviewed matters so much. When Shear's team first launched what became Twitch, the obvious move would have been to interview viewers — the people actually watching game streams. Instead, they focused on **broadcasters**, because the actual behavior of the market was that people followed specific broadcasters around, not platforms — content, in Shear's words, brought the viewers with it. Get the broadcasters right, and the viewers follow. Interviewing the wrong side of a two-sided

market can produce technically honest, carefully collected feedback that still points you in the wrong direction entirely.

A related discipline: talk to a genuinely varied set of people, not just whoever is easiest to reach. Six to eight interviews are usually enough to stop learning new things — but only if those six to eight people are meaningfully different from each other. Six interviews with people just like you or just like your existing users will teach you far less than six interviews spanning real extremes — power users and people who've never tried anything like your product at all, since each of those groups reveals a different kind of blocker.

Live conversation — in person or over video, not email — matters more than it might seem. Written exchanges lose the moment where a founder notices something unexpected and asks "tell me more" on the spot, which is usually where the most valuable insight in the entire conversation shows up. Shear also recommends recording interviews (with permission) and playing the recordings back for the rest of the team, rather than just summarizing findings secondhand — a team hearing a real user struggle in their own words tends to be far more convinced than a team reading a bullet-point summary of the same conversation.

The honesty curve, and why paid feedback is the most valuable kind

Not everyone who gives you feedback is being equally honest, and the pattern is predictable enough to plan around. For a free product, feedback gets *more* honest the less the person knows you: your own mother will tell you your product is wonderful regardless of the truth; friends will be fairly honest, since they care about you but aren't quite as invested as family; strangers with no relationship to you at all will be the most bluntly honest of anyone, precisely because they have no reason to spare your feelings.

Charge money, and the curve changes in an important way: your mother still won't tell you the truth. But strangers who've paid become dramatically more honest than they were when the product was free, because now there's money on the line rather than just an opinion. This is part of why Cheung recommends getting to a paid version of the product as quickly as is reasonable — not necessarily charging from the very first day, but getting there fast, because the quality of feedback genuinely changes once real money is involved.

Manual before automated, and temporary brokenness over permanent paralysis

At the earliest stage, resist the urge to build automated systems for problems you could solve manually and learn from instead. Homejoy's early process for accepting cleaning professionals onto the platform was entirely manual — phone screens, in-person interviews, trial cleanings, all done by hand, with only a three to five percent acceptance rate. That manual process is exactly what let the founders learn which specific questions actually predicted a good or bad cleaner, data that only existed because a human was paying close attention to every single case. Automating that process before those lessons were learned would have locked in guesses instead of evidence.

The broader principle behind this: temporary brokenness is far better than permanent paralysis. Perfection at this stage is close to irrelevant, because whatever you perfect for ten users won't matter once you have a thousand, and whatever breaks at a thousand wasn't worth preventing when you only had ten. Build for the size you're at now, not the size you hope to be at eventually.

One more warning from this same stage: watch for what Cheung calls the Frankenstein approach — talking to users, collecting a long list of requested features, and building every one of them in the order they were requested. Most individual feature requests aren't actually the right fix. They're a proxy for a deeper problem the user is having, one that a pile of small features tends to paper over rather than solve. The discipline is to ask why a feature was requested before building it, not to treat every request as a ready-made spec.

Knowing when to pivot

Homejoy wasn't Cheung's first idea, or her second. By her own count, it was the thirteenth concept she and her brother fully built and genuinely tried to make work. That number is worth sitting with, because most founders quietly assume something is wrong with them personally when their first idea doesn't take off — not that grinding through a dozen dead ends might just be a normal part of the process.

Cheung's actual test for when to abandon an idea has three parts: the product genuinely isn't growing despite real effort, the features you keep shipping don't seem to improve how many users stick around, or the underlying economics don't work even when

everything else goes right. The hardest of the three to apply honestly is growth, because there are enough stories of founders who stuck with a struggling idea for years before it suddenly took off that "just keep going a little longer" always sounds reasonable in the moment. Cheung's fix for that ambiguity: write down a specific, honest, week-by-week growth plan before you need one — starting from a single user and doubling from there — so that "are we on track" becomes a comparison against a number you committed to in advance, rather than a judgment call made under the emotional pressure of a bad week. Her own rule of thumb: three or four consecutive weeks of flat or negative growth, while genuinely executing hard, is a real signal to reconsider — not proof you're a bad founder, and not something to simply power through by working harder at the same approach.

Common Mistakes

- **Building in stealth and launching cold, expecting the coverage itself to bring loyal users.** Without an early feedback loop, a splashy launch tends to bring a spike of visitors who don't stick around, and no data about why.
- **Asking users directly whether they'd use a proposed feature.** The polite, honest-sounding "yes" you get back rarely predicts real behavior. Ask about what they already do instead.
- **Interviewing whoever is easiest to reach, rather than whoever the business actually depends on.** Twitch's own early team nearly optimized for viewers before realizing broadcasters were the side of the market that actually mattered most.
- **Automating a process before you've learned enough, by hand, to know what the automated version should actually do.**
- **Treating every user feature request as something to build, instead of a clue about an underlying problem worth understanding first.**
- **Waiting for a polished, complete product before shipping,** on the theory that a competitor might otherwise copy an unfinished idea — when the real risk is spending months building the wrong thing with no outside feedback at all.



Tradeoffs and Edge Cases

Doesn't "become a cog in the industry" contradict Chapter 2's point about not needing startup expertise? No — they're about different kinds of expertise. Chapter 2's argument (via Paul Graham) is that you don't need to be an expert in *starting companies*. This chapter's argument is that you do need to become an expert in the *specific industry and users* you're building for — which is exactly the "expertise in your users, not in startups" distinction Chapter 2 already drew. Cheung became an expert cleaner. She didn't need to become an expert in venture financing to do it.

What if you can't literally do the job yourself, the way Cheung cleaned houses herself? Chapter 2 already covered the relevant exception: a founder who is a genuine domain expert — someone who spent years actually working inside the industry professionally, even if not as a hands-on practitioner of the exact task — can close much of the same gap. The point isn't that founders must physically do the job. It's that someone on the team needs to have lived close enough to the problem to know what an outsider's research would miss.

Does user research work the same way across every market? Not entirely — Shear is candid that international and cross-language user research is genuinely harder, and that even careful efforts to interview users in a market with a different language tend to oversample people who are already unusually fluent in the founders' language, which skews the sample in a way that's difficult to fully correct for. It's still worth doing; it's just worth knowing the sample is imperfect in a specific, predictable direction.

Is talking to users the same thing as watching how they actually use a shipped product? No, and it's worth keeping the two separate. Early discovery interviews — the subject of most of this chapter — are about deciding what to build in the first place. Once something is shipped, in-product feedback tools and usage analytics answer a different question: where does the thing you already built break down for real users. Both matter. Neither substitutes for the other.



Applying the Test

Take a feature or product idea you're currently considering, and write down the direct question you'd be tempted to ask a user about it — something like "would you use a feature that does X?" Now rewrite it as a behavioral question instead: what do you currently do to solve this problem, and what's frustrating about that? Notice how much harder the second question is to answer with a polite, meaningless "yes." That difficulty is the whole point — it's forcing a real answer instead of a comfortable one.

Then check who you'd actually ask. If your honest answer is "whoever's easiest to reach," identify the two most different people you could talk to instead — someone deep inside the exact behavior you're trying to change, and someone who hasn't engaged with anything like it at all. The gap between what those two people tell you is usually where the real insight is hiding.



Chapter Summary

- Storyboard the entire user experience — how someone finds you, signs up, uses the product, and comes back — before writing any code.
- MVP means minimum *viable* product. Cutting features is fine; cutting out the thing that actually solves the user's problem is not.
- Become a genuine insider in the industry you're building for, even though "disruption" culture says outsiders see more clearly — Cheung became a cleaner, then worked at an existing cleaning company, before building Homejoy.
- Position the product in one plain sentence ("get your place cleaned for \$20 an hour"), and recruit your first users manually, however unglamorous that looks.
- Ask about current behavior and existing frustration, not whether someone would use a proposed feature — the honest-sounding answer to the second question is usually worthless.
- Who you interview matters as much as what you ask — Twitch's early team had to interview broadcasters, not viewers, because broadcasters were the side of the market

that actually drove behavior.

- Feedback gets more honest the less someone knows you, and paid feedback beats free feedback, because real money changes what people are willing to say. Do things manually before automating, and treat feature requests as clues, not ready-made specs.
- Know your test for abandoning an idea in advance: no real growth despite real effort, features that don't improve retention, or economics that don't work — Cheung's own Homejoy was the thirteenth idea she tried before one of them met that bar.



Exercises

1. **Recall:** Explain why asking a user "would you use this feature?" tends to produce an answer that doesn't predict real behavior, and what question to ask instead.
2. **Apply:** Storyboard the full experience of a product idea you're considering — from how someone first hears about it to what happens after their first use. What does the smallest *viable* version of that story actually require?
3. **Apply:** Identify two people you could interview about a product idea who are genuinely different from each other — not just two people who happen to be easy to reach. What would each of them likely teach you that the other couldn't?
4. **Confront a counterexample:** This chapter argues you should become a genuine insider in your target industry before building for it. But Chapter 2 also covered founders who succeeded by "living in the future" of a technology they understood deeply, without necessarily being industry insiders in the market they disrupted (the VoIP graduate student, for instance). Reconcile these two pieces of advice — when does each one apply?



Further Reading

- Adora Cheung's account of building Homejoy, and the Pathjoy failure that preceded it, is the primary source for most of this chapter's storyboarding and MVP material.
- Emmett Shear's user-interview methodology, developed at Twitch, is covered at greater length in his original CS183B lecture than this chapter had room for.
- Chapter 10 picks up where this chapter's manual-process discussion leaves off, covering the retention metrics (cohort curves, CLV) that tell you whether a shipped MVP is actually working.



Chapter 8 — Building Products Users Love

Wufoo raised a total of \$118,000 before it was acquired by SurveyMonkey in 2011. The average venture-backed startup, over the same kind of timeline, raises around \$25 million and returns roughly 676% to its investors. Wufoo's investors made roughly 29,561% — more than forty times better than average, on a fraction of the capital. The company that pulled this off was ten people, building an online form builder, running entirely remotely out of Florida, with no office at all.

Kevin Hale, one of Wufoo's founders, doesn't attribute this to some clever growth hack. He attributes it to something close to the opposite: a founding obsession with making customers feel genuinely, unconditionally cared for — treated less like users of a database tool and more like people in an actual relationship with the company. This chapter is about what that obsession looked like in practice, and why it produced numbers that a \$25 million marketing budget usually can't.



The Core Idea: Growth Is Conversion Minus Churn, and Both Sides Run on Love

Strip away the complexity, and growth comes down to the gap between two numbers: **conversion** (how many new people you bring in) and **churn** (how many existing people you lose). Most founders instinctively focus on the first number, because it feels more like traditional marketing. Hale's argument, backed by Wufoo's own five-year growth curve — built entirely on word of mouth, with zero paid advertising — is that the churn side of the equation is just as powerful and consistently underinvested in, because it's less visible and less glamorous than acquisition.

Hale organizes the whole idea of "making users love you" around two human relationship metaphors that map cleanly onto a company's actual lifecycle: acquiring a new user is like **dating**. Keeping an existing user happy for years is like a **marriage**. Both relationships fail

for predictable, well-studied reasons — and both can be engineered, deliberately, the same way a product is engineered.

Why It's True: First Impressions, the Feedback Loop, and What Marriage Research Teaches About Customer Support

Dating: first impressions carry disproportionate weight

The first moments of any relationship get remembered and retold far out of proportion to how much time they actually took — which is why Hale treats first impressions as a deliberate design surface, not an accident. Japanese manufacturing has two separate words for quality that are useful here: *atarimae hinshitsu*, meaning the basic, taken-for-granted functionality a product needs to work at all, and *miryokuteki hinshitsu*, an enchanting quality layered on top that has nothing to do with function and everything to do with how a product makes someone feel.

Wufoo's own small touches — a dinosaur icon on the login link that says "RARRR" when you hover over it — put a smile on nearly every new user's face during testing, a reaction the team noticed specifically because they were looking for it. Other companies do versions of the same thing: Vimeo's launch page made subtle sounds when you searched certain words; a sign-up form for the wine-focused social network Cork'd read like a short poem instead of a bland list of field labels; Stripe's documentation automatically filled in a logged-in developer's own API credentials into every code example, saving a small but real piece of friction most companies never bother to remove. MailChimp redesigned its plain help documentation to look like a set of magazine covers, and readership went up while support tickets about the same topics went down — proof that even the least glamorous part of a product (the help pages) is a legitimate place to apply this kind of care.

None of this works, Hale is careful to note, if the underlying functional quality isn't there first. A witty detail bolted onto a product that doesn't actually work reads as tone-deaf,

not charming. Enchanting quality is what you add *after* the taken-for-granted kind is solid, not a substitute for it.

Marriage: the four things every relationship fights about

For the longer-term relationship — an existing customer, month after month — Hale turns to a specific, well-known body of marriage research from psychologist John Gottman. Gottman's famous claim: watching a couple argue for fifteen minutes lets him predict, with roughly 85% accuracy, whether they'll still be together in four years. Extend the observation to a full hour, including a conversation about their hopes and dreams, and the accuracy climbs to about 94% — a level of predictive power that trained marriage counselors, priests, and psychiatrists shown the same tapes couldn't come close to matching by guessing.

Gottman's underlying finding isn't that happy couples never fight. It's that everyone fights about almost exactly the same things: money, kids, sex, time, and a catch-all category of things like jealousy and in-laws. Hale's insight is that this maps directly onto customer support: **money** becomes billing and pricing complaints, **kids** become problems with a customer's own end users or clients, **sex** becomes questions about performance, and the catch-all category becomes competitors, partnerships, and anything else adjacent to the relationship. Customer support isn't a separate function bolted onto the product. It's the same set of recurring relationship stressors every long-term relationship has, just wearing a support-ticket disguise.

Gottman also names four specific behavior patterns — he calls them the Four Horsemen — that predict a relationship's end: **criticism** (attacking the person instead of the issue), **contempt** (deliberate insult), **defensiveness** (refusing to own a mistake), and **stonewalling** (shutting down entirely). The first three show up in companies as they age. The fourth — stonewalling, simply not responding to a customer's problem at all — is, in Hale's direct experience, one of the biggest single causes of early startup churn, precisely because it's the easiest of the four to fall into by accident. A founder buried in product work looks at a growing pile of support emails and decides, implicitly, that responding can wait. That silence is the relationship-ending behavior, not a neutral default.

Support-Driven Development: fixing a feedback loop that breaks by default

Most engineering teams develop a specific, predictable pathology as they grow, and Hale's name for the fix — Support-Driven Development, or SDD — is a direct response to it. Before a product launches, building it feels close to perfect: every decision is yours, every line of code feels correct because nothing has proven otherwise yet. After launch, reality arrives in the form of support tickets, bug reports, and confused users — and the natural instinct for a technical founder is to treat these as lesser, distracting work, and delegate them away from the people who actually built the product as soon as possible.

SDD reverses that instinct on purpose: everyone on the team, including the founders and engineers, does real customer support shifts. Paul English, a cofounder of Kayak, installed a physical red telephone in the middle of the engineering floor that rang directly with customer support calls. When someone questioned paying skilled engineers a high salary to occasionally answer a phone, English's answer was that after the second or third call about the same underlying problem, the engineer fixes the actual bug instead of waiting for someone else to report it through a queue. The fix isn't cheaper support labor. It's putting the people with the power to fix the root cause in direct, unfiltered contact with the consequences of not fixing it.

Hale points to Airbnb as another company that learned this lesson directly, if less visibly: in the company's early years, cofounder Joe Gebbia was frequently seen with a phone headset stuck to his head, doing support calls himself, constantly. The part of that story most people miss, focused as they are on the more famous professional- photography growth hack from the same period, is that Airbnb's growth genuinely started compounding once the founders personally understood how to match support capacity to actual incoming demand — a lesson that came directly out of Gebbia's own time on the phones, not from a dashboard.

Wufoo ran this at real scale: roughly 500,000 registered users, five million people touched by Wufoo forms whether they knew the brand or not, and all of that supported by the same ten-person team, one person on shift per day. Response times held at seven to twelve minutes during the day, about an hour in the evening, and no more than 24 hours on weekends — numbers that most companies with dedicated, much larger support teams struggle to match.

Give people a place for their emotions, and they get more rational

One specific experiment at Wufoo is worth describing in full, because the result surprised the team that ran it. Someone suggested adding an "emotional state" dropdown field to Wufoo's own support form, on a hunch that customers' feelings about a technical problem might matter as much as the technical details — even though there was no way to read facial expressions or tone of voice over a web form. The team expected almost nobody to bother filling it out.

It was filled out 75.8% of the time — nearly as often as the browser-type field, a purely functional piece of information, which was filled out 78.1% of the time. People were, in effect, telling Wufoo directly: how I feel about this problem matters to me almost as much as the technical facts you need to fix it. The unexpected second effect was even more interesting: once customers had a legitimate outlet for their frustration, the tone of their actual support messages measurably calmed down. Wufoo's team tracked the three clearest written signals of strong emotion — exclamation points, curse words, and ALL CAPS — and all three declined once the emotion field existed. Giving people somewhere to put a feeling, it turned out, made them more rational everywhere else in the conversation.

Direct exposure changes what you build, and the "knowledge gap" explains why features aren't always the answer

Jared Spool, a well-known figure in user-experience research, has data supporting a specific, checkable claim: the amount of time a product's builders spend in *direct* contact with real users — not filtered through reports, dashboards, or secondhand summaries — correlates directly with how good the resulting design gets. His threshold: a minimum of two hours, at least every six weeks, or the product's design measurably degrades over time. Wufoo's own engineers were getting four to eight hours of direct user exposure *every week*, far above that floor, largely as a side effect of everyone doing support shifts.

Spool has a second, related concept worth carrying forward: the **knowledge gap** — the distance between what a user already knows and what they'd need to know to use your product well. There are only two ways to close that gap: increase the user's knowledge (better documentation, onboarding, contextual help) or decrease how much knowledge the product requires in the first place (simplification). Hale's observation about where most engineering teams default: adding a new feature almost always *increases* the knowledge gap rather than closing it, because every new feature is one more thing a user

has to learn exists. Wufoo deliberately spent about 30% of its engineering time on the other side of that tradeoff — self-service tools, contextual help links that pointed to the exact relevant documentation instead of a generic help page, and continuously A/B-tested documentation. One single redesign of their documentation cut support volume by 30% overnight — a bigger overnight improvement than most new features ever produce.

The math that makes churn worth taking seriously

Here's the part of this chapter's core idea worth stating with real numbers behind it: a 1% improvement in conversion and a 1% reduction in churn have mathematically identical effects on a company's growth curve. Most founders chase the first number and neglect the second, right up until churn has quietly become the larger problem — even though, according to Hale, reducing churn is very often the cheaper and easier lever of the two to pull.

Wufoo built one specific mechanism aimed directly at churn: a feature they called the "Since You've Been Gone" alert, which compared a returning user's last login date against everything Wufoo had shipped since then, and showed them exactly what was new. It became, by Hale's account, the single most talked-about feature in the entire product — not because it was technically sophisticated, but because it made customers feel a continuous, ongoing sense of value from a subscription whose price hadn't changed. Paired with this: a weekly ritual of writing genuine, handwritten thank-you notes to users, an idea borrowed directly from one founder's memory of his mother making him write thank-you letters as a kid.

That ritual nearly went wrong in its second year, and the failure is as instructive as the success. In year one, with only three founders, the team wrote handwritten holiday cards to every single customer. In year two, with too many customers to repeat that, they switched to writing only to their highest-paying customers, reasoning that this was the more "scalable" version of the same gesture. A long-time, loyal — but not top-paying — customer wrote in, hurt, to say he'd loved the card the first year and was still looking forward to his second one. Hale's lesson from that moment: the best way to exceed expectations is not to set any in the first place. The fix wasn't abandoning the ritual. It was making it continuous and unpredictable — small gestures happening in an ongoing way —

rather than a single annual event that inevitably created an expectation the company then had to either meet for everyone or risk this exact kind of quiet disappointment.

Common Mistakes

- **Adding a witty or playful detail to a product that doesn't work yet.** Enchanting quality only lands once the basic, functional quality is solid — added earlier, it reads as tone-deaf rather than charming.
- **Delegating customer support away from the people who build the product, as soon as it feels like an option.** This breaks the exact feedback loop that made the product good in the first place.
- **Going silent on unresolved customer problems (Gottman's "stonewalling") instead of responding, even slowly.** This is one of the most damaging, and most avoidable, causes of early churn.
- **Chasing conversion rate while ignoring churn,** even though a 1% improvement in either one moves growth by the same amount — and reducing churn is frequently the cheaper lever.
- **Building a new feature to close a "knowledge gap" that a simplification, or better documentation, would have closed more cheaply.**
- **Creating a one-time gesture of appreciation (an annual card, a single special email) without realizing it sets an expectation you'll then have to either maintain for everyone or risk disappointing someone who remembers the first one.**

Tradeoffs and Edge Cases

Does Support-Driven Development scale past a ten-person team? Wufoo never grew past about ten people before its acquisition, so it's fair to ask whether this only works at small scale. The underlying principle — keeping the people with the power to fix root causes in direct contact with the consequences of not fixing them — doesn't have an

obvious size limit, but the specific mechanism (everyone personally answering tickets) clearly gets harder as a company scales past a few hundred people. Chapter 21 covers what changes about customer-facing feedback loops once a company is large enough that most engineers will never personally touch a support ticket — the goal at that scale is usually preserving some structured version of direct exposure (Spool's two-hours-every-six-weeks minimum), not literally rotating every engineer through a support queue forever.

Is remote work a prerequisite for this kind of culture? No, but Wufoo built all of this while fully remote, out of Florida, with a deliberately short workweek (four and a half days, with Friday reserved for meetings and a strict fifteen-minute limit on any single real-time conversation before tabling it for later). That structure isn't required to build products people love — it's one specific, disciplined way one team made deep focus time and customer contact both possible at once, without an office.

Doesn't chasing "customer intimacy" conflict with moving fast? Not in Hale's framing, and it's worth being explicit about why: the *Discipline of Market Leaders* research he cites identifies three separate paths to market dominance — best price (operational efficiency), best product (R&D), and best overall solution (customer intimacy) — and argues a company should pick one and organize itself around it, not try to win all three. Customer intimacy is the path this chapter has been describing, and its practical advantage over the other two is that it requires almost no capital to start pursuing, at any company size — just genuine attention and humility, both of which are available from day one, long before there's money for either better logistics or a bigger R&D budget.



Applying the Test

Pick a real support interaction from your own product, or a hypothetical one if you don't have users yet. Run it through Gottman's four-category lens: is the underlying complaint really about money, about the customer's own users ("kids"), about performance ("sex"), or about something adjacent like a competitor or partner ("others")? Naming the category usually reveals whether you're looking at a one-off problem or a pattern worth fixing at the root.

Then check your own team's response time against the Wufoo benchmark from this chapter — minutes during the day, an hour in the evening, a day on weekends — not because you need to match it immediately, but because knowing the gap tells you whether stonewalling (even unintentional, even just "I'll get to it later") is quietly becoming your biggest source of churn.

✓ Chapter Summary

- Growth is the gap between conversion and churn — both matter equally, and churn is chronically underinvested in because it's less visible than acquisition.
- Acquiring new users is like dating: first impressions (Japanese *miryokuteki hinshitsu*, "enchanted quality") carry outsized weight, but only once basic functionality already works.
- Keeping existing users is like marriage: Gottman's research shows relationships fight about money, kids, sex, and time — which map directly onto the categories of customer support tickets.
- Stonewalling — simply not responding — is one of the most damaging and most avoidable causes of early churn.
- Support-Driven Development (everyone does support, including engineers and founders) fixes the feedback loop that breaks by default once a product launches.
- Giving customers an explicit outlet for emotion (Wufoo's "how do you feel" field) measurably calms down the rest of their communication.
- Direct user exposure (a minimum of two hours every six weeks, per Jared Spool) correlates with better design; adding features usually widens the knowledge gap rather than closing it, and simplification or better docs often closes it more cheaply.
- A one-time gesture of appreciation creates an expectation you'll have to maintain — make ongoing care continuous and unpredictable instead of an annual event.



Exercises

1. **Recall:** Explain why "stonewalling" — simply not responding — is described as one of the most damaging behaviors a startup can exhibit toward its customers, even more than an honest, slow, or imperfect response.
2. **Apply:** Pick one feature you're considering building. Using Spool's knowledge-gap framework, decide whether it closes the gap (makes the product easier to use) or widens it (adds one more thing a user has to learn). What would the closing-the-gap version of the same idea look like instead?
3. **Apply:** Design your own version of Wufoo's "Since You've Been Gone" feature for a product you're building or considering. What would you actually show a returning user, and why would it make them feel like they're getting ongoing value?
4. **Confront a counterexample:** This chapter argues customer intimacy (Support-Driven Development, direct user exposure) is a viable path to market dominance requiring little capital. But Chapter 16 will cover enterprise sales motions that rely heavily on dedicated sales teams and account management rather than founders personally answering tickets. At what point does a company need to shift away from the founder-does-everything model this chapter describes, and what should it try to preserve when it does?



Further Reading

- Kevin Hale's original CS183B lecture goes into further detail on Wufoo's specific support metrics and the "King for a Day" internal product-direction ritual, which this chapter didn't have room to cover in full.
- John Gottman's marriage research, referenced throughout this chapter, is described in much greater depth in his own books on relationship psychology.

- Michael Treacy and Fred Wiersema's *The Discipline of Market Leaders* is the source of the best-price/best-product/best-overall-solution framework covered in this chapter's tradeoffs section.



Chapter 9 — Doing Things That Don't Scale

DoorDash didn't begin with an idea about delivery. It began in a macaroon store. During his junior year at Stanford, Stanley Tang was interviewing small business owners about an entirely different prototype he was working on, and sat down with Chloe, the owner of Chantal Guillon, a macaroon store in Palo Alto, mostly to get feedback on that unrelated project. Chloe brought up delivery on her own, and pulled out what Tang describes as "a really really thick booklet" — page after page of delivery orders she'd had to turn down, because she had no drivers and no way to fulfill them herself without personally delivering every one. Tang and his cofounders followed that single conversation outward: over the next few weeks, they talked to roughly 150 to 200 small business owners, and heard some version of the same complaint from almost all of them — no delivery infrastructure existed, and it was a real, painful gap. That was the moment the idea turned from a hunch into a hypothesis worth testing.

Testing it meant Tang and his cofounders spent an afternoon building a website called PaloAltoDelivery.com. It had no backend, no driver algorithm, and no real infrastructure of any kind. It was a single ugly page listing PDF menus they'd copied from local restaurants, with one phone number at the bottom — their own personal cell phone. They launched it without expecting much of anything.

Someone called. They wanted Thai food. Tang and his cofounder got in a car, bought the food, and delivered it themselves to a stranger on Alpine Road, who turned out to be the author of a book called *Weed the People*. It was, in Tang's own words, the best and worst delivery they could have asked for. The next day, two more calls came in. Then five. Then seven, then ten. That ugly, hour-old landing page became DoorDash.

Nothing about that first delivery scaled. There was no algorithm, no dispatch system, no professional anything. That's exactly the point of this chapter: at the very beginning of a company, the things that work are frequently the things that could never work at a million users — and doing them anyway, deliberately and without embarrassment, is one of the real advantages a small company has that a large one has already lost.



The Core Idea: A Bridge, Not a Business Model

"Doing things that don't scale" describes a specific, narrow category: growth and operations strategies that are genuinely unsustainable — they will not carry you to a million users, and they're not supposed to. Walker Williams, who built Teespring the same way, defines the category by where it breaks: usually time, sometimes money, always eventually. The point isn't that these strategies are secretly efficient. The point is that they're a bridge — the only available way to get from zero customers to enough customers that a real, scalable strategy becomes possible at all.

Every founder eventually has to build the scalable version. The mistake this chapter is about isn't failing to plan for scale. It's giving up the unscalable, ugly, personal version too early — before it's actually stopped being useful — because it starts to feel embarrassing, or beneath the company you're trying to look like.



Why It's True: Testing Fast, Recruiting by Hand, and Building the Wrong Codebase on Purpose

Test the hypothesis before you build anything real

DoorDash's own origin makes the case better than any abstract advice could: before writing a single line of real infrastructure, Tang and his cofounders wanted to know whether people would actually pay for restaurant delivery in Palo Alto at all. Rather than guess, or research it, they built the cheapest possible real test — a landing page with a phone number — and let actual behavior answer the question. No drivers were hired in advance. No dispatch software was written. No assumptions were left unchecked that could be checked in an afternoon instead.

This is worth separating clearly from over-planning. The test wasn't a business plan or a slide deck estimating a total addressable market. It was a live, working (if barely) version of the actual transaction, thrown at the smallest possible real audience, fast enough that the answer came back in days instead of months.

Hack the operations together with whatever's already on hand

Once real orders started arriving, DoorDash's actual operations ran on tools that were never built to run a business. Tang and his cofounders were the delivery drivers themselves, showing up between classes. There was no dispatch software — driver locations were tracked using Apple's Find My Friends, an app built for keeping tabs on your friends, not managing a delivery fleet. Orders were logged in a shared Google Doc. Payments ran through Square, and the volume of small, rapid transactions grew fast enough that Square's own systems flagged the account for suspected money laundering — a problem resolved only because one of the cofounders happened to know people at Square personally.

None of this was a considered strategy. It was whatever actually got food from a phone call to a customer's door that same day. Tang offers one moment from this period that captures its texture well: after wrapping up a meeting with a restaurant partner, the team wanted to try a new ice cream shop called Cream that had just opened nearby. A text came in asking for more drivers on the road immediately. The decision took about ten seconds — they went to deliver, not to get ice cream — and Tang describes it as exactly the kind of tradeoff this entire stage of a company runs on: do the unglamorous thing now, on the promise that scaling far enough eventually buys you the ice cream too.

There is no silver bullet for your first users

Walker Williams is blunt about a specific fantasy nearly every founder has starting out: the belief that somewhere out there is a single, high-leverage channel — a viral mechanic, an ideal partnership, an affiliate deal — that will make the first hundred users appear with minimal effort. In his experience talking to founders across many companies, that channel essentially doesn't exist. Even companies that look, from the outside, like they had an effortless growth curve almost always had a period where the first users were "impossibly hard to get."

Teespring's own beginning bears this out: in 2012, closing a sale meant days of back and forth, free custom designs, and multiple rounds of revision, all to sell about fifty shirts to a single nonprofit for roughly a thousand dollars in total revenue. Williams's metaphor for this stage is pushing a boulder up a smooth hill: the incline is steepest, and the effort required per unit of progress is highest, right at the very start. It gets easier as you go —

but only if you push through the beginning instead of waiting for it to get easier on its own.

One specific trap to avoid at this stage, despite how tempting it looks: don't give the product away for free to manufacture early traction. Free users behave differently from paying ones in ways that create a false sense of security — a founder can convince themselves that a free user will convert later, without ever testing whether that's true. Early users need real, personal effort from the founder — cold emails, phone calls, personal networks — not a price cut standing in for that effort.

Turn early users into champions, and treat every failure as an opportunity to prove it

Getting a user is only half the job. The other half is turning them into someone who actively tells other people about you — what Williams calls a champion. The most reliable, if unglamorous, way to create champions is talking to users constantly and personally. Williams, even once Teespring had scaled well past its earliest days, continued to personally answer ten to twenty customer support tickets a day and read through the company's entire social media mentions every night — not because it scaled, but because it was the only way to actually understand what was breaking for real customers.

There are three specific channels for this kind of contact worth naming: running support yourself in the early days (Williams and his cofounder handled all of Teespring's support personally until the company was doing about \$130,000–\$140,000 a month in revenue), proactively reaching out to customers who've already left rather than only focusing on new ones, and actively monitoring social media and online communities for both praise and complaints. The asymmetry between the last two matters: a single, visibly unresolved complaint can undo the goodwill of ten genuine champions, simply because a detractor is louder and more memorable than nine quiet, satisfied customers.

The instinct when something goes wrong — a botched order, a mistake affecting a large share of a month's revenue — is to minimize it, or explain it away as only partially wrong. Teespring had real stretches early on where a bulk order came out with the wrong colors or the wrong size, wrong enough that the ruined order alone amounted to half of the company's entire GMV for that month. Teespring's own experience was the opposite lesson: customers who started out the most frustrated by a real mistake, once that mistake was made right without excuses, frequently became the company's longest-term

and most vocal champions. Making a real mistake fully right, rather than defending it, is one of the more reliable ways to create a champion out of someone who had every reason to become a detractor instead.

The product you launch with is not the product that scales

Once early demand is real, a second kind of "doesn't scale" work becomes necessary: the product itself, as built, usually cannot survive contact with real growth, and the engineering instinct to build it properly the first time can actively slow a company down. Teespring's own example: a handful of larger nonprofit customers wanted enterprise features the core product didn't have. Building those features the "right" way — cleanly integrated into the existing platform — was estimated at about a month of engineering time, which in a fast-moving startup is close to a year in ordinary time. Instead, the company's CTO duplicated the entire codebase and database, built a separate, disposable version of the product specifically for those customers, and shipped it in three to four days. The features that turned out to matter were folded back into the core product later, once real usage had shown which ones were actually worth keeping.

The underlying discipline here: worry only about the next order of magnitude, not the final one. A company with ten users shouldn't be designing for a million; it should be figuring out how to serve a hundred. Teespring went through real stretches where the site crashed every single night, and the team slept with their phones on loud specifically so they could wake up and restart the server — a genuinely painful, unsustainable practice, and also, in Williams's own accounting, worth it, because it bought speed at a stage where speed mattered more than stability.

The closing point Williams makes is the one this whole chapter is really about: there is no natural milestone — not a funding round, not a revenue number — at which "doing things that don't scale" is supposed to end on its own. The unscalable, personal version of doing business is a genuine competitive advantage for as long as a company can keep doing it, precisely because a smaller, hungrier competitor can still do it even after you've grown past it. Williams's own framing: you shouldn't give it up voluntarily. It should have to be taken from you, by growth that's outpaced your ability to keep doing it by hand — not surrendered early because it started to feel beneath the company you're trying to look like.

Common Mistakes

- **Waiting for the perfect growth channel instead of manually finding the first customers yourself.** Williams's experience across many companies: that channel essentially never exists for the first users, no matter how it looks from the outside later.
- **Giving the product away for free to manufacture early traction.** This creates a false sense of momentum and doesn't test whether anyone would actually pay.
- **Explaining away or minimizing a real mistake instead of making it fully right.** Teespring's most frustrated early customers, once treated well, became some of its strongest champions — the opposite would likely have happened had the mistake been downplayed.
- **Building the "correct," fully scalable version of a feature before you know it's worth building at all.** Teespring's duplicated-codebase shortcut shipped in days what a proper integration would have taken a month to build, for a feature whose long-term value wasn't even confirmed yet.
- **Designing for the size you hope to be, instead of the size you actually are.** Optimize for the next order of magnitude, not the final one.
- **Abandoning manual, personal, unscalable work once it starts to feel embarrassing,** rather than continuing it until growth genuinely makes it impossible to keep doing.

Tradeoffs and Edge Cases

Isn't this the same lesson as Chapter 7's "manual before automated"? Essentially yes, and it's worth naming the connection directly rather than treating them as unrelated: Homejoy's manual cleaner-vetting process (Chapter 7) and DoorDash's founder-driven deliveries are the same underlying principle — expressed by two different speakers, in two different lectures, about two completely different businesses. That repetition across independent sources is itself evidence the principle is real rather than a one-company anecdote. The angle is slightly different in each case: Chapter 7 frames manual work as a

learning mechanism (you discover what to automate by doing it by hand first). This chapter frames it as a *growth and resilience* mechanism (unscalable effort is a genuine, temporary competitive advantage). Both are true at once, and both usually apply to the same early-stage decisions.

When does "doing things that don't scale" actually have to stop? Not at a specific funding round or revenue milestone, per Williams — but genuinely, eventually, once growth outpaces what founders and a small team can keep doing by hand. The signal isn't a calendar date. It's the same kind of signal Chapter 7 described for automating a manual process: once you've learned enough, from doing something by hand, to know exactly what the automated or delegated version should look like, that's the right time to build it — not before, and not long after either.

Does this conflict with Chapter 19's emphasis on building systems and dashboards?

Not really — they apply at different stages, and the difference matters. This chapter is about a company's earliest days, before there's enough signal to know what a good system should even look like. Chapter 19 assumes a company has already learned those lessons and is now building the durable infrastructure — dashboards, delegation, editing rather than writing — to operate at a much larger scale. Skipping straight to Chapter 19's systems thinking without first doing the unscalable, hands-on version this chapter describes tends to produce systems built on guesses instead of real, hard-won evidence.



Applying the Test

Take an idea you're considering, and design the ugliest, fastest, most personally labor-intensive version of a real test for it — the way PaloAltoDelivery.com tested DoorDash's core hypothesis in an afternoon. What's the equivalent of a landing page with your personal phone number at the bottom? What would you personally have to do, by hand, for the first ten customers, that a "real" version of the business would never ask a founder to do forever?

Then ask the harder follow-up question: if that unscalable version actually worked, would you be tempted to hand it off too early because it felt beneath what you were building — or would you keep doing it until growth genuinely forced you to stop?

Chapter Summary

- "Doing things that don't scale" describes deliberately unsustainable early strategies — a bridge to real traction, not a permanent operating model.
- Test a hypothesis with the cheapest possible real version of the product, the way DoorDash tested restaurant delivery with a landing page and a personal phone number.
- Early operations can run on borrowed consumer tools (Google Docs, Find My Friends, Square) — the goal is getting the transaction done today, not building it properly yet.
- There is no silver-bullet acquisition channel for your first users — expect to recruit them by hand, and don't give the product away for free to fake early traction.
- Turn early users into champions through constant, personal contact, and by making real mistakes fully right rather than minimizing them — a single unresolved complaint can undo many champions' worth of goodwill.
- The product you launch with is rarely the product that scales — optimize for speed, not clean architecture, until you know which features are actually worth keeping.
- Worry only about the next order of magnitude, not the final one.
- There's no natural milestone at which unscalable work is supposed to end voluntarily — it should be taken from you by growth, not given up because it feels beneath the company you're trying to look like.



Exercises

1. **Recall:** Explain why Walker Williams argues there's no natural milestone — no funding round, no revenue number — at which a founder should voluntarily stop doing things that don't scale.
2. **Apply:** Design the cheapest, fastest, most personally labor-intensive real test you could run this week for an idea you're considering — your own version of PaloAltoDelivery.com.
3. **Apply:** Describe a real mistake (yours or a company you've used as a customer) that was either minimized or fully made right. What was the actual difference in how you, as the customer, felt afterward?
4. **Confront a counterexample:** This chapter argues you shouldn't give up unscalable, personal work voluntarily. But Chapter 19 will argue that building systems and delegating is essential once a company grows. How would you know, in the moment, whether you're avoiding a system you genuinely need — or correctly holding onto hands-on work you haven't outgrown yet?



Further Reading

- Paul Graham's essay "Do Things That Don't Scale" is the origin of this chapter's title and covers additional examples beyond what Stanley Tang's and Walker Williams's lectures had time for.
- Chapter 7 covers the same underlying principle from a product-development angle (Homejoy's manual cleaner-vetting process) rather than this chapter's growth-and-resilience framing.
- Chapter 10 picks up immediately after this chapter, once a company's earliest, by-hand traction has to turn into a real, measurable retention curve.

- Chapter 19 covers the systems and delegation practices that eventually replace the unscalable, hands-on work this chapter describes, once a company has genuinely outgrown it.



Interlude — Building Hardware in a Software World

This is an optional interlude. It covers hardware and connected-device products specifically — a narrower case than the software-only startups the rest of this book focuses on. If you're building pure software, you can skip this and go straight to Part Four. If you're building anything with a physical component — a wearable, a connected device, anything in the "internet of things" space — this is for you.

In 2010, the market for wireless speakers was, by Hosain Rahman's own account, effectively zero percent of the overall speaker market. Bluetooth speakers barely existed as a category. By Christmas of 2013, three years later, wireless speakers were 78% of the entire speaker market — a category that had existed in roughly its current form since the 1950s, turned upside down in three years. Rahman's company, Jawbone, built the product — the Jambox — that helped create that shift.

Here's the detail worth sitting with: if you had asked people in 2010 whether they wanted a \$199 wireless speaker for their phone, when a phone already had a speaker built in for free, almost nobody would have said yes. That's not a hypothetical — Rahman is direct that this is exactly what would have happened, and it's the same trap this book has already named twice. Rahman's own illustration of the right way to ask: don't ask someone "do you want a digital portable music player," ask "if you could put a thousand songs in your pocket and take them anywhere, would that be cool?" Emmett Shear's Twitch interviews (Chapter 7) don't ask people if they'd use a proposed feature, because the honest- sounding answer is nearly worthless. Hardware runs into this same problem, just with higher stakes, because a bad guess costs months of tooling and manufacturing, not a wasted sprint.



The Core Idea: Hardware, Software, and Data, as Three Equal Legs

A hardware startup — especially anything connected, sensor-driven, or "smart" in the now-familiar sense — has to be genuinely excellent at three different disciplines at once, not one. **Hardware:** a physical object good enough that someone will actually keep it on or nearby all the time, since a wearable left in a drawer collects no data and solves no problem. **Software:** an application layer with the same bar for engagement as a pure software company's product — Rahman's own comparison point is Instagram and WhatsApp, not other hardware companies. **Data:** the ability to actually do something useful with everything the hardware is sensing, rather than just displaying numbers back at the user.

Most companies are naturally strong at one of these and weak at the other two — people who build great hardware tend to come from a mechanical and electrical engineering background, with a completely different skill set and completely different production cycle than people who build great software. Combining the two into one team, Rahman is candid, creates real internal friction: hardware teams have to learn to move faster than their instincts want to, and software teams have to learn to think through problems more completely before shipping, since a hardware bug can't be patched with a Friday-afternoon deploy.



Why It's True: The WHYs Framework, and Asking About Behavior Instead of Features

The WHYs: a single question that governs every product decision

Jawbone's internal process for deciding what to build centers on a question they call the WHY: what specific user problem does this solve, such that once it's solved, the user genuinely can't live without it. Every product decision gets tested against this question repeatedly, at every stage, rather than being decided once at the start and assumed to hold.

This connects directly to a lesson this book has already covered from a completely different industry: you cannot get a reliable answer to the WHY question by asking users directly. The parallel to Chapter 2's "one sentence" test and Chapter 7's "ask about

behavior, not features" is exact, and Rahman states the hardware version of it plainly: the questions that actually produce insight aren't "would you pay for this," but questions about existing behavior — how often do you listen to music around other people, do you use headphones or a speaker, how often do you want a personalized versus a shared experience. The underlying problem the product solves gets found by understanding behavior deeply enough to notice the gap yourself; it doesn't get handed to you by a user filling out a survey about a feature that doesn't exist yet.

Track, understand, act: turning sensors into a reason to keep wearing something

Jawbone's wearable health products (UP, then UP24) organize around a three-part framework worth borrowing for any sensor-based product: **track** (the hardware and sensor layer — batteries, materials, the physical habit of actually keeping the device on), **understand** (turning raw sensor data into something a person can actually interpret — a heart rate of 75 means nothing on its own; whether it's good or bad depends entirely on context), and **act** (turning that understanding into a concrete behavior change, like a well-timed notification suggesting a workout at the time of day the data shows it actually helps that specific person sleep better).

Data on its own, without the "understand" and "act" layers built on top of it, isn't a product. It's a stream of numbers a user will stop looking at within a week. The discipline this framework enforces is refusing to ship the "track" layer alone and calling it done.

Resolving tradeoffs across an entire system, not one team at a time

A hardware product is a system in a more literal sense than most software products are: a decision in battery design affects sensor performance, which affects signal reliability, which affects what the software layer can promise a user. Rahman's account of building a later-generation wearable (UP3) with an entirely new sensing system involved daily three-hour calls between hardware, sensing, and software engineers, for months, specifically to negotiate these tradeoffs in the open rather than let one team make a decision in isolation that quietly broke another team's plan. This is slower and more painful than most software-only development, and it's not optional — a hardware product that ships with an

unresolved tradeoff between, say, battery life and sensor accuracy can't be patched the way a software bug can.

Common Mistakes

- **Asking users directly whether they'd want or pay for a proposed hardware feature.** The Jambox itself would have failed this test in 2010 — nobody asks for "a thousand songs in your pocket" or "a \$199 phone speaker" until they've experienced why it matters.
- **Treating hardware, software, and data as separate teams with separate roadmaps,** rather than one integrated system where a decision in one area constrains the others.
- **Shipping the sensor ("track") layer of a product without the "understand" and "act" layers,** and mistaking a stream of raw data for a finished product.
- **Letting one team resolve a system-wide tradeoff in isolation,** rather than negotiating it openly across hardware, software, and data teams before committing to it.

Tradeoffs and Edge Cases

Should a software-focused founder ever choose the hardware path in the first place? This deserves an honest answer, not just enthusiasm for the category. Hardware development cycles are dramatically slower than software — tooling alone can take sixteen weeks per iteration, compared to a software team that can ship and learn from a change within a day. Chapter 17's advice about speed, momentum, and rapid iteration still applies conceptually, but the unit of iteration is months, not days, which changes the entire cadence of a company. Chapter 2 already flagged that some categories genuinely require more capital and a longer runway before they work at all; hardware is usually one

of them. This interlude isn't arguing you should build hardware — it's covering how to do it well if the idea genuinely requires a physical product, which some real, valuable ideas do.

Does the WHYs framework replace the monopoly and market-sizing arguments from earlier chapters? No — it operates one level down. Chapters 2 and 3 are about *which* market and business to build toward. The WHYs framework is about deciding *what specific product decision* serves that strategy, once the strategic question is already answered. A hardware company still needs a real answer to "why now" and a plan for durability, the same as any other startup this book covers.



Applying the Test

Take a hardware or connected-device idea you're considering. Write down the direct version of the pitch — "would you pay for X feature" — and then rewrite it the way Rahman's team would: as a question about existing behavior instead. What do people already do, physically, around the problem your device addresses? Then check your idea against the track-understand-act framework: does it just track something (a number with no context), or does it also help a user understand what that number means and act on it in a way that changes their behavior? A product stuck at "track" is a sensor, not yet a reason for anyone to keep wearing it.



Chapter Summary

- A hardware company needs to be excellent at three disciplines at once — hardware, software, and data — not just one, and combining all three on one team creates real, manageable friction between different working cultures.
- The WHYs framework tests every product decision against one question: what problem does this solve that makes it something a user can't live without.
- You cannot get a reliable answer to that question by asking users directly whether they'd want a feature — ask about existing behavior instead, the same lesson Chapters

2 and 7 already established from completely different industries.

- Track, understand, act: sensor data alone isn't a product. It becomes one once a user can understand what the data means and act on it to change their behavior.
- Hardware tradeoffs have to be resolved across the whole system, in the open, because one team's decision constrains what every other team can promise.
- Hardware's iteration cycle is measured in months, not days — a real, structural difference from the software-only startups this book otherwise focuses on.



Exercises

1. **Recall:** Explain why asking "would you pay for this hardware feature" produces an unreliable answer, using the Jambox's own 2010 launch as your example.
2. **Apply:** Take a hardware or connected-device idea and run it through the track-understand-act framework. Which layer does your current concept actually reach, and what would the next layer require?
3. **Apply:** Pick a sixteen-week tooling cycle constraint (real or assumed) for a hardware idea you're considering, and sketch what you'd need to validate through software or a non-manufactured prototype before committing to that cycle — the same discipline this interlude argues Rahman applied before shipping the Jambox.
4. **Confront a counterexample:** This interlude argues hardware requires a fundamentally slower iteration cycle than software. Are there hardware categories where this isn't true — where iteration can happen nearly as fast as software — and what would need to be true about the product for that to work?



Further Reading

- Hosain Rahman's original CS183B lecture covers Jawbone's full development process — exploration, validation, concept, and development phases — in more detail than this

interlude's condensed version.

- Chapter 2 and Chapter 7 cover the "ask about behavior, not features" principle this interlude borrows, from a software-only perspective.



Chapter 10 — Retention Is the Root of Growth

When Alex Schultz joined Facebook's advertising team, the product he was hired to grow was three days old. Within 90 days of that product launching — not a year, ninety days — his team used early cohort data to predict what an advertiser's value would be to the company over their entire first year. When the real number came in twelve months later, the prediction had been accurate to within 97%.

That kind of forecast sounds like it should require years of data. It doesn't, because of a specific technique this chapter is built around: instead of waiting to see what happens to users over a year, you look at how every user behaves in their first days and weeks, and let that early curve tell you where things are headed. This chapter is about that technique, and about the deeper claim underneath it — that retention, not acquisition, is the actual foundation everything else in growth gets built on.



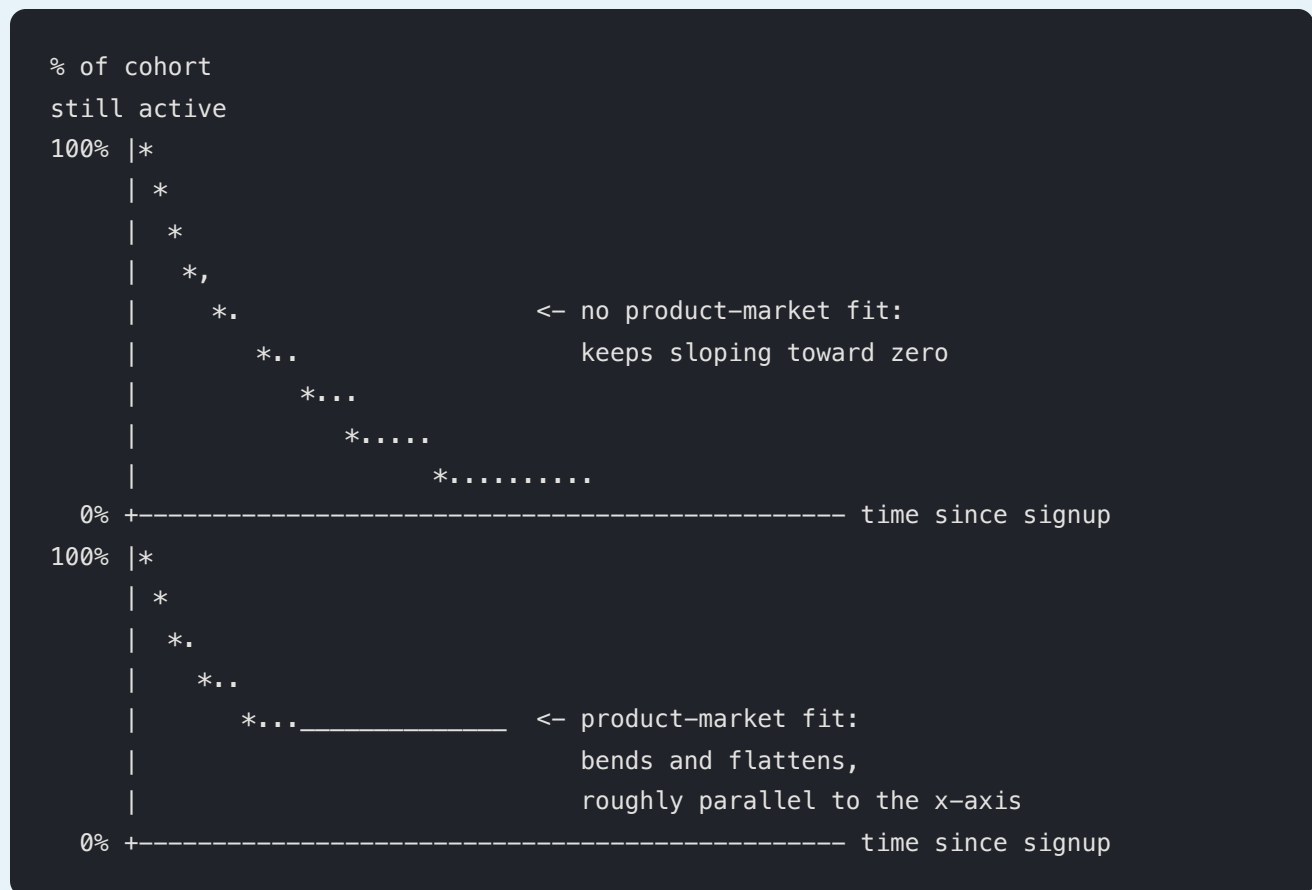
The Core Idea: Product, Then Customers, Then Retention — In That Order

Ask a room full of founders what matters most for growth, and most eventually arrive at some version of the right answer, just not in the right order. A great product gets you customers. But what determines whether those customers turn into real growth is whether they **stay** — and whether, by staying, they tell other people. Schultz's own summary: retention is the single most important thing for growth, full stop.

The practical tool for measuring this is a **retention curve**: on one axis, the number of days since a user first showed up; on the other, what percentage of users from that starting cohort are still active. Every product's curve starts at 100% on day one and declines from there — the question that matters is *how* it declines. A curve that keeps sloping down toward zero, no matter how long you wait, describes a product without real product-

market fit. A curve that bends and flattens out — becoming roughly parallel to the x-axis instead of continuing toward zero — describes a product that has found a real, durable core of users who aren't leaving. Adora Cheung, building Homejoy in a completely different industry, arrived at the identical tool from the opposite direction, using it to track cohort-by-cohort retention rather than growth-team forecasting. Two speakers, two industries, one identical curve — which is itself decent evidence the underlying pattern is real rather than a growth-team fashion.

Figure 10.1 — Two retention curves, and only one of them is a business



The instruction that follows directly from this: if your own curve hasn't flattened yet, do not hire a growth specialist, do not invest in virality mechanics, and do not worry about acquisition channels. Go back to Chapter 2 and Chapter 3's foundations — product-market fit has to exist before growth tactics have anything to amplify.



Why It's True: What Good Retention Looks Like, North Stars, and the Marginal User

Estimating what "good" even means, with almost no data

A common, reasonable-sounding question founders ask is some version of: is 5% monthly retention good? Schultz's answer is that you can usually figure this out yourself with publicly available numbers, using the same kind of rough, order-of-magnitude reasoning that let physicists estimate the yield of a nuclear test from a single published photograph, using nothing but the visible dimensions of the blast and some basic physics. You don't need precision. You need a ballpark.

Applied to a real product: if there are roughly 2 billion people online worldwide, and Facebook has around 1.3 billion active users, that ratio alone tells you Facebook has captured a very large share of everyone who could plausibly use it — a signal, on its own, that its retention must be extraordinarily strong. The same trick works for WhatsApp, which had around 600 million active users, against the global smartphone population, or Amazon against the population of its home market. None of these give you an exact figure. All of them tell you whether you're in the right neighborhood.

The right neighborhood, importantly, is different for every category. Retaining 20–30% of users on a monthly basis is a genuinely strong result for an e-commerce business. That same number would be a catastrophic result for a social network, where the earliest cohort of users typically needs to stay around 80% retained for the product to have any real chance of becoming huge. Benchmarking your own retention against the wrong category's expectations — comparing a social product's numbers to an e-commerce company's, or the reverse — will make a genuinely fine result look like a failure, or a genuinely bad one look acceptable.

The North Star metric, and why it matters more once you're not in the room

Once retention shows a real, flattening curve, the next problem is coordinating an entire company's day-to-day decisions around growing it. Schultz's advice here is deliberately contrarian: a startup shouldn't build a separate "growth team" walled off from the rest of

the company. The whole company should function as the growth team, and the CEO — not a specialized hire — should personally be the one setting the target everyone else optimizes toward.

That target is what Schultz calls a **North Star metric**: a single number, chosen deliberately, that the entire company holds itself accountable to. Facebook's version was monthly active users, not total registrations — a specific decision by Mark Zuckerberg to measure and publish the number that actually reflected engagement, rather than the larger, more flattering number of everyone who'd ever signed up. WhatsApp measured messages sent. Airbnb measured nights booked. eBay measured Gross Merchandise Volume — the total value of goods moving through the platform — rather than eBay's own revenue, since GMV was the number that actually captured how much commerce the platform was enabling: eBay's GMV consistently ran at roughly **10 times** its reported revenue, so a metric anchored to revenue alone would have badly understated the business. eBay's own leaders — Pierre Omidyar, Meg Whitman, and John Donahoe, across their respective tenures — consistently pointed back to that GMV figure rather than letting revenue stand in for it.

The reason a single, clearly chosen metric matters this much connects back to an idea this book already covered from a completely different angle: Patrick Collison's observation, in Chapter 6, that culture is a solution to a bandwidth problem — the thing that keeps decisions aligned once a founder can't personally be in every room. A North Star metric does the identical job for growth decisions specifically. You cannot control what a hundred different employees each decide to prioritize on a given afternoon. What you can do is make sure they're all optimizing toward the same specific number, so that even a decision made without you in the room tends to push in the right direction.

eBay's own affiliate program is a clean, concrete illustration of how much the *choice* of metric shapes behavior. The company originally paid affiliates for every **confirmed registered user** (CRU) they brought in — someone who signed up and verified their account. Switching the payment model to reward **activated** confirmed registered users (ACRUs) — people who went on to actually bid on or list an item — caused total CRUs to drop by about 20% overnight. But ACRUs, the number that actually mattered, dropped by only about 5%, and then began growing faster than before. The mechanism: paying for raw registrations sent affiliates traffic straight to a generic sign-up page, with no compelling reason to complete it. Paying for activation sent that same traffic to an actual

search results page — showing someone searching for a specific item exactly what they wanted — which is what Schultz calls the **magic moment**: the specific point where a new user genuinely understands, viscerally, what your product is for.

Every product has its own magic moment, and identifying it precisely is worth real effort. Facebook's is seeing a friend's face in your feed for the first time — which is why the company measured and optimized hard for getting new users to ten friends within their first fourteen days, treating that specific number as a leading indicator of whether the magic moment had landed. Airbnb's is finding the one listing you actually want to stay in. eBay's is finding the exact collectible you've been looking for. None of these are abstract — they're the specific instant a new user stops evaluating your product and starts using it because they want to.

Optimize for the user who's about to leave, not the user you already have

A subtle, easy-to-miss mistake Schultz calls out directly: when a company thinks about its own product, it naturally thinks from the perspective of its most engaged users — the people already using it constantly, the people most like the founders themselves. Growth decisions made from that perspective are frequently wrong, because the most engaged users are not the ones whose behavior determines whether the company keeps growing.

His example is notifications. Power users complain about getting too many. The founder's instinct, hearing that complaint, is to reduce notification volume across the board. But the growth-relevant question isn't "are my best users annoyed" — it's "is the person who's about to churn getting the one notification that would have brought them back." Schultz's own growth-accounting framework at Facebook broke users into three buckets: new users, resurrected users (people who'd been gone for 30+ days and came back), and churned users. Once a product has been around for a few years, resurrected and churned users dominate the numbers far more than new signups do — and the users in both of those categories reliably had low friend counts and had never really been connected to the product's magic moment in the first place. The fix wasn't reducing notifications for everyone. It was increasing the *specific* notifications that helped low-connection users reach the magic moment, while leaving already-engaged power users alone.

Common Mistakes

- **Investing in growth tactics before the retention curve has actually flattened.** Virality, paid acquisition, and dedicated growth hires all amplify a product that already works. None of them fix a product that doesn't.
- **Building a separate "growth team" instead of making growth the whole company's job, led personally by the CEO.**
- **Benchmarking retention against the wrong category.** A perfectly healthy e-commerce retention rate looks catastrophic by social-network standards, and the reverse is just as misleading.
- **Choosing a North Star metric that flatters the company rather than one that reflects real engagement** — total registrations instead of monthly active users, revenue instead of the volume of value actually flowing through the platform.
- **Designing growth decisions around your own experience, or your most engaged users', instead of the marginal user who is about to leave.**

Tradeoffs and Edge Cases

What if the retention curve looks ambiguous — not clearly flattening, but not clearly failing either? This is a real, common case, and Chapter 7 already covered the practical answer from a different angle: Cheung's advice was to set a specific, honest growth plan in advance and treat three or four consecutive weeks of no improvement as the signal to reconsider, rather than waiting for total clarity that may never arrive. The retention curve and the growth-plan check are two views of the same underlying question, and it's worth using both together rather than either alone.

Does every company need exactly one North Star metric, forever? Not necessarily forever — a company's North Star can and sometimes should evolve as the business matures. What matters more than permanence is clarity at any given time: everyone in the company, at any moment, should be able to name the same single metric if asked, the way

eBay's leadership consistently pointed to Gross Merchandise Volume rather than letting different teams quietly optimize for revenue, registrations, or listings instead.

Isn't optimizing for the "marginal user" in tension with building a great product for your best users? Not in practice, though it can feel that way. Chapter 8 already established that a great product requires deep, direct engagement with real users — that's still true, and still necessary for product quality. The distinction Schultz is making is narrower: product *quality* decisions can reasonably lean on your most engaged users' feedback, but *growth* decisions specifically — the ones about who to notify, re-engage, or push toward the magic moment — need to be evaluated from the perspective of users who haven't gotten there yet, not the ones who already have.



Applying the Test

Using only public information, make a rough estimate of your own product's plausible retention ceiling the way Schultz estimated Facebook's: what's the total realistic population who could ever use what you're building, and how does your current active user count compare to it? Then identify your product's magic moment as specifically as you can — the exact instant a new user would say "oh, *that's* what this is for" — and check honestly whether your onboarding gets a new user there quickly, or buries it behind several steps that have nothing to do with it.



Chapter Summary

- Retention is the actual foundation of growth — a great product produces customers, and whether those customers stay is what determines whether the company actually grows.
- A retention curve that flattens (asymptotic) signals real product-market fit. A curve that keeps declining toward zero signals its absence — Cheung's cohort-analysis

method from Chapter 7 and Schultz's growth-forecasting method arrive at the identical tool independently.

- If your retention curve hasn't flattened, don't invest in growth tactics yet — fix the product first.
- What counts as "good" retention differs by category — 20–30% monthly is strong for e-commerce; a social product typically needs closer to 80% in its earliest cohort.
- A single, clearly chosen North Star metric — not a separate growth team — keeps a whole company's decisions aligned even when the founder isn't in the room, the same role culture plays in Chapter 6.
- What you choose to measure and pay for shapes real behavior — eBay's shift from paying for registrations to paying for activated users changed what its affiliates actually did.
- Every product has a specific "magic moment" worth identifying precisely and driving new users toward as fast as possible.
- Growth decisions should be optimized for the marginal user who's about to leave, not for your own experience or your most engaged users.



Exercises

1. **Recall:** Explain why a retention curve that keeps sloping toward zero is a signal to stop investing in growth tactics rather than a signal to invest harder in them.
2. **Apply:** Identify your own product's (or a product idea's) magic moment as specifically as you can. What does your current onboarding flow actually do to get a new user there quickly, and what's standing in the way?
3. **Apply:** Choose a North Star metric for a real or hypothetical product, and explain why it's a better choice than the two or three other metrics you considered and rejected.
4. **Confront a counterexample:** This chapter argues against having a dedicated growth team. But many well-known, successful companies do have dedicated growth teams and growth hires. Under what conditions does a dedicated growth team make sense,

and how would you tell whether your company has actually earned the right to have one yet?



Further Reading

- Alex Schultz's original CS183B lecture covers additional growth-accounting detail (the new/resurrected/churned framework) beyond what this chapter had room for.
- Chapter 11 picks up directly where this chapter leaves off, covering the specific channels and tactics (virality, SEO, paid acquisition) that only work once retention is already solid.
- Chapter 6's discussion of culture as "a solution to a bandwidth problem" is worth rereading alongside this chapter's North Star metric section — they're the same underlying mechanism applied to two different problems.



Chapter 11 — Growth Channels and Tactics

When Hotmail launched, several well-funded competitors were already running television ads and billboard campaigns for free webmail. Hotmail had far less money. What it had instead was a single line of text, appended automatically to the bottom of every email its users sent: "Sent from Hotmail. Get your free email here." No production budget, no media buy — just a sentence that rode along on every message a user was already sending anyway. Hotmail outgrew its better-funded rivals.

That single footer line is a clean example of a channel doing exactly what a channel is supposed to do: amplify something that already works. Chapter 10 established that retention has to come first. This chapter picks up from there — assuming a retention curve that actually flattens — and covers the specific channels and tactics that turn a working product into a growing one: virality, search, messaging, international expansion, and paid acquisition.



The Core Idea: Three Kinds of Growth, One Question Underneath All of Them

Adora Cheung, building Homejoy, organizes growth into three categories: **sticky** growth (getting existing customers to come back and spend more), **viral** growth (existing customers bringing in new ones), and **paid** growth (spending money directly to acquire customers). Every tactic in this chapter is a specific implementation of one of these three.

The question that determines whether any of them is worth pursuing is the same one underneath all three: is this **sustainable**? A viral loop with no retention behind it produces a spike that evaporates. Paid acquisition that costs more than a customer is ever worth produces a business that loses money on every sale, no matter how much revenue it books. Sticky growth is the safest of the three by construction, since it's just retention

(already covered in Chapter 10) compounding on itself — but it's also the slowest, which is exactly why founders reach for the other two.

Why It's True: The Mechanics of Each Channel

Virality has a formula, and it explains why different products go viral differently

Sean Parker's framework, adopted at Facebook, breaks virality into three multiplied factors: **payload** (how many people a single "viral blast" can reach), **conversion rate** (what fraction of those people actually sign up), and **frequency** (how often the blast happens). Multiply enough of a user base through those three factors and you get a **K-factor** — roughly, how many new users each existing user brings in. A K-factor below 1 means the mechanism alone will eventually flatline. Above 1, and it compounds.

The formula also explains why superficially different products can each be "viral" in completely different ways. Hotmail had a low payload (one email at a time) but very high frequency (people email constantly) and very high conversion (anyone still tied to a slow, ISP-locked email address had a real, immediate reason to switch). PayPal had low payload and low frequency, but an extremely high conversion rate on both sides of its two-sided market: telling a seller "I'm going to send you money this way" converts almost automatically, and offering a new user real cash (\$10 for signing up, \$10 for the friend who referred them) converts nearly as well. Facebook, notably, **wasn't** viral in either of these senses — it had no built-in mechanism for emailing someone outside the network the way Hotmail or PayPal did. It grew through old-fashioned word of mouth, driven by a product good enough that people talked about it anyway, which is a reminder that virality is one path to growth, not the only one, and the retention work from Chapter 10 is what made the word-of-mouth version possible at all.

The K-factor calculation itself compounds a chain of conversion rates — a framework credited to Ed, who ran growth at Uber after doing the same job at Facebook, extending Parker's payload/conversion/frequency model into a concrete funnel: of everyone who imports their contacts, how many send invites; of those invited, how many click; of those

who click, how many sign up; of those who sign up, how many go on to import their own contacts and repeat the cycle. Worked through with real numbers: if 100 people get an invite per person who imports their contacts, and 10% of those invited click, and 50% of those who click sign up, and only 10–20% of new signups go on to import their own contacts, the resulting K-factor lands somewhere around 0.5–1.0 — not quite viral on its own.

Figure 11.1 — The invite-chain loop Ed's framework compounds into a K-factor

```
imports contacts --> sends invites --> invite is clicked --> recipient signs up
  ^      (payload: e.g. 100           (conversion:           |
  |      people per importer)         10% click,           |
  |                                   then 50% of           |
  |                                   clicks sign up)        |
+----- some of these new signups import their own -----+
               contacts too (only 10–20%) and repeat the loop

K-factor = payload x conversion (click) x conversion (signup)
          x share of signups who repeat the loop
```

A cautionary case worth naming directly: some products, like the short-video app Viddy, pushed their K-factor comfortably above 1 through aggressive prompts and still collapsed, because none of that viral growth was backed by real retention underneath it. Virality without retention doesn't just under-deliver — it can actively mislead a team into believing they've solved a problem they haven't touched.

Search: the wrong keyword, and the power of one navigation change

Search engine optimization breaks into three parts worth separating clearly. First, **keyword research** — and getting this wrong is a specific, avoidable failure mode. Schultz describes spending a full year optimizing an early cocktail website to rank for "cocktail making," dominating that exact term completely — only to discover that, in the UK market he was targeting, "cocktail making" was searched about **500 times a month**. Ranking first for it delivered roughly **400 visitors a month**. The much larger search volume was for "cocktail recipes" (or "drink recipes" in the U.S.) — a completely different phrase describing functionally the same intent. Winning the wrong keyword perfectly is still

losing, and the gap between those two numbers is the whole lesson: a year of effort had bought a rounding error of traffic.

Second, **links and authority** — the mechanism underneath most of what Google's ranking algorithm rewards. The clearest illustration of how much this can matter: early in Facebook's public-profile push, the company had built public profile pages, but they were buried behind several clicks with no direct path in from anywhere on the site. Google's crawlers, unable to find an efficient way to those pages, effectively ignored them. Adding a single directory page that linked to public profiles more directly — one structural change — multiplied SEO traffic to those pages roughly a hundredfold. Third, basic technical hygiene (sitemaps, correct headers) matters, but it's the least differentiating of the three; most of the real leverage sits in doing the first two well.

Messaging channels reward relevance, not volume

Email, SMS, and push notifications share a set of rules, starting with **deliverability** — a channel you can't reliably reach an inbox through isn't a growth channel at all, whatever its theoretical potential. Beyond that baseline, the rule that matters most is relevance: a blanket newsletter sent identically to every user, regardless of how long they've used the product or how engaged they are, treats a brand-new user and a three-year veteran as though they need the same message, which they almost never do.

The better default is triggered, contextual messaging tied to a specific, personal event. A brand-new user's first "like" or first connection is a magic moment worth celebrating with a notification. The same event, happening for the thousandth time to a veteran user with hundreds of connections, is noise, and notifying them about it risks training them to ignore or disable notifications altogether — which then makes it harder to reach them with a message that genuinely does matter later. eBay's own best-performing email campaign, by click-through rate, was triggered by a customer's very first cross-border transaction — a specific, timely, personally relevant event, not a generic blast sent to the whole user base on a schedule.

Growing into other languages and markets is a long-lead investment, not a feature flag

Facebook internationalized later than it should have, by its own leadership's admission, and paid a real cost for the delay: while the product stayed U.S.-focused, clones sprang

up across the world — some blatant enough to leave the site's own template filenames unchanged in the copied code. Catching up required building real infrastructure, not just translating text: an internal system for extracting every string in the product, plus a community translation platform that let Facebook's own users translate the product themselves rather than waiting on professional translators alone. French was fully translated within twelve hours of that system going live. Within a few years, the product supported over a hundred languages, the large majority contributed by the community rather than paid translators.

The subtler lesson sits in *which* languages got prioritized first, and it's a direct echo of Chapter 2's "why now" argument applied to markets instead of products: build for where the world is going, not where it already is. The four languages that mattered most in Facebook's early international push — French, Italian, German, Spanish — mostly shrank in relative importance within a few years, while languages that barely registered at first, like Hindi, quadrupled. A translation and localization strategy built only around today's largest markets would have missed the shift entirely; the infrastructure that mattered was the kind flexible enough to expand into whichever language turned out to matter next.

Paid acquisition: a worked example

Paid growth is the most directly quantifiable of the three categories, which is exactly why it deserves real numbers rather than just a description. Two formulas govern whether paid acquisition is a sound investment or a leaky bucket:

CAC (*customer acquisition cost*) = $\text{cost per click} \div \text{conversion rate}$

Profit per customer = $\text{CLV (customer lifetime value)} - \text{CAC}$

Here is an illustrative example to make the formula concrete — the dollar figures below are invented for demonstration, not numbers Cheung gave in her lecture, though the two formulas themselves are hers. Suppose an ad costs \$2 per click, and 4% of the people who click go on to sign up. CAC is $\$2 \div 0.04 = \mathbf{\$50}$ per acquired customer. If that customer is worth \$300 to the business over their first year (their CLV), the math looks straightforwardly good: $\$300 - \$50 = \$250$ in profit per customer acquired. Multiply that across enough customers and paid acquisition looks like free money.

Two complications make this less simple in practice, and both are worth taking seriously before scaling a paid channel. First, **CAC and CLV both vary enormously by customer segment**, and averaging across all customers hides that variation in a dangerous way. A marketplace for country music memorabilia, to use an intentionally vivid example, will see a far higher CLV from a customer in Nashville than from one in a market with almost no interest in the category — buying ads at a single blended CAC, rather than a CAC set per segment, means overpaying badly in the low-value segments while potentially underpaying (and under-investing) in the high-value ones.

Second, **the timing of when a customer actually pays you back matters as much as whether they eventually do**. Say a customer is genuinely worth \$300 over twelve months, but only \$100 of that arrives in the first month, with the rest spread across the rest of the year. If the CAC to acquire that customer is \$200, the business is \$100 underwater on that customer until the remaining \$200 of value actually arrives — and if something goes wrong before then (the customer churns early, the market shifts, the company runs low on cash), that gap never closes. A payback period of around three months is a conservative, safe default; stretching to twelve months is riskier but sometimes reasonable for a well-capitalized company; going beyond twelve months moves into territory where a founder is effectively betting the company's cash position on a prediction about a customer's future behavior that hasn't been tested yet.

Common Mistakes

- **Chasing a K-factor above 1 without retention underneath it.** Viddy's own story is the direct cautionary example: real virality, no durable product, ultimately no company.
- **Winning the wrong keyword perfectly.** A full year spent dominating "cocktail making" produced a fraction of the traffic "cocktail recipes" would have, because nobody was searching for the first phrase.
- **Sending the same newsletter to every user regardless of how engaged they already are.** A first-time magic moment and a thousandth repetition of the same event call for completely different messages, if they call for a message at all.

- **Calculating CAC and CLV as a single blended average across all customers**, hiding enormous segment-level variation that should be driving very different acquisition spend in different segments.
- **Accepting a long payback period without the cash reserves to survive it if a customer's expected value doesn't materialize on schedule.**
- **Waiting too long to invest in international infrastructure**, and then discovering competitors have already cloned the product into markets you weren't paying attention to.



Tradeoffs and Edge Cases

Doesn't the whole "virality formula" framing suggest every product needs a formal referral loop? No — Facebook's own history is the clearest evidence against that assumption. It grew primarily through word of mouth, without ever building a Hotmail- or PayPal-style formal viral mechanic, because the product itself was compelling enough to talk about. The formula in this chapter is a tool for understanding and improving a viral mechanic where one exists and makes sense for the product, not a requirement that every company build one.

How aggressive should paid acquisition be, given the payback-period risk described above? This depends heavily on how much of a cash cushion a company actually has, which is a question this book returns to properly in Part Five. The short version for now: treat the three-month payback benchmark as the safe default, and only extend it further if you have both the capital reserves to survive the wait and real confidence — not hope — that the later-arriving value will actually show up on schedule.

Is international expansion always worth the infrastructure investment this chapter describes? Not for every company at every stage — it's a genuinely large, long-lead investment, and building it too early, before a company has solid domestic product-market fit, can be a serious distraction. The lesson from Facebook's own experience isn't "internationalize immediately." It's "don't wait until competitors have already claimed the

markets you'll eventually want, and when you do build for other languages, build for where those markets are headed rather than only their current size."

Applying the Test

Pick one growth channel you're considering (or already using) and run the actual math from this chapter's worked example on it: what's your real CAC, broken out by at least two meaningfully different customer segments rather than blended into one average? What would your CLV need to be, in each segment, for that channel to be worth scaling? If you don't currently know your CLV by segment, that's the more urgent problem to solve before spending more on the channel itself.

Chapter Summary

- Growth channels only amplify a product that already retains users — none of the tactics in this chapter substitute for the product-market fit covered in Chapter 10.
- Virality has a real formula: $\text{payload} \times \text{conversion rate} \times \text{frequency}$ determines your K-factor, and different products (Hotmail, PayPal, Facebook) go viral in structurally different ways.
- Virality without retention is actively misleading — Viddy pushed its K-factor above 1 and still collapsed, because nothing durable sat underneath the growth.
- SEO success depends on researching the right keyword (not just winning whichever one you chose) and on links and site structure — a single navigation change multiplied Facebook's profile-page SEO traffic roughly a hundredfold.
- Messaging channels reward relevance over volume — a triggered, personally relevant message beats a generic blast sent to everyone at once.
- International growth requires long-lead infrastructure investment, prioritized toward where a market is headed, not just where it is today.

- Paid acquisition math ($CAC = \text{cost per click} \div \text{conversion rate}$; $\text{profit} = CLV - CAC$) only gives an honest answer when broken out by customer segment, and payback timing matters as much as the final CLV number.



Exercises

1. **Recall:** Explain why Facebook is described in this chapter as growing through word of mouth rather than through a formal viral loop, and why that doesn't contradict the virality framework covered earlier in the chapter.
2. **Apply:** Using this chapter's CAC/CLV worked example as a template, calculate CAC and expected profit per customer for a real or hypothetical paid channel, broken out by at least two different customer segments.
3. **Apply:** Design one triggered, contextual message (email, SMS, or push) for a product you're building, tied to a specific, personally relevant event — not a generic weekly update.
4. **Confront a counterexample:** This chapter argues international expansion should be prioritized toward where a market is headed rather than its current size. What risks does that approach carry if the prediction about where a market is headed turns out to be wrong, and how would a founder hedge against that risk?



Further Reading

- Alex Schultz's original CS183B lecture covers additional detail on Facebook's international growth strategy and its internal translation tooling, plus a "mousetrap" analogy for why a growth mechanism has to actually work before it's worth amplifying, and a reference to Ben Silverman's early growth work at Pinterest.
- David Ogilvy, *Ogilvy on Advertising* — recommended in Schultz's lecture as a classic primer on the persuasion side of paid and messaging channels.

- Adam Penenberg, *Viral Loop* — recommended in Schultz's lecture for a longer history of viral growth mechanics across several companies.
- Adora Cheung's lecture (also covered in Chapter 7) is the source of the sticky/viral/paid growth taxonomy this chapter builds on.
- Chapter 13 returns to unit economics from the fundraising side — investors will ask about CAC, CLV, and payback period using exactly the framework introduced in this chapter's worked example.
- Appendix C collects every formula from this chapter and Chapter 10 — K-factor, CAC, CLV — on a single page for quick reference.



Chapter 12 — Getting Press

PayPal was in the news for freezing merchant accounts without much explanation, and developers were furious about it. A YC company called WePay saw the opportunity, froze a block of ice with real cash inside it, and dropped it outside a PayPal developer conference — a stunt with essentially no production budget, tied directly to a story already in the news, timed to a specific event where the right reporters would already be standing around with nothing else to write about. It worked. WePay, a company nobody had covered before, got attached to a story everybody was already covering.

That's the pattern this chapter is about: press isn't a meritocracy where the most important company automatically gets covered. It's closer to a specific, learnable skill — knowing what counts as a story, who to tell it to, and how to get a reporter's attention without a budget most startups don't have anyway.



The Core Idea: Goal and Audience Come Before Anything Else

The instinct many first-time founders have is to want press simply because press feels like validation — a sign the company is real and important. Justin Kan's first, blunt correction: if you don't have an actual goal for a piece of coverage, getting it accomplishes nothing for the business, whatever it does for your own feeling of momentum.

The goal determines the audience, and the audience determines which outlets are worth targeting at all. Kan's own examples make the point concrete: Socialcam wanted to be positioned as "video Instagram" for Silicon Valley investors and influencers, so tech press was the target. Exec, a local cleaning service, only needed San Francisco residents who could actually use the service — national coverage would have reached almost entirely the wrong readers, so local outlets like the San Francisco Chronicle were the real target.

TwitchTV needed the gaming industry itself — developers and advertisers — to see it as a serious, important place, so the target was gaming industry trade publications and

developer blogs, not general consumer press. Three completely different audiences, three completely different outlets, because three completely different business goals sat underneath each campaign.



Why It's True: What Counts as a Story, and How the Mechanics Actually Work

Not everything you do is a story

The single most useful filter Kan offers: before pitching anything, ask honestly whether you'd want to read this story if you weren't the company's founder. Founders reliably overestimate how interesting their own routine progress is — a minor feature update or a small version bump rarely clears this bar, however meaningful it feels internally.

There are a handful of story types that reliably do clear it: product launches, fundraising announcements (which the press covers readily even though, as Kan notes, they're rarely genuinely interesting), real milestones and metrics, business-strategy stories (usually reserved for companies already established enough to be worth profiling), creative stunts tied to a live news moment (WePay's ice block), hiring announcements for a sufficiently senior or newsworthy hire, and contributor or opinion pieces written by the founder for an outlet.

A story also doesn't need to be original in an absolute sense — it needs to be, in Kan's phrase, "original enough." Being the first company in a specific, narrow category to hit a milestone gets covered; being the second gets ignored, even if the second company is otherwise just as interesting. Kan's own contrast makes the point sharply: you don't want to be the second company to raise \$5 million on Kickstarter, because the first one gets all the news and the second gets none of it. The first video game console to raise \$10 million on Kickstarter, by contrast, was genuine news, despite many other projects having raised comparable amounts on Kickstarter before it in different categories — the specific combination was new enough to count as a real story.

Treat it like a sales funnel, and get the introduction from someone else's success

Most pitches to most reporters don't convert into coverage, and that's the normal, expected outcome — not a sign the story or the company is bad. The practical mechanics that improve the odds are worth following in order. The single highest-leverage move is getting introduced to a reporter by someone that reporter has already covered successfully, rather than cold-emailing. This benefits everyone involved: the founder making the introduction loses nothing by connecting a reporter to another interesting company, and the reporter, receiving a referral from someone they already decided was worth covering, extends some of that trust to the new introduction by association.

Once in touch, give a reporter real lead time — roughly a week or more before any news needs to run, not a same-day request. Reporters have their own schedules and existing commitments; showing up with no notice reads as a lack of professionalism rather than urgency. Where possible, get an actual phone call or in-person meeting rather than relying only on email exchanges. There's a real, human mechanism behind why this works: the more time and attention a reporter has already invested in a conversation, the more likely they are to follow through and actually write the story, simply because that invested time now feels wasted if they don't.

The pitch itself should be prepared in advance, as a specific set of bullet points memorized well enough to guide the conversation naturally, so the reporter's own notes end up capturing what the founder actually wanted covered — the cofounder's name spelled right, the specific feature or metric that matters, the framing that fits the goal set at the very start. A short follow-up close to the actual publication date, with logistics and any supporting material (photos, video, correct spellings), closes the loop.

PR firms can't tell you what's interesting about your own company

Firms exist to manage contacts and logistics at scale, and that's a real, if limited, value. What they generally cannot do is figure out what's actually newsworthy about a specific company — that judgment has to come from the founder, who understands the business, the product, and the goal well enough to make the call themselves. Kan's own experience with PR firms, across several companies, is that they've never once told him something genuinely interesting about his own company that he hadn't already figured out — only handed him a list of reporters to contact. Combined with real cost (commonly somewhere

between five and twenty thousand dollars a month), the case for hiring a firm early is weak. Learning the mechanics in this chapter yourself, first, is both cheaper and more effective — and it's a skill that transfers to every future round of press the company will ever need, in a way that outsourcing the whole function from day one doesn't build.

Press is a vanity metric, and it doesn't scale

It's worth stating plainly, because the temptation to conflate the two is strong: getting covered feels like succeeding, but it isn't the same thing as succeeding. A company can generate a steady stream of press with a flat usage graph and no revenue, and a company can be genuinely thriving with almost no coverage at all. Press correlates loosely with real success because successful companies tend to attract more of it over time — but the correlation runs in that direction, not the reverse, and chasing coverage as a substitute for the metrics covered in Chapters 10 and 11 (retention, growth, revenue) mistakes a symptom for a cause.

It's also worth being explicit about a limit this chapter's advice runs into: press works well as a bootstrapping tool for a company's first hundred, or few hundred, customers — exactly the audience a single well-placed story can plausibly reach. It does not scale into an ongoing acquisition channel the way the tactics in Chapter 11 can, because there is no way to generate an unlimited supply of stories a reporter will find genuinely new. Once a company needs sustained, large-volume growth, this chapter's tools stop being the right tool for that specific job, even though they remain useful for the goals — investor perception, industry credibility, hiring — press is actually good at achieving.

Maintaining relationships pays off later, in both directions

Once a reporter has covered a company, staying in touch — without an immediate ask attached — makes it more likely that reporter covers the company again later, simply because there's now a real relationship rather than a one-time transaction. This matters in both good times and bad: if a company ever faces negative coverage, having an existing relationship with at least a few reporters makes it meaningfully easier to get a fair hearing and get the company's own side of the story included.

The same logic extends outward, not just backward: recommending other founders' stories to reporters, even when there's no direct benefit to you, tends to come back

around, because reporters remember who reliably sends them good leads, and other founders remember who helped connect them.

Twitch's own biggest viral moment, "Twitch Plays Pokémon," illustrates how this all fits together honestly. The company didn't invent the idea — a segment of its own community did, entirely organically. Twitch's actual contribution was making sure the groundwork was already in place: enough prior, smaller press coverage that when a major outlet like the BBC went looking for context on this strange new internet phenomenon, there was already a trail of earlier stories to draw on, and a company ready and available to talk to reporters and suggest natural follow-up angles as the story kept developing. Twitch Plays Pokémon itself drew roughly 100,000 people watching at once at its peak, and the relationship-building paid off again on the follow-up: when the community moved on to play through "Crystal," the next Pokémon game, Twitch already had the press contacts in place to turn that into a second round of coverage rather than letting the story die after its first spike. The company didn't manufacture the moment. It had done the unglamorous relationship-building work in advance that let it capitalize on a moment it didn't create.

Common Mistakes

- **Pursuing press without a specific goal or target audience.** Coverage that reaches the wrong readers doesn't accomplish anything, however good it feels.
- **Pitching a story that isn't actually a story** — a minor update that wouldn't interest anyone who isn't already invested in the company's success.
- **Cold-emailing reporters instead of getting a warm introduction from someone they've already covered.**
- **Giving a reporter no lead time**, then being surprised when a same-day request goes nowhere.
- **Hiring a PR firm early, expecting it to identify what's interesting about the company.** That judgment has to come from the founder.

- **Treating press coverage as a proxy for real business success**, or as a scalable growth channel once a company needs volume rather than an initial hundred customers.



Tradeoffs and Edge Cases

Doesn't calling press "a vanity metric" contradict recommending it as a tool for the first hundred customers? Not really — the two claims describe different uses of the same tool. Press is a legitimate, useful bootstrapping mechanism at very small scale, precisely because a single story genuinely can reach a company's first cohort of users. The warning is against mistaking that early, real usefulness for a signal of the company's overall health, or expecting the same mechanism to keep working at the volume Chapter 11's growth channels are built for.

Is there ever a good time to hire a PR firm? Yes, once a company has a genuine, ongoing cadence of newsworthy events and the volume of contact management becomes a real logistical burden rather than a judgment problem — at that point, a firm's actual strength (managing relationships and logistics at scale) becomes relevant. The mistake this chapter warns against is hiring one early, when the actual bottleneck is deciding what's interesting, not managing contact lists.

What if a company can't identify a "goal" for press beyond general visibility? That's a signal worth taking seriously rather than working around — if there's no specific business objective a piece of coverage would advance, that's a sign this isn't the right time to be pursuing press at all, not a sign to lower the bar for what counts as a goal.



Applying the Test

Before pitching any story, write down two things separately: the specific business goal this coverage is meant to advance, and the specific audience that goal implies. Then apply the honesty test directly — if you had no connection to this company, would you

actually want to read the story you're about to pitch? If the honest answer is no, the fix isn't better writing. It's a different story, or waiting until there's a real one to tell.

✓ Chapter Summary

- Press is not a meritocracy — define your specific business goal and target audience before pitching anything, since different goals point to completely different outlets.
- A story has to clear a real bar: would you want to read it if you weren't the founder? Most routine updates don't clear it.
- A story doesn't need to be original — it needs to be original enough, usually meaning first in some specific, narrow category.
- Get warm introductions from founders a reporter has already covered, rather than cold emailing; give real lead time; prepare a memorized bullet-point pitch in advance.
- PR firms manage contacts and logistics well but can't tell you what's actually interesting about your company — that judgment has to be the founder's.
- Press correlates with looking successful, but it isn't the same as being successful, and it doesn't scale past a company's first few hundred customers the way real growth channels do.
- Maintain reporter relationships even without an immediate ask, and help other founders get coverage — both tend to come back around later.



Exercises

1. **Recall:** Explain why "original enough" is a more useful standard than "completely original" when deciding whether something is a pitchable story.
2. **Apply:** Pick a real or hypothetical company. Define its specific press goal and target audience, the way Kan did for Socialcam, Exec, and TwitchTV. What outlets would actually reach that audience?

3. **Apply:** Draft the bullet-point pitch you'd bring into a reporter meeting for a real or hypothetical story, structured so a reporter's own notes would capture what you actually wanted covered.
4. **Confront a counterexample:** This chapter argues press doesn't scale as a growth channel past a company's first few hundred customers. Are there business models or company types where ongoing press coverage genuinely does function as a durable, scalable growth channel? What would have to be true about that business for it to work?



Further Reading

- Justin Kan's original CS183B lecture recommends Jason Kincaid's overview of pitching press (Kincaid is a former TechCrunch reporter) for more tactical depth than the lecture had time for.
- *Trust Me, I'm Lying* by Ryan Holiday, referenced in the same lecture, gives a deliberately cynical account of how stories spread online — useful context even though its author's own methods aren't this chapter's recommendation.
- Chapter 11 covers the growth channels that take over once a company needs acquisition volume beyond what press alone can ever realistically provide.



Chapter 13 — How to Raise Money

About four thousand companies try to raise venture capital in a given year. Roughly two hundred of those get funded by a top-tier firm. Of those two hundred, about fifteen will eventually reach \$100 million in revenue. Those fifteen companies — out of the original four thousand — will generate somewhere around 97% of all the returns the entire venture capital industry makes that year.

Marc Andreessen, cofounder of the venture firm Andreessen Horowitz, shares that number not to be discouraging, but to explain something that otherwise looks irrational about how investors behave: venture capital is a business built entirely around finding extreme outliers, and almost every practice covered in this chapter — what investors look for, how they price risk, why they sometimes pass on companies that look perfectly reasonable — makes sense once you understand that a fund's entire return depends on finding one of those fifteen companies, not on a portfolio of merely solid ones.



The Core Idea: Fundraising Is About Peeling Away Risk, One Round at a Time

Andreessen borrows a framework from investor Andy Rachleff that reframes what a funding round is actually for: think of a brand-new startup as carrying an entire onion's worth of risk, layered one on top of another. Founding-team risk — can these people even work together. Product risk — can they build the thing at all. Technical risk — does it require some breakthrough that might not happen. Launch risk. Market-acceptance risk. Revenue risk. Cost-of-sales risk, if there's a sales force. Viral-growth risk, if the model depends on one.

Each funding round exists to peel away a specific layer or two of that onion, not to provide a general cash cushion. A seed round should get you far enough to prove the founding team can execute and the product can exist. A Series A should get you far enough to prove real customers want it. Each round's size should be calibrated to the specific layers

of risk it's meant to remove — not simply "as much money as investors will offer," which is a mistake this chapter returns to directly.



Why It's True: What Investors Actually Look For, and the Deals That Prove It

Strength, not the absence of weakness

The default, checkbox way to evaluate a startup goes something like: good founder, check; good idea, check; good product, check; good initial customers, check.

Andreessen's observation, after years of doing this, is that checkbox deals — reasonable on every dimension — routinely lack the one thing that actually produces an outlier: an extreme, specific strength along some dimension that makes the company remarkable rather than merely solid.

The uncomfortable flip side of this: companies with a genuinely extreme strength frequently have real, visible flaws elsewhere. If a firm's policy were to avoid every company with a serious flaw, Andreessen notes, it would have passed on nearly every big winner in the firm's history. The actual discipline isn't finding flawless companies — it's finding the specific strength worth betting on, and being willing to tolerate real weaknesses alongside it, which is a much harder judgment call than working through a checklist.

Ron Conway, an angel investor who has backed several hundred companies since the 1990s, adds a specific, practical version of what he's evaluating in the first minute of any pitch: is this person a natural leader, and are they obsessively focused on the product. He's hoping the motivation traces back to a personal problem the founder actually had, not an idea manufactured to look fundable — the exact same instinct Chapter 1's "you can't not do it" test was built around, arrived at independently by an investor rather than a founder. Conway's other filter, echoing a principle this book already established in Chapter 2: a founder should be able to explain what their product does in one compelling sentence. By his own account, roughly a quarter of the founders he meets still can't clear

that bar after their first sentence — and after enough years doing this, he simply stops the pitch and asks them to restart.

What investors actually check when they evaluate "revenue risk"

Two layers of Andreessen's onion — revenue risk and cost-of-sales risk — get evaluated through a specific lens worth naming directly, since it connects to a framework this book already covers in full: unit economics. An investor looking at "revenue risk" is really asking the same question Chapter 11 built a worked example around — does customer lifetime value (CLV) exceed customer acquisition cost (CAC), broken down by segment rather than blended into a single average — and an investor looking at "cost-of-sales risk" is asking whether a company's sales motion (Chapter 16 covers this directly) can scale without costing more than the revenue it brings in. A founder walking into a fundraising conversation without a clear, segment-level answer to this math is, in effect, asking an investor to take the revenue-risk layer of the onion on faith.

This is also where a real structural difference shows up between consumer software and enterprise software, worth flagging honestly: Andreessen notes that enterprise and SaaS companies frequently have no MVP-equivalent stage at all. A consumer app can launch something minimal and iterate in public; an enterprise customer, by contrast, usually won't adopt anything short of a genuinely complete product, which means an enterprise company often needs to raise five to ten million dollars just to build the first version customers will actually buy — a "cold start" with no scaled-down early version to test demand first. This isn't a reason to avoid enterprise software (Chapter 16 covers why it can still be an excellent bet) — it's a reason the onion-theory precision this chapter recommends for capital-intensive businesses applies to certain software categories too, not just to hardware and biotech.

Being fundable is easier than almost everything that follows

Andreessen's own comparison, delivered as a direct challenge to the idea that fundraising is the hardest part of building a company: raising venture capital is, in his assessment, one of the *easiest* things a founder will ever have to do, compared to recruiting a twentieth engineer, selling into a large enterprise, or generating real viral growth in a consumer product. His reference point is a line from comedian Steve Martin's memoir: the key to success is being so good that people can't ignore you. Applied to fundraising, that

means the highest-leverage use of a founder's time is almost never refining the pitch deck — it's making the underlying business good enough that raising money becomes close to a formality.

Parker Conrad's own story, told directly in this same panel, makes the point uncomfortably concrete. Pitching an earlier company, he and his cofounder were rejected by roughly sixty different venture firms. At one of those meetings, a partner at Khosla Ventures told him bluntly, in Conrad's own retelling of the moment: if they'd been "the Twitter guys," they could have pitched anything at all and been funded on the spot; since they weren't, they needed to have all their analysis ready to justify the bet. Conrad took the opposite lesson from the one the partner intended — not "prepare better analysis," but "figure out how to actually become the kind of company that doesn't need the analysis." His next company, Zenefits, was deliberately conceived as a business that could plausibly grow without ever raising outside money at all — and that same quality, a business viable on its own, was exactly what made investors want to fund it eagerly once he did decide to raise.

Choosing an investor is closer to choosing a spouse than closing a transaction

Both Andreessen and Conway independently describe the investor relationship in the same terms: a serious startup is potentially a ten-to-twenty year project, which is longer than the average American marriage, and the investors who join the board for that stretch of time are people a founder will be relying on through real crises, not just during the good years. Andreessen's advice: weigh the choice of investor with something close to the seriousness you'd bring to choosing a life partner — not because the check size doesn't matter, but because when things go wrong (and on a long enough timeline, something always does), the difference between an investor with real domain expertise and trustworthy judgment and one who simply wrote a check becomes the whole story.

Two real investment stories from this same conversation illustrate what that difference actually looks like in practice. Ron Conway's account of investing in Google in 1999 came entirely through personal trust: a Stanford professor mentioned an early student project called "BackRub," built around a then-unproven idea — ranking search results by which pages other people linked to, rather than by keyword matching alone. Conway had to wait months and prove himself before the founders would even meet with him, and getting into

the deal required bringing in Sequoia Capital first, because the founders wanted Sequoia's existing relationship with Yahoo secured before they'd accept Conway's money.

Andreessen's account of Airbnb runs the opposite direction: his firm initially passed on the company entirely, missing it the first time around because — by his own admission — the idea sounded almost deliberately absurd (a website for staying in strangers' spare rooms, combined with a website where you could rent out your own spare room to strangers, which he calls "the second most stupid idea you could possibly have" layered on top of the actual most stupid one). The firm came back later, at a much higher valuation, once the numbers had clearly proven the initial read wrong — and the decision to invest heavily at that point rested as much on directly observing how mature and grounded the three founders had become under real pressure as on the growth numbers themselves.

Common Mistakes

- **Asking investors to sign an NDA before discussing the business.** Both Conway and Andreessen treat this as close to a disqualifying signal — professional investors see enough deal flow that a request like this reads as distrust rather than caution, and it damages the relationship before it starts.
- **Treating a verbal commitment as a real commitment.** Conway's specific fix: as soon as an investor commits to something in a conversation, send a follow-up email confirming the details in writing immediately — investors, in his experience, frequently and genuinely forget specifics later, and a written record avoids relitigating it.
- **Raising as much money as investors will offer, rather than exactly enough to peel away the next layer of risk.** More capital raised now means more dilution locked in permanently, regardless of whether that capital was actually needed for the milestones ahead.
- **Choosing an investor primarily for valuation or check size, rather than domain expertise and trust.** Andreessen is direct that his own firm will walk away from an otherwise-attractive deal if a company's cap table is already too diluted or already includes investors who add no real value, because both problems demotivate the founding team in ways that outlast any single round.

- **Obsessing over the fundraising process as if it were the actual milestone.**

Conway's advice: treat it as one of the smallest steps in the company's life and get it over with efficiently, rather than treating a successful raise as a personal victory.



Tradeoffs and Edge Cases

Does the onion-theory framework change for capital-intensive businesses (hardware, biotech, infrastructure)? The underlying logic stays the same, but the required precision goes up substantially. Andreessen's advice for these cases: be exact about what each round of a much longer, more expensive funding path is meant to prove, because the tolerance for wasted capital or a misjudged milestone is much lower when total capital needs stretch into the hundreds of millions across many rounds. Venture debt and lease financing become relevant options in this category in a way they rarely are for a capital-light software company — but both still require the same operational discipline this chapter's onion framework describes, arguably more of it.

What percentage of the company is reasonable to sell in a given round? There's no universal formula, but rough norms exist: a Series A commonly sells something in the twenty-to-thirty percent range; a seed round is often closer to ten to fifteen percent. Ron Conway's specific warning: think in advance about the point at which your own remaining ownership would start to feel demotivating, because a founder who's already diluted down to a very small stake after just one or two early rounds has, in a practical sense, already undermined their own long-term motivation — a problem no later success can fully undo.

Does board composition matter as much as founders often fear? Less than expected, in both Andreessen's and Parker Conrad's account, as long as trust is genuinely high. Most of an investor's real influence comes from protective provisions written into the financing terms, not from formal board votes — and in Andreessen's own two decades on startup boards, he reports never once being part of an important vote that actually came down to a close count; the real decisions get worked out through discussion and consensus well before any formal vote would matter. The exception, worth noting honestly: when a company gets into serious trouble, the terms of a prior round often get

renegotiated anyway, regardless of what was originally signed — which is itself another argument for choosing investors who can be trusted to renegotiate fairly, not just investors who offered the best terms going in.

Applying the Test

Take your own company's current or hypothetical funding stage, and list the layers of Andreessen's onion in order: founding-team risk, product risk, technical risk, launch risk, market-acceptance risk, revenue risk, cost-of-sales risk, viral-growth risk. Mark which layers your last round of funding (or your current savings, if you haven't raised yet) was actually meant to peel away. Then ask honestly: is the amount of capital you're considering raising next sized to remove the *next* layer specifically, or is it sized to "raise as much as we can get"? If it's the second, that's worth revisiting before you start the process.

Chapter Summary

- Venture capital is a business built around extreme outliers — roughly fifteen out of four thousand companies a year generate almost all the industry's returns, which explains much of how investors actually behave.
- Each funding round should be sized to remove a specific layer of risk (the "onion theory"), not simply to raise as much as investors are willing to offer.
- Investors look for extreme strength, not the absence of weakness — checkbox-perfect companies frequently lack the one differentiating quality that produces an outlier.
- Being genuinely fundable is, according to Andreessen, easier than almost everything a founder does afterward — the highest-leverage move is making the business better, not refining the pitch.
- Choosing an investor is closer to choosing a spouse than closing a transaction — a ten-to-twenty year relationship where domain expertise and trust matter more than

check size or valuation.

- Never ask for an NDA, and always follow up investor commitments in writing immediately.
- Board seats and formal votes matter less than expected when trust is genuinely high — most real influence runs through informal discussion and the terms written into the financing itself.



Exercises

1. **Recall:** Explain the "onion theory of risk" in your own words, and why it argues against simply raising as much money as investors are willing to offer.
2. **Apply:** List the risk layers in Andreessen's onion framework for your own company or a hypothetical one, and identify which specific layer your next round of funding should be sized to remove.
3. **Apply:** Draft the one-sentence description of your product that Ron Conway would need to hear in his first minute of a pitch, per the standard set in Chapter 2.
4. **Confront a counterexample:** Andreessen's firm initially passed on Airbnb, judging it "the second most stupid idea" layered on top of an already bad one, before returning to invest heavily at a much higher valuation. What would you need to see in your own snap judgment of an unusual idea to know whether you're seeing real weakness, or just an idea that sounds bad for the reasons Chapter 2 said good ideas often do?



Further Reading

- Andy Rachleff's original writing on the "onion theory of risk," referenced by Andreessen in this lecture, develops the framework in more depth than the panel format allowed.

- Chapter 2's discussion of ideas that "sound bad but are actually good" is worth rereading alongside this chapter's Airbnb story — the same pattern, seen this time from an investor's side of the table instead of a founder's.
- Chapter 15 covers the legal and financial mechanics (cap tables, vesting, SAFEs) that turn this chapter's strategic advice into an actual signed term sheet.



Chapter 14 — Pitching and Fundraising Mechanics

Y Combinator ran a mock pitch exercise once, live, in front of a class. A founder played himself. A partner played an investor. The founder described his company as "a communication platform that will allow businesses and consumers to collaborate on one single platform." Asked what the product actually does, he circled the same sentence again, slightly reworded. Asked about traction, he said he didn't want to "open the kimono." Asked about the business model, he said they hadn't precisely figured it out yet, but were thinking maybe a monthly fee, they weren't sure how much. The investor ended the meeting politely and moved on.

Then they ran it again, with a second, better-prepared founder pitching a similar kind of idea. This one opened with something concrete: "when you walk into a Crate and Barrel, you can send the manager a message like, 'there's puke in the hallway.'" Pressed on traction, he gave real numbers without flinching: 350 locations, growing 11% a week, three months in. Pressed on what he'd learned, he offered something the investor didn't already know: more than half the messages sent through the product weren't feedback at all — they were simple questions, like "when do you open?" Asked about the business model, he said plainly that they didn't have a fully settled answer yet, and explained the two paths they were actually testing. At the end, he made a specific, real ask: a defined amount, on defined terms, with part of the round already committed by named investors, closing on a specific date.

Same format, same amount of time, two completely different outcomes. This chapter is about the difference between those two pitches, and about the mechanics of fundraising that surround the pitch itself — when to start, how to get the meeting, and what happens after.



The Core Idea: Thirty Seconds, Then Two Minutes, Nothing Longer

Michael Seibel, a YC partner who has founded two companies himself, argues that a founder needs exactly two versions of their pitch memorized: one that takes thirty seconds, and one that takes two minutes. Nothing longer is actually necessary, and longer pitches tend to hurt more than they help — every additional minute of talking is another chance to say something an investor doesn't like, with no offsetting benefit.

The **thirty-second pitch** is three plain sentences: what the company does, how big the market is, and how much traction exists so far. The **two-minute pitch** adds four more components on top of that foundation: a specific insight nobody else has, how the company makes money, who the team is, and — only in this longer version — the actual ask. Everything in this chapter is really just an elaboration of these seven pieces, and the discipline of keeping each one to a sentence or two is not a stylistic preference. It forces a founder to know, with total clarity, what actually matters about their own company.



Why It's True: What Each Sentence Has to Do, and Why the Mechanics Matter as Much as the Words

The thirty-second pitch: what, how big, how fast

The first sentence has to describe the product in language so simple that a non-technical relative would understand it — Seibel's own name for this is the "mom test," a direct echo of the one-sentence clarity rule Chapter 2 already established for evaluating ideas in the first place. If the honest, simple version of the sentence sounds underwhelming, that's still better than a longer, vaguer sentence that sounds impressive but doesn't actually explain anything — Ron Conway's complaint in Chapter 13, that a quarter of founders can't clear this bar, is describing exactly this failure.

The second sentence states the market size, aiming at a number in the billions where a credible one exists — not because the number itself proves anything, but because it tells an investor whether the company could ever become one of the outliers Chapter 13 described. The third sentence states traction: real growth numbers if they exist, or, if the company hasn't launched yet, evidence of how fast the team is moving — a launched beta

within weeks of starting, for instance — since speed is the best available substitute for traction when traction doesn't exist yet.

The two-minute pitch: insight, money, team, ask

The two-minute version adds four things, each disciplined down to a sentence or two.

Unique insight is the specific thing the founder understands that the biggest players in the space either don't understand or haven't acted on — the "aha" moment an investor should feel on hearing it. If the insight produces a shrug of recognition rather than genuine surprise, it hasn't been sharpened enough yet.

How you make money should be stated plainly, using the actual, unhedged mechanism — advertising, direct sales, subscription, whatever it genuinely is — rather than a list of several possible models offered as if to hedge against being wrong. Naming several possible business models at once doesn't read as flexible. It reads as not yet having figured out the business, which is a different and worse impression to leave.

Team should mention truly impressive past accomplishments if they exist (having built something that made real money for people, for instance) and otherwise stick to concrete, checkable facts: how many founders, how many are technical, how long the team has known each other, whether everyone is committed full-time, and how the team actually met. Padding this section with unrelated academic credentials or generic descriptors tends to draw attention to what's missing rather than what's there.

The ask is the one place in the entire pitch where using precise financial jargon is correct instead of a warning sign. Everywhere else in this chapter, plain language is the goal — but stating the ask vaguely ("we're raising some money") signals a founder who hasn't done the homework, in a section where the homework is exactly what an investor is checking for. The ask needs the exact instrument (a convertible note or a SAFE), the valuation cap, the total amount being raised, and the minimum check size an investor could write. A founder who doesn't know these numbers cold isn't ready to be having this conversation yet — Chapter 15 covers what each of these terms actually means in enough depth to use them confidently.

The two live pitches, side by side

The contrast from this chapter's opening is worth breaking into its specific, transferable lessons, because each failure in the first pitch maps directly onto one of the seven components above. The weak pitch never landed a clear one-sentence description, even after several attempts — a direct failure of the "mom test." It avoided giving real numbers when asked about traction, using the phrase "don't want to open the kimono" instead of either real data or an honest "we're pre-launch." Its business model answer hedged across several unconsidered options rather than committing to an actual mechanism. Its team description was vague and unconvincing rather than concrete. And it never arrived at an actual ask at all — the meeting simply ran out of time.

The stronger pitch did the opposite at every single point. Its one-sentence description used a specific, vivid, concrete example (the Crate and Barrel "there's puke in the hallway" message) instead of an abstract category description. Its traction numbers were exact and unhesitating. Its unique insight was genuinely surprising — that most messages sent weren't complaints at all, but simple logistical questions, an insight the company had only discovered by building the product and watching real behavior, not by guessing in advance. Its business-model answer was honest about genuine uncertainty while still demonstrating that two specific paths were actively being tested, rather than a vague shrug. Its team answer included a past failure, disclosed plainly rather than hidden, and concrete facts about who the founders were and how they'd worked together. And it closed with a fully specified ask — dollar amount, instrument, valuation cap, already-committed capital from named investors, and a specific closing date — that gave the investor everything needed to actually say yes on the spot.

When to actually start, and how to set up the meetings

Investors, unsurprisingly, prefer to invest based on real traction — so the honest strategic question isn't just "am I ready," it's "how do I put myself in the strongest possible position before I start." If traction doesn't exist yet, two substitutes carry real weight: whether investors are already proactively asking to be involved (a strong signal you're already talking about the company effectively), and whether the team has already minimized how much money is actually needed to get to market — Michael Seibel's estimate is that the large majority of startups can get an initial version of a product built and launched on very little capital. The goal in either case is the same: never put yourself in a position where the pitch implicitly says "we can't do anything until you fund us." The stronger, and usually

true, position is "this is moving already, with or without you" — which flips the power dynamic in the room in a way no amount of polish on the pitch itself can substitute for.

Once you're ready to start real conversations, the practical mechanics matter almost as much as the pitch content. A warm introduction — from another founder, or from a previous investor — is close to mandatory; a cold approach with no connection starts from a real disadvantage. Fundraising should run as a sprint, not a marathon: schedule as many meetings as possible in the same tight window, rather than spreading them out over months, so that offers and momentum can compound instead of going stale one at a time. A useful, slightly counterintuitive tactic for creating that clustering: when investors respond with interest, ask to meet in two or three weeks rather than immediately, citing real work you're heads-down on — this naturally bunches meetings into the same period across every investor you're talking to, and it signals, correctly, that you're not desperate for the money right now. Finally, one person on the founding team — not the whole company — should own the fundraising process, so the rest of the team can keep building instead of getting pulled into a process that, per Chapter 13, is supposed to be one of the easier and smaller parts of the company's life.

Common Mistakes

- **Failing the "mom test" on the very first sentence.** If a non-technical relative couldn't repeat back what the company does, the sentence needs more work before anything else in the pitch matters.
- **Hedging the business model across several possible options instead of stating the real one.** This reads as not having figured out the business yet, not as admirable flexibility.
- **Being evasive about traction instead of giving real numbers, or an honest "pre-launch."** "I don't want to open the kimono" tells an investor there's something to hide, whether or not that's true.
- **Vague deal terms in "the ask."** This is the one part of the pitch where precise jargon is required, not optional — an imprecise ask signals a founder who isn't actually ready to close a deal.

- **Spreading investor meetings out over months instead of clustering them into a tight window.** Fundraising momentum compounds when meetings happen close together and fades when they're spread thin.
- **Pitching from a position of needing the money to do anything at all,** instead of demonstrating the company is already moving regardless of whether this particular investor says yes.



Tradeoffs and Edge Cases

Doesn't "use precise jargon in the ask" contradict this book's general preference for plain language? No, and it's worth being explicit about why the two don't conflict.

Chapters 1 and 2 argued for plain, jargon-free language when *describing an idea*, because jargon there usually hides a lack of clarity. The ask is a different kind of statement — a specific, checkable financial term, not a description of what a product does — and precision there signals competence rather than obscuring anything. The rule across this whole book is really one rule, not two: use the simplest language that is still fully accurate for what you're actually saying.

What if you genuinely have no traction and no way to fake speed either? This is a real, common position for a first-time, pre-launch founder, and it's worth being honest that the "how much traction" sentence is weakest exactly here. The available substitutes — team commitment (everyone full-time), evidence of speed (what's shipped in the last few weeks), and a sharp, specific "why now" (Chapter 2) — won't fully replace real traction, but they're the honest, available alternative, and overstating traction that doesn't exist is a worse option than admitting its absence plainly.

Is it ever fine to skip a formal ask and just build relationships first? Yes — plenty of real fundraising conversations start as relationship-building meetings with no immediate ask attached, particularly with investors a founder expects to approach again later. The seven-part structure in this chapter describes what a meeting where you *are* asking for money should contain, not a requirement for every single investor conversation a founder ever has.



Applying the Test

Write out your own thirty-second and two-minute pitches, using the exact structure from this chapter — one sentence per component, nothing longer. Then read the thirty-second version out loud to someone with no context on your company or industry. If they can't repeat back what your company does afterward, in their own words, the first sentence isn't done yet, no matter how good the rest of the pitch is.



Chapter Summary

- Memorize exactly two pitches: thirty seconds (what, market size, traction) and two minutes (adds unique insight, business model, team, and the ask).
- Use the "mom test" for the one-sentence description — the same clarity standard Chapter 2 established for evaluating ideas in the first place.
- State the business model plainly, using the real mechanism, rather than hedging across several possible options.
- The ask is the one place precise financial jargon belongs — vague deal terms signal a founder who isn't actually ready to close.
- Never pitch from a position that implies the company can't move without this specific investor's money — demonstrate it's already moving regardless.
- Get warm introductions, cluster meetings into a tight window rather than spreading them out, and have one person own the fundraising process rather than distracting the whole team.



Exercises

1. **Recall:** List the seven components of the full two-minute pitch, and explain why the ask is the one place where precise jargon is appropriate rather than a warning sign.

2. **Apply:** Write your own thirty-second and two-minute pitches for a real or hypothetical company, holding strictly to one or two sentences per component.
3. **Apply:** Test your one-sentence description on someone unfamiliar with your industry, using the "mom test." Revise it based on what they couldn't repeat back correctly.
4. **Confront a counterexample:** The strong pitch in this chapter's cold open disclosed a past failed startup honestly rather than hiding it. Are there situations where disclosing a past failure this openly would hurt a founder's credibility rather than help it, and how would you tell the difference in advance?



Further Reading

- The full mock-pitch roleplay this chapter's cold open condenses is preserved in Appendix D, for readers who want the complete, unedited exchange rather than this chapter's summary of its lessons.
- Chapter 2's "one sentence" rule for evaluating ideas and this chapter's "mom test" for pitching them are the same underlying discipline, worth reading together.
- Chapter 15 covers what a valuation cap, a SAFE, and a convertible note actually mean in enough mechanical depth to state an ask with real confidence.



Chapter 15 — Legal and Financial Mechanics

A Y Combinator company once incorporated as an LLC in Connecticut, on the advice of a local lawyer, and was told to convert to a Delaware corporation once it joined YC — the standard structure nearly every venture-backed startup uses. The conversion paperwork, filed by that same local law firm, contained a small, technical mistake. The company believed, for roughly two years, that it was a Delaware corporation. It wasn't. The error surfaced only when the company was raising a large round of financing and its lawyers started digging through the cap table in detail. Untangling it required four separate law firms across two states. The bill came to roughly half a million dollars, for a mistake that would have cost nothing to avoid in the first place, by simply using a standard, well-tested structure from day one.

This chapter is not the most exciting one in this book. It's arguably the one where a mistake is most expensive, precisely because these mistakes tend to surface at the worst possible moment — during a fundraise, an acquisition, or a founder dispute — rather than quietly, when they'd be cheap to fix. The goal here is not to make you a lawyer or an accountant. It's to make sure you understand this material well enough to know when something needs a specialist's attention, and well enough that a specialist can't quietly take advantage of you by burying a real problem in language you don't understand.

Before Anything Else: Reading a Business's Financial Vocabulary

Founders in this book's audience are assumed to have zero business background, so before covering startup-specific mechanics, it's worth building the basic vocabulary that everything else in this chapter — and the fundraising chapters before it — depends on.

A **profit and loss statement** (P&L, sometimes called an income statement) shows revenue coming in, expenses going out, and the difference between them over a period of time — a month, a quarter, a year. A company that's losing money simply has expenses larger than revenue on this statement; that's normal and expected for most early-stage startups, not automatically a sign of trouble.

A **balance sheet** is different — it's a snapshot at a single moment, not a period of time. It shows what a company owns (assets: cash in the bank, equipment, money owed to it) against what it owes (liabilities: unpaid bills, debt) and what's left over (equity — the value that belongs to the founders and investors).

A **cash flow statement** tracks actual cash moving in and out of a bank account, which is a different number from the P&L's "profit," because a P&L counts revenue when it's earned, not necessarily when the cash actually arrives. A company can show a profit on paper and still run out of actual cash, if customers are slow to pay.

Two more terms matter more to an early startup than almost anything on the statements above: **burn rate** is how much cash a company spends, net, every month, beyond what it brings in. **Runway** is how many months of cash remain at the current burn rate before the company runs out of money entirely — literally, cash in the bank divided by monthly burn. Every founder should be able to state their own runway, in months, without having to go look it up. It's the single number that determines how much time exists to hit the next milestone before a company is forced into an emergency fundraise, or worse.



The Core Idea: Keep It Standard, So You Can Stop Thinking About It

Every recommendation in this chapter follows from one underlying principle, stated plainly by Carolynn Levy and Kirsty Nathoo, who handle exactly this kind of work for Y Combinator companies: use the most standard, most common structure available at every decision point, and save your actual attention for the parts of the business that are genuinely novel. A startup's legal and financial structure is not the place to be creative. The Connecticut-LLC story above is what happens when a founder treats a solved, standardized problem as though it needed a custom solution.



Why It's True: Incorporation, Equity, Fundraising Documents, and the Rules of Being an Employer

Incorporation: Delaware, for reasons that compound over time

A startup needs to exist as a separate legal entity, mainly to protect its founders from personal liability if the company is ever sued — the company's assets are at risk, not the founders' personal bank accounts. Delaware is the default choice for nearly every venture-backed startup, not because of any special local advantage, but because Delaware's corporate law is unusually well-tested and predictable, and because investors are already comfortable with it — every deviation from that default adds friction and cost to every future financing round, exactly the kind of avoidable cost the opening story illustrates.

Actually incorporating involves a handful of standard documents: articles of incorporation (the two-page filing that creates the shell), bylaws, a board of directors, officer titles (Delaware requires a CEO, President, and Secretary to be named, even at a two-person company), and IP assignment agreements — paperwork confirming that anything a founder invents or codes belongs to the company, not to them personally. Services like Clerky handle this with standard, inexpensive templates; there's rarely a good reason to pay a law firm to draft custom versions of documents this well-trodden. Whatever gets signed, keep the signed originals somewhere safe and organized — the moment they matter most is almost always a high-stress one, like a due-diligence process during a large fundraise or an acquisition, and "we're not sure where those are" is a genuinely bad thing to have to say at that moment.

Equity: split close to equal, and vest everyone including yourself

Chapter 4 already covered the *decision* to split cofounder equity close to equally, and to make that decision early. This section covers the mechanics that actually implement that decision.

First, a founder doesn't simply own shares by agreeing to found the company — buying shares from the company (in exchange for cash, or for assigning IP and past work to it) is a real, two-way legal transaction, formalized in a stock purchase agreement. Skipping this step because "we already agreed on the split" is a common and avoidable mistake.

Second, that stock is almost always **restricted stock** — meaning it vests over time, rather than being fully owned the moment it's purchased. The standard structure in Silicon Valley is **four years, with a one-year cliff**: nothing vests until the first anniversary of the

grant, at which point 25% vests all at once, and the remaining 75% vests monthly over the following three years.

Figure 15.1 — A standard 4-year vesting timeline, with a 1-year cliff

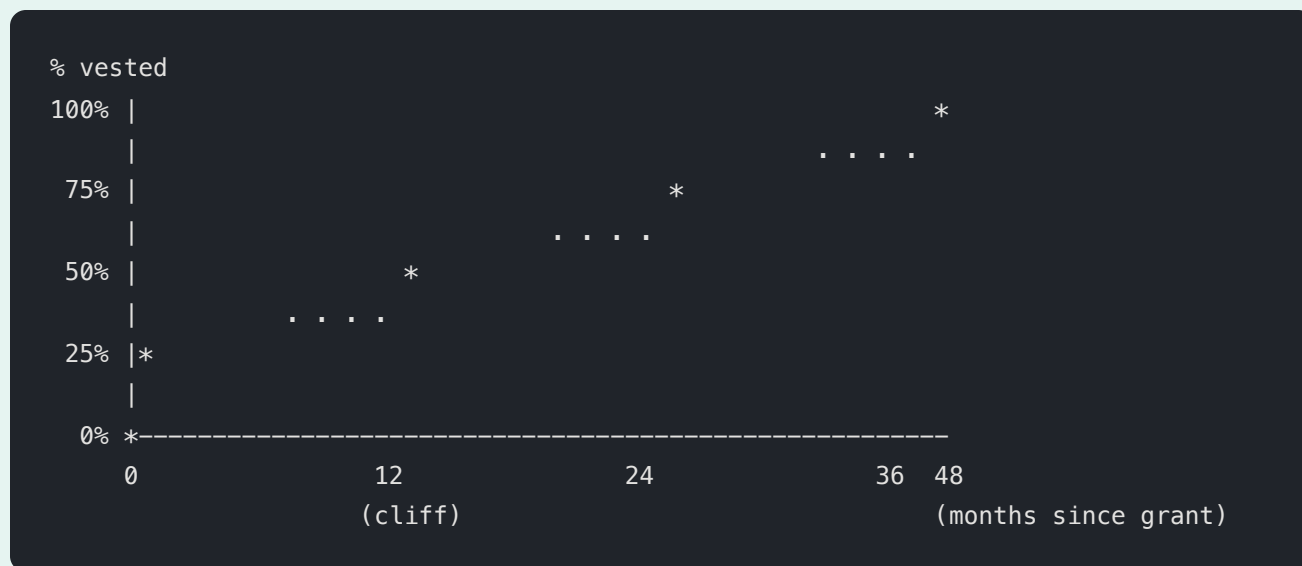


Table: A founder's vesting timeline on a standard 4-year grant with a 1-year cliff

Time since grant	Percentage vested	What happens if the founder leaves now
Before 12 months	0%	Company repurchases 100% of shares at original price
Exactly 12 months (the cliff)	25%	Founder keeps 25%; company repurchases the remaining 75%
24 months	50%	Founder keeps 50%; company repurchases the remaining 50%
36 months	75%	Founder keeps 75%; company repurchases the remaining 25%
48 months	100%	Founder keeps all of it — nothing left to repurchase

Vesting exists for a specific reason: without it, a founder who leaves after a few months walks away with a permanent, full share of a company they no longer work on, which is

both unfair to the founders who stayed and, in practice, close to fatal for future fundraising — investors will rarely fund a company with a large, inert equity stake sitting with someone no longer involved. This applies to solo founders too, not just teams: an unvested solo founder sets the wrong precedent for every future employee whose own equity will vest, and signals a level of commitment investors will specifically check for.

One more filing deserves special attention because it can't be fixed later if missed: the **83(b) election**, a one-page IRS form that must be filed within 30 days of receiving restricted stock. Filing it locks in a founder's tax basis while the company's value is still low (often close to zero), rather than owing tax later based on the stock's value each time a chunk vests, which can turn into a serious, avoidable tax bill as the company becomes more valuable. Missing this specific 30-day window is one of the few mistakes in this chapter that genuinely cannot be undone after the fact — send it certified mail, and keep the proof.

Fundraising documents: what you're actually signing

Chapters 13 and 14 covered the strategy and the pitch. This section covers what the paperwork that follows an investor's "yes" actually says.

An **unpriced round** — usually called a seed round — doesn't set an exact valuation for the company. Instead, an investor signs a **SAFE** (simple agreement for future equity) or a **convertible note**, handing over cash now in exchange for the right to receive actual shares later, once a **priced round** (typically a Series A or later) sets a real valuation for the first time. Until that conversion happens, a SAFE or note holder isn't a shareholder yet and has no voting rights — they're holding a promise of future shares, not shares themselves.

The mechanism that rewards an investor for taking on early risk is the **valuation cap**: an upper limit on the valuation used to calculate how many shares their investment converts into, regardless of what the company is actually worth by the time a priced round happens. Here's the mechanic worked through with real numbers:

An investor puts in \$100,000 on a SAFE with a \$5 million valuation cap. A year later, the company raises a priced Series A at a \$20 million valuation. Because the SAFE converts at the lower of the cap or the actual round price, this investor's \$100,000 converts as though the company were worth \$5 million, not \$20 million — roughly

four times more favorable than a new investor coming in at the \$20 million price for the same dollar amount. That better price is the reward for having taken the risk of investing before there was a real valuation to point to.

The dilution this produces is easy to underestimate if you don't run the numbers in advance. Suppose a company raises \$2 million across several SAFEs, all with a \$6 million cap. Under the cap convention this course described — where the cap is treated as a *pre-money* figure, meaning the money being raised gets added on top of it — the resulting post-money value at conversion is \$6 million plus the \$2 million raised, or \$8 million, and the SAFE holders collectively own $\$2 \text{ million} \div \8 million : roughly **25%** of the company once those SAFEs convert (not simply \$2 million divided by the \$6 million cap, which is a different, larger number — the extra step of adding the raise to the cap first is exactly the calculation a pre-money-style cap requires, and exactly the calculation the newer post-money SAFE format was designed to make unnecessary). If the same priced round that triggers this conversion also brings in new investors who want to own 20% of the company themselves, the founders have now given up close to 45% of the company in a single round — a number that's easy to miss if you only look at the priced round's headline terms and forget to add back everything that was already promised through earlier SAFEs.

Appendix A works through a complete, multi-round version of this same math in detail — founding through a Series A, with actual share counts — using the cleaner post-money SAFE convention described in the note below, specifically so the arithmetic at each step stays checkable rather than compounding into the kind of easy-to-miss error this section just walked through by hand.

A note on how this has evolved: *the mechanics above reflect the standard structure at the time this course was taught. Since then, Y Combinator introduced an updated, "post-money" version of its standard SAFE template, specifically to make this kind of dilution easier for founders to calculate in advance rather than after the fact. The underlying tradeoff — a valuation cap rewards early risk, and stacking several rounds of SAFEs compounds dilution in ways worth modeling before you sign — hasn't changed.*

Investors typically ask for a few standard things beyond the core investment, and it's worth knowing which requests are normal and which deserve real pushback. A **board**

seat request should usually be declined unless the specific investor will add real, ongoing strategic value — a board seat isn't a favor to hand out for the size of a check alone. **Advisor** arrangements should generally come free with an investment, not require additional equity on top of it; an investor asking for extra shares in exchange for occasional introductions is, bluntly, asking for a freebie. **Pro-rata rights** — the right to invest again in future rounds to maintain a fixed ownership percentage — are common and not inherently a bad request, but they do mean founders bear correspondingly more dilution over time as investors exercise that right. **Information rights** — regular updates, typically monthly — are reasonable and often genuinely useful, since investors who are kept informed can help with introductions and hiring; a request for weekly reports or granular budget oversight has usually crossed into overreach.

Running the business: whose money is it, and who's actually an employee

Once money is in the bank, it belongs to the company, not personally to the founders — a distinction that sounds obvious and is nonetheless violated often enough that Nathoo and Levy tell a specific cautionary story about it: a founder who used investor money for a personal trip to Las Vegas is, in their blunt framing, no longer with the company, and for good reason — that's not a gray-area expense, it's converting investor money to personal use. The practical test for anything less obviously egregious: if you had to read an investor a line-by-line list of what you spent their money on, would any single line embarrass you? If yes, it's probably not a legitimate business expense.

Founders themselves are legal employees of the company they founded, not exempt from labor law by virtue of being the boss — which means founders must actually be paid, at least a minimum wage, and the company must handle payroll taxes on that pay correctly. Skipping this is not a minor technicality: one company that avoided payroll taxes on founder pay for three years created a serious, expensive legal problem for itself, with criminal liability a real (if rare) possibility in extreme cases. A related, less obvious cost of skipping this: if a cofounder relationship later breaks down and one founder is let go, unpaid wages become real leverage for that departing founder to extract extra equity or vesting acceleration in exchange for signing a release — a problem this section of the chapter, alongside Chapter 4's advice to decide vesting and equity early, exists specifically to prevent. The practical fix is simple: run payroll properly for founders from day one, the same as for any other employee, and treat the arrangement with the same

seriousness Nathoo and Levy suggest thinking of as a "marital pre-nup" for the relationship — decided in advance, not renegotiated under stress later.

Hiring anyone else requires knowing whether they're a **contractor** or an **employee**, a distinction the IRS cares about and penalizes getting wrong. A contractor sets their own hours and location, uses their own equipment, works toward a defined deliverable, and receives a 1099 form at year-end with no taxes withheld by the company. An employee has taxes withheld by the company and receives a W2. Both need to sign IP assignment agreements; only one of them requires the company to carry workers' compensation insurance and verify legal work authorization. A payroll service (rather than manually handling this) is worth paying for from the first hire — the cost of getting this wrong, as with the three-year payroll-tax story above, is far larger than the cost of the service itself.

Firing someone, when it's necessary, should happen quickly rather than being delayed — Chapter 5 already covered why waiting is usually worse for everyone involved, including the person being let go. The legal and financial mechanics that go with a firing: pay all outstanding wages and accrued vacation immediately, since this is a legal requirement in most places, not a negotiable timeline; cut off physical and digital access right away, including shared accounts (one company learned this the hard way when a departing founder held the company's GitHub account password as leverage during a dispute); and repurchase any vested shares from a departing employee promptly, per the terms already set by their vesting agreement.

Common Mistakes

- **Using a non-standard entity structure or state of incorporation to save a small amount of money upfront.** The Connecticut LLC story is the clearest illustration of how a small shortcut here compounds into an enormous cost later, right when the company can least afford the distraction.
- **Treating a founder's cofounder agreement as settled without buying the stock and putting vesting in place.** A verbal agreement on equity split isn't the same as an actual, documented, vesting stock purchase.

- **Missing the 83(b) election's 30-day window.** This is one of the only mistakes in this chapter that cannot be fixed after the fact.
- **Looking only at a single round's headline dilution, without modeling the cumulative effect of prior SAFEs or notes converting at the same time.**
- **Spending investor money in a way you'd be embarrassed to itemize for them directly.**
- **Not paying founders a real salary, or not running proper payroll,** which creates both a legal liability and, in the event of a founder dispute, real leverage for the departing founder.
- **Misclassifying an employee as a contractor (or the reverse),** which the IRS penalizes and which leaves the company without required protections like workers' compensation coverage.



Tradeoffs and Edge Cases

Is Delaware always the right choice? For the venture-backed, hyper-growth companies this book focuses on, yes, close to universally — the benefits of investor familiarity and legal predictability outweigh any state-specific advantage almost every other state might offer. A small business not pursuing outside venture capital has more real flexibility here, since it will never face a Delaware-sophisticated investor's due diligence process in the first place.

Do solo founders really need the same vesting discipline as teams? Yes, for reasons distinct from the team case. A team needs vesting to protect against one founder leaving with a large, unearned stake. A solo founder needs it for a different reason: it signals real commitment to investors, and it sets the correct precedent for every employee who joins later and will be expected to vest their own equity the same way.

Are all investor requests for extra rights (board seats, pro-rata, information) red flags? No — the point of this chapter isn't that every request should be refused. It's that each request should be evaluated on whether it reflects real, ongoing value (a board seat from someone who will genuinely help), a standard and reasonable ask (basic information

rights, pro-rata), or overreach dressed up as a normal request (an advisor wanting extra equity for occasional introductions). Treating every request identically, in either direction, misses the actual judgment this section is trying to teach.

Applying the Test

Using this chapter's dilution table as a template, build your own rough cap table projection for your actual or hypothetical company across a seed round and a Series A. Include an employee option pool of 10–15% at the Series A stage. What's your own realistic ownership percentage after both rounds, and does that number still feel motivating? If the honest answer is no, that's exactly the signal Chapter 13 warned about — worth addressing at the term-sheet stage, not after the round has already closed.

Chapter Summary

- Learn the basic financial vocabulary — P&L, balance sheet, cash flow, burn rate, runway — before anything else in this chapter; every founder should know their own runway in months without looking it up.
- Use standard, well-tested structures (Delaware incorporation, standard vesting, Clerky- style templates) rather than custom solutions — the Connecticut LLC story shows how expensive a shortcut here can become.
- Vesting (typically four years with a one-year cliff) protects the company from a founder leaving early with a large, unearned stake, and applies to solo founders too.
- File the 83(b) election within 30 days of any restricted stock grant — one of the few mistakes in this chapter that can't be corrected after the fact.
- A valuation cap on a SAFE or note rewards early investors with a better effective price once a priced round sets a real valuation — model the cumulative dilution across multiple rounds before you sign, not just the headline number of the round you're currently negotiating.

- Evaluate investor requests (board seats, advisor equity, pro-rata rights, information rights) individually, based on real value versus overreach, rather than accepting or rejecting all of them uniformly.
- Founders are real employees who must be paid and covered by proper payroll — skipping this creates both tax liability and leverage for a departing cofounder later.
- Contractors and employees are legally distinct categories with different tax and insurance obligations — get the classification right from the first hire.



Exercises

1. **Recall:** Explain what a valuation cap actually does, using the worked \$100,000-on-a-\$5-million-cap example from this chapter.
2. **Apply:** Calculate your own company's current runway (or a hypothetical one) in months, using the burn-rate definition from this chapter's opening section.
3. **Apply:** Build a simple projected cap table across a seed round and a Series A for your own company, including an option pool, using this chapter's worked table as a template.
4. **Confront a counterexample:** This chapter argues board seats should usually be declined unless an investor adds real ongoing value. Under what specific circumstances would giving up a board seat actually be the wrong call, even for an investor who isn't especially strategic?



Further Reading

- Y Combinator's own published library of standard startup legal documents (the same templates referenced throughout this chapter) is the most direct, practical next step for founders about to incorporate or raise a round.

- Appendix A of this book expands the cap table walkthrough in this chapter into a full, multi-round numeric example with an employee option pool included at each stage.
- Appendix B covers how to read an actual term sheet clause by clause, building directly on the vocabulary this chapter introduces.



Chapter 16 — Selling: First Customer to Enterprise Deal

Aaron Levie, who built Box into a company serving hundreds of thousands of businesses, opens his own account of choosing enterprise software with a joke at his own expense: enterprise sales, in the popular imagination, means a guy named Chuck showing up with a briefcase to sell software nobody's excited about. Consumer startups, by contrast, sound like parties. The real numbers tell a different story. Consumer app spending and global digital advertising combined come to something like \$170 billion a year. Enterprise IT spending — the servers, software, and services businesses buy every year — is roughly \$3.7 trillion. Levie chose the unglamorous option on purpose, because the actual pool of value being spent there dwarfs the "exciting" one by more than twenty times over.

This chapter covers selling in both directions this book has already set up: the early, personal, founder-led sales process every company goes through with its first customers (covered here through Tyler Bosmeny's account of building Clever), and the specific patterns that make enterprise software sellable at all (covered through Levie's account of building Box). Both are really the same skill Chapter 7 already introduced — talking to users — applied at the point where a conversation turns into a signed contract.



The Core Idea: Sales Is a Funnel, and Founders Are the Sales Team

Selling, done properly, is a funnel: prospecting (finding people who might buy), conversations (understanding what they need and whether you solve it), and closing (getting an actual signature and payment). Nothing about this requires a background in sales — Bosmeny's own path to founding Clever ran through a sales job he took almost by accident, and his central claim is that a founder's genuine passion for the product and deep understanding of the problem outweigh formal sales experience by a wide margin.

The uncomfortable part for many technical founders: this means the founders themselves have to actually do it, especially early — there is no version of "hire the sales people later" that works before a company has real signal about what's actually being sold.

Why It's True: The Funnel, the Contract Traps to Avoid, and What Changed to Make Enterprise Software Sellable

Prospecting is a numbers game, and the numbers are worse than they feel

Everett Rogers's technology adoption curve puts a specific number on a discouraging but useful fact: only around 2.5% of any market — the "innovators" — will seriously consider adopting something built by a company with no users, no revenue, and no track record. Framed as a discouraging statistic, that's demoralizing. Framed correctly, per Bosmeny, it's a numbers problem: if only 2.5% of a market will even take your call, the volume of outreach has to be large enough to reach that thin slice at all. His own experience at Clever: over four hundred companies emailed in the first two months of Y Combinator, just to start getting real conversations going.

The two channels that produced results most reliably: a founder's personal network (unglamorous, but real), and conferences — attended with real preparation, meaning the attendee list obtained in advance and meetings requested before ever arriving, so every minute on-site is spent with someone worth talking to rather than wandering a show floor. Cold email works too, with a specific, deliberately short template: who you are, what you're building, a request to talk, and a proposed time — long enough to be genuine, short enough to actually get read.

On the call, the discipline is silence, not eloquence

The single biggest surprise for most first-time founders doing sales calls: the best salespeople talk less, not more. Bosmeny's own benchmark, borrowed from watching top

performers, is roughly 30% talking and 70% listening — asking questions like why someone agreed to take the call at all, how they solve the problem today, and what an ideal solution would look like to them, rather than walking through every feature of the product being sold. The instinct to fill the silence with detail about how great the product is works against a founder here, not for them — every minute spent talking is a minute not spent learning what would actually make someone buy.

Follow up relentlessly, and get to a real yes or no

A specific, humbling detail from Clever's early sales process: Bosmeny once documented the full sequence of touchpoints — emails, calls, voicemails — that it took to close the company's second-ever deal. It ran to roughly sixty separate steps, several of which were simply re-sending an email that had gone unanswered the first, second, and third time, from a buyer who ultimately did want the product. The lesson isn't that persistence alone guarantees a sale. It's that even a genuinely interested buyer's silence rarely means no — it usually just means busy, and giving up after two unanswered emails throws away deals that were still winnable.

The more important discipline sits on the other side of this: a founder's time is the scarcest resource in an early company, and it can't be spent chasing every lead forever. The goal of follow-up isn't just persistence — it's getting to a definite yes or a definite no as fast as possible. A stalled pipeline full of "maybes" is worse than a shorter pipeline with clear answers, because a maybe consumes exactly as much attention as a live deal while producing none of the results.

Two specific traps in closing: the feature request and the free trial

Once a deal is close to closing, two very reasonable-sounding customer requests routinely derail it, and both deserve a specific, prepared response rather than an improvised one.

The first is the feature-gating request: "we'd use this, but only if it had X." The instinct is to go build X immediately. The better response is one of two things: either get the customer to sign a real agreement committing to the deal contingent on that feature shipping, so the request has real weight behind it — or wait until multiple, independent customers ask for the same thing before building it at all, since a single customer's request is often idiosyncratic rather than a sign of broader demand.

The second is the free-trial request. It's a reasonable ask from the customer's side — nobody wants to commit blind — but a free trial gives a founder none of what an early company actually needs: no revenue, no real validation, no binding commitment. The better structure is an annual agreement with a genuine opt-out window in the first 30 to 60 days, which gives the customer the same practical protection a free trial would, while still producing a real signed commitment and real revenue from day one rather than a renegotiation months later.

Once a deal is signed, the actual redlining process shouldn't become a monthslong argument over minor clauses — starting from a standard, well-tested agreement (Y Combinator has published open-source versions of exactly this kind of document) and signing promptly once the substance is settled beats winning every small point at the cost of momentum.

Knowing which business you're actually building

Once a founder has closed enough deals to see a pattern, a useful frame from entrepreneur Christoph Janz helps clarify what kind of company is actually being built: a \$100 million company can come from roughly a thousand customers paying \$100,000 a year ("elephants"), from ten thousand customers paying \$10,000 a year, or from a hundred thousand customers paying \$1,000 a year ("rabbits"), among other combinations along the same curve. Each end of that spectrum implies a completely different sales process — an elephant-sized deal justifies months of relationship-building and a dedicated account executive; a rabbit-sized deal has to close through an almost entirely self-serve, low-touch process, because the deal size can't support a human sales cycle at all. Building a high-touch sales process for a rabbit-priced product, or the reverse, is a mismatch that shows up as either an unsustainable cost of sales or a sales cycle too slow to ever close.

What changed to make enterprise software worth building at all

Levie's own case for choosing enterprise, beyond the sheer size of the market, rests on a specific "why now" argument — the same discipline Chapter 2 established for evaluating any idea. Several structural shifts converged over roughly the decade before Box was built: computing moved from expensive, owned, on-premise servers to cheap, on-demand cloud infrastructure; software moved from heavily customized, bespoke platforms toward

standardized products extended through APIs; the same product became able to serve both a two-person company and an organization of hundreds of thousands of employees, rather than enterprise software requiring an enormous budget just to get started; platforms began reaching international customers within weeks instead of the years it once took to expand a sales organization abroad; and, perhaps most significantly, the arrival of smartphones shifted technology adoption inside companies from a top-down, IT-department decision to something individual employees brought in themselves, which changed the actual sales motion from "convince the CIO" to "win over the user first, and the organization follows."

That last shift deserves its own emphasis, because it connects directly to a pattern this book has now seen from three completely independent sources. Levie's own advice for entering enterprise markets is to start intentionally small — find a narrow wedge that slips into the gaps of whatever existing tools a company already uses, rather than attempting to replace an entire suite on day one, and expand from there once the initial wedge has proven itself. Gusto (originally ZenPayroll) is his example: it started as just payroll for small businesses, a narrow, unglamorous slice of a much larger HR-and-benefits problem, and expanded into adjacent services only after establishing itself in that one specific wedge. This is, structurally, the identical concentric expansion pattern Chapter 2 introduced from Sam Altman's lecture and Chapter 3 independently confirmed from Peter Thiel's — a third, unrelated speaker, in a completely different context, arriving at the same underlying shape. That kind of repeated, independent convergence is about as strong a signal as this book can offer that a pattern is genuinely load-bearing rather than one company's specific story.

A second pattern worth carrying forward from enterprise software specifically: look for **asymmetries** — things an established, incumbent competitor structurally can't or won't do, because doing so would break their existing business model. Levie's example is Zenefits, an HR platform that didn't charge small businesses directly for its software at all; it earned money instead through commissions from the insurance companies whose plans it helped employees select, which let it undercut every existing competitor's pricing in a way none of them could copy without abandoning their own revenue model entirely. This is worth connecting explicitly to Chapter 3's monopoly framework: an unusual, hard-to-copy business model is itself a form of durable advantage, not just a clever pricing trick.

Finally, enterprise software specifically benefits from resisting a false choice between consumer-grade design and effective sales. The best enterprise products keep the same attention to usability and design a consumer product would demand — Levie's own framing is that this makes adoption easier and makes the product more likely to spread inside an organization on its own — while still recognizing that a real sales function, helping a customer navigate procurement, competition, and internal buy-in, remains necessary alongside good design, not replaced by it. The two aren't in tension; they solve different parts of the same problem.

Common Mistakes

- **Treating a "maybe" as an acceptable long-term outcome.** A stalled pipeline of maybes drains as much founder attention as real deals, while producing nothing.
- **Building a requested feature immediately for a single customer,** rather than getting a binding commitment first, or waiting for the request to repeat across multiple independent customers.
- **Giving away free trials to reduce customer friction.** This produces neither real revenue nor real validation — a short opt-out window on a signed agreement achieves the same customer protection while still producing a real commitment.
- **Over-negotiating minor contract clauses for weeks,** at the cost of the momentum a faster close would have preserved.
- **Building an elephant-style, high-touch sales process for a rabbit-priced product, or the reverse** — a mismatch between deal size and sales motion that shows up as either unsustainable cost or an impossibly slow cycle.
- **Assuming enterprise software must sacrifice good design and usability,** or that a well-designed product needs no real sales function at all — both are false choices.

Tradeoffs and Edge Cases

Is there ever a legitimate case for a free trial? Yes — the objection in this chapter is specifically to giving away a trial as a default, reflexive response to customer hesitation, not to every possible trial structure. A short, clearly bounded opt-out window attached to a real signed agreement captures most of the same benefit for the customer while preserving a real commitment for the company, which is why this chapter recommends that structure over an open-ended free trial rather than rejecting trials outright.

Does every early-stage company really need to close deals through founder-led, manual sales, even if the long-term plan is a self-serve, low-touch model? Largely yes, at least at the very beginning — even Mixpanel, an example of a genuinely product-led company where developers can sign up with no sales conversation at all, still layers an inside sales process on top once individual developer adoption inside a company signals a larger organizational deal is available. Chapter 9's "doing things that don't scale" principle applies directly here: manual, founder-led sales early on is how you learn what a scalable, self-serve version of the same process should actually look like.

Does the "small wedge first" pattern from enterprise software ever not apply? It's worth being honest that some enterprise categories genuinely require a broad, full platform from day one to be usable at all — Marc Andreessen's point in Chapter 13, that enterprise software categories sometimes require a full product before any customer will buy at all, rather than an MVP a consumer company could ship. The wedge strategy works best when a genuinely useful narrow slice of value can stand on its own; it's a poor fit for products where partial functionality isn't valuable to anyone.



Applying the Test

Using Christoph Janz's elephants-to-rabbits framework, place your own company (or a hypothetical one) on the spectrum: roughly how many customers, at roughly what price point, would it take to reach a meaningful revenue milestone? Then check whether your actual sales process — how much founder time goes into each deal, how much customization each customer expects — actually matches that position on the spectrum, or whether it's built for a different kind of company than the one your pricing implies.



Chapter Summary

- Selling is a funnel — prospecting, conversation, closing — and founders, not professional salespeople, have to run it in a company's early days.
- Only a small slice of any market (roughly 2.5%, the "innovators") will consider buying from a company with no track record — treat early sales as a numbers game requiring real volume, not a sign something is wrong if most people say no.
- On calls, listen far more than you talk — roughly 70% listening, 30% talking, per the benchmark of top-performing salespeople.
- Follow up far more persistently than feels natural, and prioritize getting to a real yes or no over accumulating a pipeline of unresolved maybes.
- Avoid two specific traps: building a single customer's requested feature without a real commitment behind it, and giving away free trials instead of a bounded opt-out window on a signed agreement.
- Know whether your business is an "elephant" (few customers, high price) or a "rabbit" (many customers, low price), and match your sales process to that reality.
- Enterprise software became newly viable through a specific set of structural shifts — cloud infrastructure, standardized APIs, mobile-driven user-led adoption — the same "why now" discipline Chapter 2 established for evaluating any idea.
- The "start with a narrow wedge and expand" pattern shows up independently across three completely different lectures in this book (Altman, Thiel, Levie) — a strong signal it's a real, load-bearing structure rather than one company's specific story.



Exercises

1. **Recall:** Explain why Bosmeny recommends talking roughly 30% of the time and listening 70% on a sales call, and what specifically goes wrong when a founder reverses that ratio.
2. **Apply:** Using the elephants-to-rabbits framework, decide where your own company or idea sits on the spectrum, and describe the sales process that would actually match it.

3. **Apply:** Draft the specific commitment you'd ask a customer to sign before building a feature they've requested, rather than simply building it on request.
4. **Confront a counterexample:** This chapter argues for finding a narrow wedge and expanding, citing three independent examples. Are there enterprise categories where starting narrow is actually the wrong strategy, and a broader, more complete product is required from day one? What would distinguish those categories from the wedge-friendly ones?



Further Reading

- Tyler Bosmeny's original CS183B lecture includes the full sixty-step follow-up sequence from Clever's second deal, in more detail than this chapter's summary.
- Aaron Levie's lecture covers additional enterprise patterns (living-on-the-edge customers, modularizing rather than customizing) beyond what this chapter had room to cover in full.
- Christoph Janz's blog post "Five Ways to Build a \$100 Million Business" is the original source of the elephants-to-rabbits framework covered in this chapter.



Chapter 17 — Execution: Focus, Speed, Momentum

Imagine a consumer web product where every existing user brings in, on average, 0.99 new users. Run that forward, and the product eventually flatlines and dies — not dramatically, just a slow fade as the number keeps compounding downward. Now imagine the same product with a viral coefficient of 1.01 instead of 0.99 — a difference of two one-hundredths, barely visible in a spreadsheet. Run that forward, and the product grows exponentially, forever.

Sam Altman uses that comparison to make a point this whole chapter is built around: the gap between failure and runaway success is frequently much smaller than it looks from the outside, and it's usually made up of small, unglamorous amounts of extra effort and attention, sustained consistently, rather than one dramatic breakthrough. Execution is the discipline of finding and sustaining that small edge, day after day, long after the excitement of the original idea has worn off.



The Core Idea: Two Questions, and a CEO Job Nobody Talks About

Execution comes down to two separate questions: can you figure out what to do, and can you actually get it done. Most of this book — the chapters on ideas, monopoly, products, and users — is about the first question. This chapter is entirely about the second, which breaks down into two further components: **focus** (correctly identifying the two or three things that matter today, out of the hundred things competing for attention) and **intensity** (the sustained, high-effort follow-through that turns a correct priority into an actual result).

Altman names four commonly recognized jobs of a CEO — set the vision, raise money, evangelize the mission, hire and manage the team — and adds a fifth that gets far less attention but that only the CEO can actually do: **setting the execution bar**. Nobody else

in the company can define what "good enough" and "fast enough" mean at a level that the whole organization then absorbs and imitates. This chapter is about how to do that job well.

Why It's True: Saying No Constantly, Moving Fast



Without Breaking Quality, and Getting Back Momentum Once It's Lost

Focus means saying no about ninety-seven times out of a hundred

On any given day, a founder faces something close to a hundred things that each feel important — a partnership opportunity, a conference invitation, an adviser to recruit, a piece of press to chase, alongside the two or three things that actually determine whether the company survives the month. Altman's estimate of the actual ratio: founders need to say no to roughly ninety-seven out of every hundred of these, keeping only the handful that genuinely matter right now.

One of the more sobering aspects of running a startup is captured in a phrase worth keeping close: there's no credit for trying. A team that works extremely hard on the wrong things still fails, and fails without the market caring how hard the effort was. This is why the specific practice of setting a small number of company-wide goals — and then repeating them far more often than feels necessary, on walls, in one-on-ones, in every all-hands meeting — matters as much as picking the right goals in the first place. A goal that's set once and never mentioned again functions, in practice, as no goal at all, because a company of even five or six people will quietly drift toward whatever each individual privately assumes matters most.

Growth and momentum are the thing you can never stop paying attention to

Altman calls growth and momentum the "prime directive" for running a company — the one thing, if he could only tell a founder one thing about operating a startup, he would

choose. A weekly review of the metrics that actually matter (Chapter 10 already covered which metrics those should be) is one of the most reliable forcing functions available, and it's worth treating any explanation for *not* focusing on growth — "we're doing a rebrand instead," "we're not shipping this quarter because we're focused on something else" — with real suspicion. These explanations are almost always describing a company about to be in real trouble, dressed up as a deliberate strategic choice.

A specific, avoidable distraction deserves its own callout, because it directly connects to Chapter 12's warning: getting excited by a company's own press coverage. It's genuinely easy to get real attention with no underlying results, and it feels, in the moment, like genuine progress — right up until a year later, when the coverage is forgotten and the company has nothing durable to show for the attention it generated. The correct posture toward competitors making noise in the press follows the same logic, in reverse: a competitor's press release is far easier to produce than a competitor's working product, which is itself far easier to produce than a competitor actually displacing you with it. Henry Ford's own formulation of this, worth keeping close: "The competitor to be feared is one who never bothers about you at all, but goes on making his own business better all the time." The startups that make the most noise about you are, by that same logic, rarely the ones actually catching up.

Intensity means moving fast without letting quality slip — both, not either

The viral-coefficient example that opened this chapter illustrates why intensity compounds so dramatically, but the underlying discipline isn't just "work more hours." A specific, harder version of intensity is holding two things simultaneously that feel like they should be in tension: moving with real speed, and maintaining a genuinely high quality bar, at the same time. Facebook's own internal motto — "move fast and break things" — coexisted the entire time with an intense, specific obsession over product quality inside the company; the speed applied to iteration and shipping cadence, not to tolerating sloppy work. Altman's blunt summary of why this combination matters: "you don't buy people shitty computers if you don't want them to write shitty code." A company's tolerance for mediocrity in any one visible place — the tools it gives employees, the polish of a minor feature, the care put into a small interaction — tends to set the expectation for every other decision made across the whole organization.

Alongside speed and quality sits decisiveness. Indecision is one of the more reliable startup killers Altman has observed: founders who talk endlessly about several possible directions, without ever actually committing to and executing one of them, lose to founders who pick a reasonable option and move — even when the option chosen isn't obviously the best one on paper. A strong, if slightly counterintuitive, predictor of which YC founders will succeed: whether they've gotten something new, concrete, and done every single time you check in with them. Big outcomes accumulate from many small, finished pieces of work delivered on a steady cadence — not from a founder disappearing for a year and reappearing with one enormous, finished achievement. Altman's own concrete example of what showing up looks like in practice: early in his own company's life, they were about to lose a major first customer to a competitor who'd already signed a deal. Rather than accept the loss, they got on a plane, flew overnight, and were sitting in the prospective customer's office at 6 a.m. — turned away at first, waiting anyway, until someone finally agreed to talk. The company tore up its agreement with the competitor and signed with Altman's team instead about a week later. His advice, when people say "get on a plane in marginal situations," is that it's meant almost literally, not just as a metaphor for effort.

Recovering momentum once it's genuinely lost

Momentum, once lost, doesn't come back through an inspiring speech about the company's long-term vision — save vision speeches for when the company is already winning, because a team that isn't winning doesn't want to hear about the destination; it wants to see evidence that things are actually turning around right now. The reliable way back is through small, real wins, compounding — a board member's observation that "sales fix everything in a startup" is, in Altman's account, genuinely true, and the broader version of that lesson generalizes past sales specifically: find whatever small, achievable wins are available, and get them, rather than reaching for a grand narrative first.

Facebook's own experience with this, in 2008, is a useful concrete case: growth had slowed enough to be a real, company-wide concern, and Mark Zuckerberg's response was to form a specific, focused "growth group" tasked with finding small, individually unglamorous improvements — the kind of work that, described in isolation, sounds unremarkable. Collectively, those small improvements moved Facebook's growth curve back up, and the group became, according to people who worked there at the time, one of the most prestigious internal assignments at the company — not despite how

unglamorous the individual work looked, but because everyone could see it was the thing that had actually turned the company around.

A related, practical rule for team disagreement during a momentum slump: when the team is arguing about what to do next, the tiebreaker is asking users and following what they say, rather than letting the disagreement calcify into a political or personal fight. And naming the underlying problem out loud — "things aren't working right now, we don't actually hate each other, we just need to get back on track" — tends to defuse tension that would otherwise fester silently and make the slump worse.

Common Mistakes

- **Working hard on the wrong two or three things.** There's no credit in a startup for effort alone — only for the market actually wanting what got built.
- **Setting company goals once and never repeating them.** A goal mentioned a single time functions, in practice, as no goal at all once a company grows past a handful of people.
- **Getting excited by the company's own press coverage,** or by a competitor's, instead of staying focused on real growth and product quality.
- **Treating speed and quality as a tradeoff you have to choose between,** rather than two standards a company can and should hold simultaneously.
- **Responding to lost momentum with an inspirational vision speech,** instead of chasing the small, concrete wins that actually rebuild it.
- **Letting team disagreement during a difficult stretch turn into an unspoken, festering conflict,** instead of naming the real problem directly and asking users to break the tie.

Tradeoffs and Edge Cases

Doesn't "move fast" conflict with the more deliberate pace Chapter 15 recommends for legal and financial mechanics? No — they apply to different categories of decision. Chapter 15's caution is specifically about structural, hard-to-reverse mistakes (a botched incorporation, a missed tax election) where speed genuinely isn't the priority because the downside of a mistake is so much larger than the cost of getting it right the first time. This chapter's advocacy for speed is about product decisions, shipping cadence, and responsiveness — areas where the cost of a wrong, fast decision is usually much smaller and much easier to correct than the cost of moving too slowly.

Does this chapter's advice apply the same way to hardware companies, given the Interlude's point about months-long iteration cycles? The underlying principles (focus, decisiveness, momentum) still apply, but the concrete practices have to adapt — "ship something new every time someone checks in" looks different when a single iteration takes sixteen weeks of tooling rather than a day of code changes. What transfers directly: the discipline of choosing two or three priorities and saying no to the rest, and treating slipped ship dates (the hardware equivalent of lost momentum) with the same seriousness this chapter recommends for a stalled growth curve.

Is "no credit for trying" too harsh a standard? It's worth taking at face value rather than softening it, because it corrects a real, common founder mistake: mistaking effort for progress. It doesn't mean effort is worthless — it means effort has to be aimed at something the market actually responds to in order to count, which is precisely why focus (choosing correctly what to work hard on) has to come before intensity (working hard once you've chosen).



Applying the Test

Write down what you actually spent your time and your company's money on this past week — not what you intended to prioritize, but what you actually did. Compare that list against the two or three goals you'd claim, if asked, are the company's real priorities right now. Where the two lists diverge is a more honest signal of your actual focus than any

stated intention, and it's the same test Altman uses to read what a founder truly believes matters.

✓ Chapter Summary

- Execution has two parts: figuring out what to do (covered elsewhere in this book) and actually getting it done — focus and intensity, this chapter's subject.
- Setting the execution bar is a CEO job nobody else in the company can do, and it's transmitted by example, not by memo.
- Focus means saying no to roughly ninety-seven out of a hundred demands on your attention, and repeating the few goals that remain far more often than feels necessary.
- Growth and momentum are the "prime directive" — treat any explanation for not focusing on them as a serious warning sign, and don't let your own press coverage (or a competitor's) distract from either.
- Intensity means holding speed and quality simultaneously, not trading one for the other — a small extra edge (the difference between a 0.99 and a 1.01 viral coefficient) compounds into the difference between failure and exponential growth.
- The best predictor of a founder's success is whether they've gotten something new and concrete done every time you check in — big outcomes accumulate from many finished small pieces, not one dramatic reveal.
- When momentum is lost, chase small, real wins instead of giving an inspirational speech, and use user feedback to break team disagreements before they fester.



Exercises

1. **Recall:** Explain why Altman says there's "no credit for trying" in a startup, and how that idea should change which two or three things a founder chooses to focus on.

2. **Apply:** List everything you spent real time and money on this past week, and compare it honestly against your stated top priorities. Where do the two lists disagree?
3. **Apply:** Identify one small, achievable win you could pursue this week if your own project's momentum had stalled — the equivalent of Facebook's 2008 growth group — rather than a larger, more dramatic pivot.
4. **Confront a counterexample:** This chapter argues speed and quality aren't actually in tension. Can you think of a real situation where a genuine tradeoff between the two existed — where moving faster really did require accepting lower quality, at least temporarily? What made that situation different from the cases this chapter describes?



Further Reading

- Sam Altman's original CS183B lecture (the second half of Lecture 2) covers additional detail on setting weekly company goals and the operating-rhythm practices this chapter only summarizes.
- Chapter 10's discussion of North Star metrics and weekly growth reviews is worth reading alongside this chapter's momentum section — they describe the same discipline from two different angles.
- Chapter 19 covers the more deliberate, systems-oriented side of running a company that complements this chapter's emphasis on speed and focus.



Chapter 18 — The Founder's Mind: Paradox and Judgment

When Reid Hoffman started telling people his idea for LinkedIn, roughly two-thirds of his own network — smart people, people whose judgment he generally trusted — told him it wouldn't work. Their reasoning was sound, as far as it went: a network product is worthless until it has enough people on it, and the first person to join gets nothing, because there's no one there yet. Critics weren't wrong about that dynamic. They were missing something Hoffman knew that they didn't: you don't need mass utility to get the first wave of users. You need people willing to join because they believe the vision is worth being part of — a professional network is inevitable, this is going to matter — even before the network is actually useful to them day to day. That belief-driven adoption gets you to critical mass, at which point the utility everyone was worried about finally kicks in.

That's the difference this chapter is built around: being contrarian is easy. Being contrarian *and correct* requires knowing something specific that the smart people disagreeing with you don't. This chapter covers that distinction, along with a set of recurring tensions — Reid Hoffman calls them paradoxes — that don't resolve into a fixed formula, and a specific tool, the investment thesis, for navigating them without guessing blindly.



The Core Idea: You Need to Know What Others Don't, and You Need a Way to Track Whether You're Right

Contrarian ideas only matter if they're actually correct, and the test for whether a contrarian idea clears that bar is specific: can you name a smart, informed person who would disagree with you, and can you explain exactly what you know that they don't? "Everyone thinks I'm crazy" isn't itself evidence of anything — it's just as often a sign of being wrong as a sign of being ahead of everyone else. What separates the two is whether there's a real, articulable piece of information or insight the skeptics are missing.

Once a founder is operating on a genuinely contrarian, correct insight, the deeper challenge is knowing, day to day, whether that insight is still holding up. Hoffman's tool for this is the **investment thesis**: a written, specific hypothesis for why the company will work, including exactly what you believe you know that the skeptics don't. The thesis isn't a plan you write once and file away — it's a live reference point you check constantly, tracking whether your confidence in it is increasing or decreasing as new information arrives. That single signal — rising or falling confidence — is what tells you whether to stay the course, adjust the tactics, or reconsider the whole approach, which is a far more precise tool than either blind persistence or reflexive doubt.



Why It's True: Paradoxes That Don't Resolve, and a Concrete Example of Tracking Confidence

The paradoxes founders actually live inside

Hoffman names a series of tensions that founders are told, at different times, to resolve in opposite directions — and his claim is that neither direction is categorically correct. Instead, a founder has to be able to operate at close to 100% of each side, at different moments, based on judgment about which the situation calls for.

Do the work yourself, or delegate it. Both, and sometimes fully one or the other at the same time — a founder who can only do hands-on work, or only delegate, has a real gap.

Flexible, or persistent. This is the paradox the investment thesis exists to solve. Advice to "stay the course" and advice to "listen to the data and pivot" are both correct, and the way to tell which one applies right now is to check whether your confidence in the underlying thesis is rising or falling — not whether things currently feel good. Hoffman offers his own company as the example: at PayPal in August 2000, the company was bringing in \$12 million, the expense curve was exponentiating, there was no revenue, and confidence in the thesis was genuinely decreasing — "a valley in the shadows moment," in Hoffman's phrase. A decreasing-confidence reading doesn't automatically mean quit; it means the immediate job becomes figuring out what specific action would restore confidence, rather than either grimly persisting or panicking. PayPal, LinkedIn, and Airbnb,

in Hoffman's own account, all had months that looked exactly like that one — which is itself the point: whether things currently look easy or feel hard isn't the signal to watch, since things feeling hard is close to a constant in every startup regardless of whether the thesis is actually working.

Belief, or fear. Belief is what keeps you moving forward through the difficulty this book's opening chapter already described. Fear — in the sense of genuine, honest attention to warning signs — is what keeps that belief from curdling into denial. A founder needs both running simultaneously, not one replacing the other.

Focus inward on the product, or focus outward on the network of people and information around you. Building in isolation misses signals only the outside world can provide. Spending all your time on relationships and information-gathering, with nothing actually getting built, is its own failure mode. Both, in shifting proportion.

Data, or vision. Data only means something inside the frame of a vision you already hold — a metric moving in a concerning direction tells you to reconsider your tactics, not necessarily to abandon the underlying belief that got you started. PayPal's own mission — a phrase the early team put on t-shirts — was to become a new global currency for the internet. The specific tactical model underneath that mission changed repeatedly: first a banking-style model, then a debit-card model, then eventually a mass-merchant payments model. The vision provided the constant "true north." The data told them, each time, which specific tactic to abandon in service of that same unchanging vision.

Take real risk, or minimize risk. The apparent contradiction resolves once you separate the two: take one genuinely large, focused bet — the specific thing you believe you understand that others don't — and then work hard to minimize every *other* risk around that one core bet, rather than treating the whole enterprise as a single undifferentiated gamble.

Short-term focus, or long-term vision. Lose sight of the long-term vision, and you'll eventually be lost in a field with no path out. Lose focus on the specific, immediate problem in front of you today, and you won't survive long enough for the long-term vision to matter at all. A related, sharper version of this same idea: product strategy is often treated as the most fundamental layer of a startup, but it isn't. **Product distribution** — actually getting the product in front of customers — is more fundamental, because a great product nobody can reach is worthless. And **financing** is more fundamental still, because

a company that runs out of money stops being able to execute on distribution or product at all, regardless of how good either one was. Day-to-day execution, in this framing, is really product distribution work done in service of the financing runway a company has to survive on.

A concrete example: catching a bad tactic before it fully fails

PayPal's own early history gives a specific, checkable illustration of tracking confidence rather than just guessing. An early version of the company's core idea involved beaming cash between Palm Pilots — early handheld devices — so that, for example, a group of friends could split a dinner bill by transferring money device-to-device on the spot. The team was headquartered, as it happened, in an area dense with actual Palm Pilot users, and Hoffman describes testing the idea directly: in practice, restaurant after restaurant, the feature converted "zero to one" — nobody actually used it, even in the single environment where adoption should have been easiest. That was a real, immediate decrease in confidence in that specific tactic, discovered before the product had even properly launched. The team didn't abandon the underlying mission — a system for moving money easily between people — they changed the tactic, moving to Max Levchin's idea of email-based payments instead, which became the actual foundation of PayPal. The vision stayed fixed. The tactic changed the moment the evidence against it became clear, rather than being defended past the point the data justified.

Persistence, tested over years, not weeks

LinkedIn's own early years illustrate the other side of the same discipline — staying the course when confidence in the underlying thesis, while tested, hadn't actually broken. For a long stretch, the company was persistently described in the press as "Friendster for business" — a dismissive label attached to a company perceived as a smaller, derivative version of whatever the current consumer social network happened to be (first Friendster, later MySpace, later Facebook). Hoffman's own account of getting through that period isn't that it didn't hurt or that doubt never crept in — it's that his confidence in the actual underlying thesis (that the world would eventually want, and benefit from, public professional profiles) never broke, even while the *pace* of adoption was slower than he'd hoped. The distinction matters: the thesis surviving years of unflattering comparisons isn't stubbornness for its own sake. It's the same confidence-tracking discipline as the Palm Pilot example, just applied over years instead of weeks, and arriving at the opposite

conclusion because the underlying evidence, unlike the Palm Pilot case, never actually turned against the thesis.

References, not pitches, are how experienced investors spend their attention

A practical detail worth knowing as a founder approaching investors like Hoffman: he describes only taking meetings that arrive through a trusted reference, treating this as the only scalable way to allocate genuinely scarce time and attention. His account of meeting Airbnb's founders is a specific illustration: because he'd already heard about them through people he trusted, he decided within the first two minutes of the actual meeting that he wanted to invest, well before hearing the rest of the pitch — not because the pitch didn't matter, but because the reference had already done most of the work of establishing credibility that the live meeting would otherwise have had to do from scratch. This connects directly to Chapter 14's pitching mechanics: a warm introduction isn't just a nice-to-have. For investors who operate this way, it's close to a precondition for getting real attention at all.

Common Mistakes

- **Mistaking "everyone disagrees with me" for evidence of being right.** Being contrarian is easy. Being contrarian and correct requires a specific, articulable insight the skeptics are missing — without that insight, disagreement from smart people is a warning sign, not a badge of honor.
- **Treating one side of a paradox (flexibility, or persistence; data, or vision) as categorically correct,** rather than using the investment thesis to judge which one the current situation actually calls for.
- **Waiting for a company to "crash into the wall" before recognizing that confidence in the underlying thesis has actually broken.** The Palm Pilot example worked because the team caught the failure early, through direct testing, rather than waiting for a full public launch to prove it.

- **Confusing genuine adaptability with its appearance.** Some people learn to look like they're weighing feedback carefully while quietly not changing anything — this is distinguishable over time, but not always in a single meeting.
- **Building a great product with no real plan for distribution,** or building distribution without securing the financing runway needed to survive long enough to benefit from either.



Tradeoffs and Edge Cases

How do you tell genuine open-mindedness from someone who's just performing it?

Hoffman's own diagnostic, gained from years of watching founders under real pressure: it shows up over time, not in a single conversation. Someone who's actually adapting will visibly change specific decisions in response to specific new evidence. Someone who's performing adaptability will nod, ask thoughtful-sounding questions, and then proceed exactly as they would have anyway. The tell isn't in how someone talks about being open to feedback — it's in whether their actual decisions move afterward.

Hoffman's own name for the trait he's actually screening for, when he checks references, is **"infinite learning curve"** — since every entrepreneurial situation is new and unique to some degree, what matters isn't a founder's current skill set but whether their learning ever stops breaking down, or whether ego gets in the way of adapting once something stops working. It's also, by his own account, domain-specific rather than universal: he's blunt that the same person can be an excellent fit for one kind of company and a poor one for another. He tells a story on himself to make the point: a friend who worked with him at his first startup, Social Net, once told him directly that he'd never hire Hoffman to manage a McDonald's — and Hoffman's own reaction was that he agreed completely, since operational management of that specific kind was never his skill set to begin with.

Is Hoffman's "founders have no balance" claim something every reader needs to accept? It's worth stating plainly that this is a strong, contested personal view, not a universal law — Hoffman is direct that he considers "balanced lifestyle" claims from founders to be close to a signal of insufficient commitment, a position not every reasonable person in this book's audience will share, and one that connects back to

Chapter 1's honest treatment of what a hyper-growth company actually costs. The consistent thread across this book isn't that founders must have zero balance — it's that this book is honest about the real, often severe cost of the specific path it describes, rather than pretending the tradeoffs aren't real.

Do all successful founders fit one archetype? No — Hoffman is explicit that successful founders come from a wide range of backgrounds (Jack Ma was a teacher before Alibaba), and that different domains reward genuinely different specific skills: speed matters more in software, where mistakes are cheap and reversible; precision matters more in hardware, where a mistake can be fatal to the whole product, echoing this book's own hardware interlude. The paradoxes in this chapter are meant to describe a general pattern of judgment, not a specific personality type every founder needs to have.



Applying the Test

Write your own investment thesis for a real or hypothetical idea, as a short list of specific bullet points — including, explicitly, what you believe you know that a smart, informed skeptic would not. Then identify one piece of evidence, available to you within the next month, that would genuinely move your confidence in that thesis up or down. If you can't name one, the thesis probably isn't specific enough yet to actually guide a real decision.



Chapter Summary

- Being contrarian is easy. Being contrarian and correct requires a specific insight that smart, informed critics are genuinely missing — name it explicitly, or be suspicious of your own contrarian confidence.
- An investment thesis — a written hypothesis for why the company will work, including what you know that skeptics don't — is the tool for tracking whether you're still on the right track, by watching whether your confidence in it rises or falls over time.

- Founders live inside a series of real paradoxes (do it yourself vs. delegate; flexible vs. persistent; belief vs. fear; data vs. vision; risk-taking vs. risk-minimizing; short-term vs. long-term) that don't resolve into one fixed answer — the skill is judging which side a given moment calls for.
- PayPal's Palm Pilot pivot shows confidence-tracking working over weeks; LinkedIn's "Friendster for business" years show the same discipline working over years, in the opposite direction — both are examples of the same underlying judgment.
- Product distribution matters more than the product itself, and financing matters more than distribution — a company that runs out of money can't benefit from either.
- Experienced investors frequently only take meetings through a trusted reference — a warm introduction does real work before a pitch even begins.



Exercises

1. **Recall:** Explain the difference between being contrarian and being contrarian and correct, and describe the specific test this chapter offers for telling them apart.
2. **Apply:** Write a short investment thesis for a real or hypothetical idea, including what you believe you know that a smart skeptic would not, and one piece of evidence that would move your confidence in it within the next month.
3. **Apply:** Pick one of this chapter's paradoxes (flexible/persistent, data/vision, or another) and describe a real decision you're facing where you genuinely don't know which side currently applies. What additional evidence would help you decide?
4. **Confront a counterexample:** Hoffman argues founders should have no real balance in their lives, at least for some sustained period. Are there founders or companies you know of that succeeded without that level of singular focus? What would you need to know about their situation to tell whether they're a genuine exception or a case where the sacrifice was simply less visible from outside?



Further Reading

- Reid Hoffman's original CS183B lecture covers additional founder archetypes and domain-specific judgment (software vs. hardware vs. capital-intensive businesses) beyond what this chapter had room to cover in full.
- Chapter 4 covers the "myth of the super founder" — the team-composition half of Hoffman's own lecture, which this chapter deliberately doesn't repeat.
- Chapter 14's discussion of warm introductions is worth rereading alongside this chapter's account of how Hoffman evaluates founders through reference alone.



Chapter 19 — How to Operate

Bill Walsh took over the San Francisco 49ers in 1979. The team had just finished a season at 2 wins and 14 losses — about as bad as a professional football team can be. Within ten years, under Walsh, the 49ers won three Super Bowls. The first concrete thing Walsh did as head coach wasn't a new offensive scheme or a star recruit. He wrote a three-page memo teaching the team's receptionist exactly how to answer the phone.

Keith Rabois, who operated at PayPal, LinkedIn, and Square, tells that story to make a point this whole chapter is built around: building a company is a fundamentally different, and in his view harder, task than building a product, and it's won or lost in details most founders would never think to focus on — because building a company means taking a group of talented, well-intentioned, but genuinely irrational people, and getting all of them to execute at a consistently excellent level, on purpose, every day.



The Core Idea: Your Job Is Editing, Not Writing

Rabois's central device for what a founder or executive's actual job becomes, once there's a team to operate rather than just a product to build, is a metaphor: you are an **editor**, not a writer. Writers — everyone else in the company — do the actual work. An editor's job is different and, done well, largely invisible: simplify what gets produced, ask the questions that matter most, decide where scarce resources and attention go, and keep the whole organization sounding like it came from one voice.

None of this replaces the earlier, more chaotic phase of a company's life. Early on, everything should feel like triage — a constant stream of small fires, each one either a "cold" that resolves itself or something that looks minor and turns out to be serious if left undiagnosed. That messiness is a normal, healthy sign of real innovation happening. The editor's job described in this chapter is what replaces pure triage once a company has grown past the size where a founder can personally touch every decision.



Why It's True: What an Editor Actually Does, and the Tools for Delegating Without Losing Control

Simplify, ask questions, allocate, and keep one voice

The first and most important editing skill is **simplification** — taking a complicated, sprawling set of initiatives and cutting it down to two or three things a team can repeat without having to think hard about them. Complexity is not a legitimate excuse for failing to do this: some of history's largest companies are built around concepts simple enough to describe in a sentence, the same discipline Chapter 2 already established for evaluating ideas in the first place. If a founder can't simplify their own priorities down this far, the team underneath them has no real chance of executing consistently.

The second skill is asking **clarifying questions** — not every possible question, but the small number of highest-value ones that reveal what actually matters in a given decision. A useful, checkable measure of whether this is working: the amount of "red ink" — corrections, simplifications, questions — a leader has to apply should decrease over time, as the team internalizes the underlying standard for itself rather than needing it re-explained on every project.

The third is **allocating resources** — sometimes top-down, moving people onto a new priority because the company's needs have shifted, and sometimes bottom-up, letting people pitch their own initiatives and deciding which to approve. Andy Grove, whose 1982 book *High Output Management* is a major influence on this entire chapter, defined a manager's actual job as maximizing the *output* of their organization — not the *motion* of it, a distinction easy to lose sight of when a team is visibly busy but not necessarily producing the right things.

The fourth is maintaining a **consistent voice** — making sure a company's website, PR, recruiting page, and customer support all feel like they came from the same underlying sensibility, the same way a well-edited publication reads consistently no matter which individual byline appears on a given piece. Even famously disciplined companies struggle with this fully — internal tools and recruiting pages inside Apple, even under Steve Jobs, reportedly never quite matched the polish of the company's public-facing products. The

goal isn't perfection. It's noticing where the voice breaks down and treating that as a real signal worth fixing, not a cosmetic detail.

Delegating without abdicating: task-relevant maturity and a simple decision matrix

The hardest part of scaling beyond a founder personally making every decision is distinguishing real delegation from simply walking away from a decision. Andy Grove's concept of **task-relevant maturity** is the key variable: how experienced is this specific person at this specific task, right now — not in general, and not based on title. A genuinely good manager's style should differ from person to person and from task to task, rather than being one uniform approach applied to everyone. A useful signal that this is being done correctly: if a founder runs reference checks on themselves and some people describe them as a micromanager while others describe them as someone who hands over enormous freedom, that's not a contradiction to resolve — it's evidence the calibration is actually working, applied differently where different people warranted it.

Figure 19.1 — A simple test for whether to decide yourself or delegate

	Low confidence in your own judgment	High confidence in your own judgment
Low consequence if wrong	Delegate completely. Let them make the call, even if it's wrong.	Delegate, but explain your reasoning — don't just assert the answer.
High consequence if wrong	Delegate, but explain your reasoning — don't just assert.	Decide yourself. The cost of being wrong here is too large to delegate.

A concrete illustration of the low-confidence, low-consequence quadrant: an early hire at Square wanted to run a marketing experiment (handing out working payment readers at a food truck) that Rabois personally doubted would move any meaningful metric. It didn't — but letting the experiment run cost little, and it taught that employee, directly and without a lecture, how to evaluate his own future ideas — a cheap price for a lesson that would otherwise have taken much longer to land. The opposite corner — high confidence, high consequence — still calls for real explanation rather than simply pulling rank; a specific signal worth tracking personally is how often a colleague ultimately defers with some version of "fine, you're the boss" rather than being genuinely persuaded. That phrase, heard too often, is a sign of spending down trust rather than building it.

Give people exactly one thing, more often than feels natural

Peter Thiel's practice at PayPal — restricting each person to exactly one clear priority, and refusing to discuss anything else with them until it was resolved — comes from a specific insight about human behavior: left with a full list of responsibilities, most people gravitate toward the problems they already know how to solve (comfortable, "B+" problems) rather than the harder, highest-impact problem actually in front of them ("A+" problems), because the hard problem doesn't have an obvious first step and is easy to keep deferring. Rabois's own recommendation is to relax this to two or three priorities if one feels too rigid, but to keep tracking, explicitly, what would happen if each person were forced down to exactly one — because that thought experiment reveals which of the "priorities" on anyone's list is actually load-bearing.

Barrels and ammunition

A specific, memorable test for who on a team can genuinely drive an idea from concept to shipped reality — what Rabois calls a **barrel**, as opposed to **ammunition** (people who do excellent work but need direction rather than generating and driving it themselves) — is to hand someone a small, almost trivially unimportant task and watch what happens. Rabois's own example: he wanted cold smoothies delivered to engineers working late at Square, at a specific time, consistently. Multiple capable people failed to make this happen reliably. A brand-new intern, on his second day, simply said he'd handle it — and did, precisely, without further instruction. The task itself didn't matter. What mattered was learning, cheaply, that this person could be handed a larger and larger scope over time, expanding it until it eventually broke, which is the actual process for finding out how much responsibility someone can carry.

The practical ratio worth tracking as a company scales: roughly one barrel for every ten to twenty pieces of ammunition. Fall too far below that ratio, and a team accumulates people who all need direction from the same small number of barrels, which caps how much the company can actually get done regardless of total headcount — a specific, measurable check against the natural tendency for managers to treat team size itself as a proxy for importance, rather than measuring actual output relative to team size.

Transparency, metrics, and the discipline of pairing indicators

Sharing real information broadly — company metrics, board meeting takeaways, notes from any meeting involving more than two people — helps a growing team stay aligned without every person needing to be in every room. Chapter 6 already covered how far this can be pushed and what it costs to sustain (Stripe's own transparency bet, and the real tooling investment it required once the company grew past its first few dozen people). Rabois offers a compensation-specific version of the same idea, drawn from how Steve Jobs ran pay at NeXT: rather than negotiating salary individually with every hire, NeXT paid in simple, public bands by experience level — a flat \$85,000 for the "low" band, a flat \$130,000 for the "high" band, adjusted for inflation from the 1980s figures Jobs actually used — so that everyone at a given level knew exactly what everyone else at that level made, and disputes over individual negotiating leverage simply stopped being possible. This chapter's addition is a specific discipline for the metrics themselves: **pair every metric with an opposing one**, so that optimizing the first one in isolation can't quietly damage something else. A fraud-prevention team measured only on lowering the fraud rate will, predictably, start treating every customer as a suspect — which is why it needs to be measured on its false-positive rate at the same time. A recruiting team measured only on hiring volume will lower the bar to hit the number — which is why interview and hire quality need to be tracked alongside volume, not after the fact.

Physical space carries the same signal, in Rabois's account. Every conference room at Square was built with glass walls deliberately: closed-door rooms invite people to speculate darkly about what's being decided behind them, while a visibly ordinary meeting going on behind glass rarely worries anyone. Rabois also argues, more provocatively, that a startup should never share office space with other companies, however tempting the cost savings look early on — his reasoning leans on something he attributes to Peter Thiel, that "every good startup is a cult," in the specific sense that a team needs to genuinely believe it has a uniquely better way of doing things than anyone else in the world, and that belief is much harder to sustain physically surrounded by other companies doing things their own, different way.

A related discipline: pay close attention to **anomalies** in your own data, rather than only the metrics you already expected to matter. PayPal's own core market — power sellers on eBay — wasn't originally on the company's list of target customers at all. It was discovered because a small number of eBay sellers had started manually typing "please pay with PayPal" into their own listings, an anomaly the team almost dismissed before recognizing it as the actual market they should build for. LinkedIn found something similar in its own

click data: a startlingly large share of all clicks on the site — a quarter to a third — were people clicking through to view their *own* profile, which made no obvious sense for a supposedly utilitarian networking tool, until the team realized users were checking their own profile the way people check a mirror, and that this vanity-driven behavior correlated with adding more content and endorsements. Neither insight would have surfaced from a dashboard built only to track the metrics the team already assumed mattered.

| The unglamorous details that actually decide the outcome

Bill Walsh's own account of turning around the 49ers makes the case that operational excellence is mostly a matter of a huge number of small details executed consistently, not one grand strategic insight — his first move, teaching the receptionist how to answer the phone properly, is the clearest possible illustration of this. Steve Jobs's insistence that the circuit board inside the original Macintosh be laid out beautifully, despite the fact that no ordinary customer would ever open the case to see it, is a similar signal: a company's tolerance for sloppiness in places nobody's watching tends to show up eventually in places everyone is watching. Food, office space, and the tools a company gives its own employees fall into this same category — not because any one of them is individually important, but because a bad lunch or an ugly office becomes something people quietly gossip about instead of doing focused, creative work, which is a real and measurable cost even though it never shows up as a line item.

Rabois offers a related story about how legible these small signals are from the outside, too. Roelof Botha of Sequoia Capital led YouTube's Series A when it was still far from an obviously successful bet, and at a board meeting later told Rabois why his confidence in the company kept climbing: every time he visited another portfolio company's office, he noticed half the room quietly watching YouTube during lunch. That one small, unglamorous observation was, in Botha's own account, a genuinely good signal — and Rabois's broader point is that these details are legible enough that an experienced investor can often tell whether a company's culture is working within moments of walking into its office.

| Time allocation: the calendar audit, and five to seven direct reports

Rabois's diagnostic for founders and new managers who feel like they're struggling in the role is deceptively simple: do a **calendar audit**. Ask them to write down, from memory,

what they believe their real priorities are and roughly what share of their time each one should take — and then pull up their actual calendar and check whether the two match. Rabois's experience running this exercise repeatedly is that they almost never do: recruiting, in particular, is the priority founders most often claim matters most while consistently being the smallest block of actual time on their calendar. There's no software that does this well; it has to be done by hand, hour by hour, which is exactly why most people never bother checking.

The same audit connects to a specific structural rule for anyone managing people directly: hold a one-on-one with each direct report roughly every one to two weeks, an idea that traces back to Andy Grove's 1982 *High Output Management* and was considered radical at the time. The reason a manager shouldn't have more than **five to seven direct reports** follows directly from that cadence — it's simply the number of one-on-ones that can actually fit into a single manager's calendar in a given week alongside everything else the job requires. The agenda for each one-on-one should be set by the employee, not the manager, since the meeting exists mainly for their benefit; sending a few bullet points in advance gives the manager time to think rather than improvising an answer on the spot.

Promote your best individual performer, not your best manager — and get a mentor, not a boss

Rabois's account of PayPal's own promotion philosophy, credited to Peter Thiel, cuts against a common instinct: rather than looking for people who are naturally good at managing others, PayPal deliberately promoted whoever was the single best individual performer in a given discipline — the best engineer became VP of Engineering, the best designer became VP of Design — and let them learn the management side of the job afterward. The logic is about morale as much as skill: nobody resents reporting to someone who is demonstrably better than they are at the actual craft, in a way people often do resent reporting to someone who is only good at managing people and not at the underlying work itself.

Making that transition from individual contributor to manager is still genuinely hard, and Rabois is direct that some of it can only be learned by doing, the way nobody learns to play an instrument purely from reading about it. His specific advice for surviving the transition: find a **mentor**, not a boss, to work through it with — someone who has

managed before and can focus purely on your own development, since a boss necessarily carries other, more complicated objectives (like how much the team is shipping) that a dedicated mentor doesn't have to weigh at the same time.

Common Mistakes

- **Trying to be involved in every decision as a company scales past the size where that's actually possible.** The editor metaphor exists specifically because a founder's job has to shift from doing the work to shaping how the work gets done.
- **Applying one uniform management style to everyone**, rather than calibrating to each person's task-relevant maturity for the specific work in front of them.
- **Measuring a team on a single metric without a paired, opposing one**, which predictably produces exactly the kind of narrow optimization (a fraud team that alienates every legitimate customer, a recruiting team that lowers its bar) the single metric was never meant to reward.
- **Ignoring anomalies in your own data because they don't fit the metrics you already expected to track.** PayPal's entire market, and a real piece of LinkedIn's user psychology, were both found this way.
- **Treating small operational details (office space, food, internal tools) as beneath a founder's attention.** These are exactly the details Bill Walsh's and Steve Jobs's own examples suggest actually determine whether a team executes at a high standard everywhere else.

Tradeoffs and Edge Cases

Does the "one thing per person" rule feel unreasonably rigid? It's meant to, and that's the point — Rabois explicitly allows relaxing it to two or three priorities in practice. What shouldn't be relaxed is the underlying discipline of asking, for every person on a team, what would happen if they had only one thing to focus on — because that question reveals

which of their stated priorities is actually driving results and which is comfortable busywork.

Isn't measuring "how much red ink you use" a strange way to judge your own management?

It's counterintuitive precisely because it inverts the instinct to measure your own value by how much you're personally correcting or directing. The logic is that a team that has genuinely internalized your standards needs less correction over time — so declining red ink is a sign of successful teaching, not declining involvement or relevance.

How does this chapter's emphasis on systems and delegation relate to Chapter 9's "doing things that don't scale"?

They describe sequential stages of the same company, not competing philosophies. Chapter 9 covers the earliest period, before there's enough signal to know what a good system should even contain. This chapter assumes a company has already learned those lessons and is building the durable structure — dashboards, task-relevant delegation, paired metrics — that a founder can't personally staff forever. Applying this chapter's systems thinking too early, before the hands-on learning Chapter 9 describes has actually happened, tends to produce systems built on guesses rather than real, hard-won evidence.



Applying the Test

Pick a task you currently do yourself that a teammate could plausibly take over. Apply Figure 19.1: how confident are you in your own judgment about it, and how bad would it actually be if it were done wrong? Use the answer to decide, honestly, whether you should delegate it completely, delegate it with an explanation of your reasoning, or keep doing it yourself for now — and if you keep doing it yourself, name the specific thing that would need to be true before you'd feel comfortable handing it off.



Chapter Summary

- Building a company is a different, often harder task than building a product — your job shifts from writer to editor: simplify, ask the highest-value questions, allocate resources, and keep one consistent voice across the whole company.
- Task-relevant maturity, not a uniform management style, should determine how much freedom versus guidance each person gets on each specific task.
- Use a simple test — your own confidence, and the consequence of being wrong — to decide whether to delegate completely, delegate with explanation, or decide yourself.
- Give people as close to exactly one priority as practical; most people default to comfortable, well-understood problems unless forced to focus on the hardest one.
- "Barrels" — people who can drive an idea from concept to shipped reality — are rare; test for them with small, low-stakes tasks before handing over anything larger, and track your barrel-to-ammunition ratio explicitly.
- Pair every metric with an opposing one, so optimizing one doesn't quietly damage something else, and pay close attention to anomalies in your data rather than only the metrics you expected to matter.
- Small operational details — how the phone gets answered, what people eat, the quality of an internal tool — are not beneath a founder's attention; they set the standard for everything else the company does.
- Run a calendar audit against your stated priorities before trusting your own sense of where your time goes, and cap direct reports at five to seven so real weekly or biweekly one-on-ones actually fit.
- Promote your best individual performer into management rather than the person who's merely best at managing, and find a mentor rather than relying only on your boss to develop as a manager yourself.



Exercises

1. **Recall:** Explain the difference between a "barrel" and "ammunition" in Rabois's framework, and describe the kind of small test that would reveal which one a new hire actually is.

2. **Apply:** Using Figure 19.1, categorize three decisions you currently make yourself by your confidence level and the consequence of being wrong. Which ones should actually be delegated?
3. **Apply:** Identify one metric your team (or a hypothetical team) currently tracks in isolation, and design a paired, opposing metric that would catch the kind of narrow optimization this chapter warns about.
4. **Confront a counterexample:** This chapter argues small operational details (phones, food, internal tools) meaningfully affect execution quality elsewhere. Can you think of a company that succeeded despite visibly neglecting details like these? What would explain the exception?



Further Reading

- Andy Grove's *High Output Management* is the direct source of the task-relevant-maturity concept and the "measure output, not motion" framing this chapter builds on.
- Bill Walsh's *The Score Takes Care of Itself* covers the 49ers turnaround and the broader argument that details, consistently executed, matter more than any single strategic insight.
- Chapter 6's discussion of Stripe's transparency practices is worth rereading alongside this chapter's paired-indicators discipline — they're complementary tools for keeping a growing team aligned.



Chapter 20 — How to Manage

A genuinely good employee walks into a CEO's office and asks for a raise. Every instinct in the room points toward saying yes: this person is talented, you want to keep them, saying no feels petty, and saying yes costs you nothing you weren't already prepared to eventually pay. Ben Horowitz, who has run companies and sat on the other side of this exact conversation many times, argues that the instinct to just say yes is almost always wrong — not because the raise isn't deserved, but because of a question nobody in the room is asking: what happens to the equally good employee, sitting two desks away, doing equally strong work, who never thought to ask?

That second employee, once they eventually notice the difference, draws a rational and corrosive conclusion: performance isn't actually what determines pay here. Asking is. And once that conclusion spreads, every good employee in the company now has, in Horowitz's phrase, "a fiduciary responsibility to their family" to start asking constantly, whether or not they have a real case — because the alternative is quietly falling behind people no more deserving than they are.

This chapter is about that specific management skill: seeing a decision through the eyes of everyone it touches, not just the person sitting across from you, and not just your own instinct in the moment.



The Core Idea: You Are Never Deciding for Just One Person

Horowitz's central claim, stated plainly: the single management mistake he sees most consistently, from first-time founders all the way up to experienced CEOs, is failing to see a decision through the eyes of the whole company. Not the person asking for something. Not your own gut reaction, formed under whatever pressure you're feeling in the moment. Every employee who will eventually hear about the decision, directly or through the grapevine, and draw their own conclusion about what it means for them.

This is hard for a specific, structural reason: you're making the decision while looking at exactly one person, under real time pressure, and the people who will actually be affected by the precedent aren't in the room to advocate for themselves. Learning to imagine them anyway — consistently, especially under pressure — is what this chapter is actually teaching.



Why It's True: Three Decisions, and One Historical Case Where Getting This Right Changed a Country

The raise nobody sees is the one that actually matters

Return to the raise request that opened this chapter. The fix isn't refusing every raise — it's removing the *timing* of the ask from the equation entirely. A formal, periodic review process — evaluated on a fixed schedule (every quarter, or every six months), gathering input systematically, applied to everyone at once rather than whenever someone happens to lobby for it — protects exactly the thing an ad hoc "whoever asks, gets" system destroys. It also gives every employee something valuable: the psychological security of knowing they don't have to constantly campaign for what they've earned, because there's a real process that will surface it on a predictable schedule regardless of who's most comfortable asking.

A demotion changes what everyone believes it takes to keep their equity

Horowitz's second case: an executive is working extremely hard, well-liked, and genuinely trying — and still failing at the job, because the role has outgrown their actual skills. Firing feels harsh, given the visible effort. Demoting — hiring someone above them, letting them stay in a smaller role — feels like the kinder, more humane choice, and it's the one most first-time founders default to.

Horowitz's question, the one that reframes the whole decision: what percentage of the company does this person actually own? In the real conversation this chapter draws from, the executive in question had been Head of Sales, and the answer was 1.5% — an amount that dwarfs what a strong individual engineer at the same company might hold (a tenth or

two-tenths of a percent). Left untouched, a demotion without any change to that equity stake sends an unavoidable message to every other employee watching: real effort, even without real results, is enough to keep a disproportionate ownership stake. That's a company-wide statement about what your equity actually requires — genuine results, or merely genuine effort — made through a single, seemingly private personnel decision. There's a cost on the other side too, worth being honest about: the demoted executive, moved from Head of Sales down to a Regional Manager role and now reporting to someone hired above them, has to work alongside people who remember them as the boss — a dynamic Horowitz describes bluntly through the demoted executive's own likely reaction: "I knew you when you were Head of Sales, now you're the Regional Manager and you're telling me what to do?" — which is its own difficult, often unworkable dynamic. Neither option here is comfortable. The point isn't that demotion is always wrong — it's that the equity question has to be part of the decision explicitly, not skipped because the personnel conversation already feels hard enough.

Should a departing employee get 90 days or 10 years to exercise their stock options?

The most detailed case in this chapter concerns a real policy debate: most startup stock option agreements require an employee, once they leave, to exercise their vested options — paying the strike price and often a real tax bill — within 90 days, or forfeit them entirely. At a company whose valuation has grown substantially, this can require far more cash than a typical employee has sitting around, which means someone can leave a company having earned real equity on paper and walk away with nothing, simply because they couldn't afford to buy it. Sam Altman, teaching elsewhere in this same course, proposed extending that window to a full ten years instead, an idea borrowed from Quora's Adam D'Angelo.

Horowitz's analysis is worth following in full, because it's a clean example of this chapter's whole method. First, the historical context: the 90-day standard exists partly because of an old accounting rule (APB Opinion 25) that made longer exercise windows create an unpredictable, stock-price-linked expense a company would have had to report — a rule that no longer applies today, which means the original reason many companies adopted the 90-day default has quietly become outdated even though the default itself persisted. Second, the perspectives actually in play: the departing employee wants fairness and enough time to come up with the cash. The employees who *stay* care about two different

things — whether unclaimed, forfeited equity returns to the option pool (benefiting everyone who stayed, since typical annual dilution runs around 10% and returned shares offset some of that), and whether watching a departing colleague get what feels like a fair deal changes their own read on whether leaving might have been the smarter move. A 10-year window is also, quietly, a real financial asset in itself — an option on a volatile stock is worth more the longer it has to run — which means an employee who stays and simply holds ordinary vested stock isn't getting that same optionality value at all.

Horowitz doesn't declare a single correct answer, and that refusal is itself the lesson. He offers two honest, opposite cultural stances a company could actually take, both defensible: **"we are straightforwardly fair — you keep the full ten years no matter what happens"**, or **"your equity is only meaningful if you vest, stay until an actual exit, and the company ends up being worth something — we're telling you that up front so you know what you're actually signing up for."** Neither is a trick. Each is a genuine, different bet about what a company wants its equity to communicate. Altman's own live response, in the same conversation, is worth noting for a different reason: he revised his position on the spot, acknowledging the proposal needed more built-in incentive to stay, since a departing employee can still end up "screwed" depending on whether they personally have the cash to exercise. That kind of visible, immediate willingness to update a public position, in front of an audience, having just heard a good counter- argument, is itself a small demonstration of the skill this whole chapter is about.

Toussaint Louverture: seeing a decision through the eyes of people you have every reason to hate

Horowitz's most extreme illustration of this chapter's core skill comes from far outside business. Toussaint Louverture was born into slavery in the French colony of Saint-Domingue — a system whose brutality, by the numeric record Horowitz cites, was markedly worse even than slavery in the United States. He went on to lead the only fully successful slave revolution in recorded history, ending slavery in the colony, defeating the forces Napoleon sent to retake it, and building what briefly became one of the most economically productive colonies in the world — one point in its history exporting more value than the United States did at the time.

What makes Louverture's story a management case study, in Horowitz's telling, is how he treated the two groups he had the most reason to treat as pure enemies. When his army

defeated opposing soldiers — Spanish, British, French — his own troops expected the normal spoils of war. Louverture refused to allow pillaging, and beyond that, actively recruited the most capable officers from the armies he'd just defeated into his own command, reasoning that the culture and expertise he wanted to build required the best available talent, regardless of which side that talent had just been fighting for. When he ultimately controlled the former plantations, his own soldiers — former slaves — understandably wanted the land and the owners' wealth seized outright. Louverture instead ended slavery, let plantation owners keep their land, required them to pay real wages to the workers who had once been enslaved on it, and lowered the owners' taxes to help the economy actually survive the transition. He didn't do this because he'd forgotten what had been done to him. He did it because he'd thought through the decision from the economic perspective of people he had every reason to despise, and concluded that a country capable of enduring needed their productive capacity intact, not just their punishment.

Sidebar: The Full Weight of What Louverture Overcame

The brutality Louverture survived deserves to be stated honestly, not softened. Horowitz's own account, drawn from historical record, describes routine torture and mutilation used to control enslaved people in Saint-Domingue — practices markedly more severe, by the numbers he cites, than plantation slavery in the American South. Of roughly two million people brought to the Caribbean's sugar colonies over the full span of the slave trade, only around seven hundred thousand remained alive by slavery's end there — a survival rate dramatically worse than the comparable figures in the United States over the same period. This is the system Louverture was born into, escaped, and eventually defeated militarily and politically. The reason his later decisions — refusing his own soldiers' demand for vengeance, keeping plantation owners' land intact — carry real moral and strategic weight is precisely because they came from someone who had absolutely no natural inclination toward mercy for the people he chose to treat fairly anyway.

The management lesson Horowitz draws from this isn't really about slavery, war, or national economics — it's that the discipline of imagining a decision through the eyes of people you have the least natural sympathy for, at the moment you have the most power

over them, is both the hardest and the most consequential version of the skill this entire chapter describes.

Common Mistakes

- **Granting a raise only because someone asked, without a formal process evaluating everyone.** This teaches an entire company that performance isn't what's actually rewarded — asking is.
- **Demoting a struggling executive without addressing what their equity stake now signals to the rest of the company.** The personnel decision and the equity question are the same decision, not two separate ones.
- **Assuming an old policy (like a 90-day option exercise window) still makes sense without checking whether the original reason for it — in this case, an accounting rule — still applies.**
- **Making a real decision quickly under pressure, without pausing to think it through from every stakeholder's perspective first.** Horowitz's own advice: it's better to say "let me think this through properly and get back to you" than to answer instantly and get it wrong — burying a real problem instead of addressing it directly tends to make it worse the longer it's left alone, not easier.
- **Assuming there's always one universally correct answer to a genuine values tradeoff** (like the stock-option window debate), rather than choosing a defensible position deliberately and communicating it honestly in advance.

Tradeoffs and Edge Cases

Is there a single right answer to the 90-day-versus-10-year option question? No, and this chapter deliberately doesn't pretend there is one. Both of Horowitz's proposed cultural stances — full fairness regardless of outcome, or an honest, upfront statement that equity requires staying until an exit — are legitimate choices a company can make.

What matters is making the choice deliberately, understanding the tradeoffs on both sides, and communicating whichever one you pick clearly, rather than defaulting to whatever the standard template happens to say without examining it.

Does "see it through everyone's eyes" mean every decision requires infinite

deliberation? No — Horowitz's own practical advice is a pause, not paralysis: when a decision is clearly consequential, say so directly ("I'm taking this seriously and need to think it through") rather than either rushing to a bad answer or freezing entirely. Most day-to-day decisions don't require this level of consideration; the skill is recognizing which ones genuinely do.

How does this chapter's approach to firing relate to Chapter 5's "fire fast" advice?

They're complementary, not in tension. Chapter 5 argues for acting quickly once it's clear someone isn't working out. This chapter is about *how* to communicate that decision once you've made it — honestly, without either cruelty or excessive apology, and in a way that preserves the departing person's dignity in front of the rest of the company, since how a firing is handled publicly shapes what everyone who stays believes about how they'd be treated in the same position.



Applying the Test

Take a real or hypothetical management decision you're currently facing — a raise, a promotion, a difficult personnel call, anything with real stakes. Before deciding, write down every group of people who will eventually learn about this decision, even indirectly: the person directly involved, their peers, anyone with a similar role, and anyone who might reasonably wonder what the decision implies about their own situation. For each group, write one sentence describing what they would conclude about "what it takes to succeed here" based on this decision alone. If any of those conclusions would surprise or alarm you, that's the part of the decision you hadn't actually finished thinking through.



Chapter Summary

- The core management skill this chapter teaches: see every real decision through the eyes of the whole company, not just your own instinct or the person in front of you.
- A raise given only because someone asked teaches everyone else that performance isn't what's actually rewarded — use a formal, scheduled process instead.
- A demotion instead of a firing isn't just about the individual — it's a statement, visible to the whole company, about whether effort alone is enough to keep a disproportionate equity stake.
- The 90-day versus 10-year stock option debate has no single correct answer — pick a defensible cultural stance deliberately, and communicate it honestly in advance.
- Toussaint Louverture's decision to treat defeated enemies and former slave owners fairly, despite having every reason not to, is the most extreme version of this chapter's core skill: imagining a decision through the eyes of people you have the least natural sympathy for.
- Pause before deciding on genuinely consequential questions — it's better to say "let me think this through" than to answer quickly and get it wrong, since unresolved people problems tend to get worse, not better, the longer they're left buried.



Exercises

1. **Recall:** Explain why giving a raise only to employees who ask for one is described in this chapter as corrosive to a company's culture, even though the raise itself may be fully deserved.
2. **Apply:** Using the "Applying the Test" exercise above, work through a real or hypothetical management decision you're facing, listing every group who will eventually learn about it and what each group would conclude.
3. **Apply:** Draft your own company's policy on stock option exercise windows, explicitly choosing one of this chapter's two cultural stances (or a third of your own) and writing the one-paragraph explanation you'd give a new hire about it.

4. **Confront a counterexample:** This chapter argues Louverture's decision to treat defeated enemies and slave owners fairly was strategically and morally sound. Are there situations where extending this kind of consideration to people who have wronged you would be the wrong call rather than the right one? What would distinguish those cases from Louverture's?



Further Reading

- Ben Horowitz's original CS183B lecture goes into further detail on the historical Toussaint Louverture case than this chapter's condensed version, drawing on C.L.R. James's historical account of the Haitian Revolution.
- Sam Altman's original blog post on stock option exercise windows, referenced directly in this chapter, is worth reading alongside Horowitz's critique of it.
- Chapter 5 covers the "fire fast" decision this chapter's firing-communication advice builds on.



Chapter 21 — Scaling the Company

Sam Altman describes a test he runs whenever he visits a company that's starting to struggle with growth. He asks the founder: if I walked around and asked ten random employees what the company's top three priorities are right now, would they all say the same thing? The founder, every single time, says yes — of course they would, we announced it at the all-hands three months ago. Altman then actually does it. Every single time, no two employees list the same three priorities, let alone in the same order.

The founder isn't lying, and the employees aren't being difficult. The gap is simply what happens by default once a company grows past the size where everyone naturally absorbs what matters just by being in the room. This chapter is about that transition — the specific, structural changes a company needs once it outgrows the informal, everyone-knows-everything stage that worked fine at ten or twenty people.



The Core Idea: The Founder's Job Changes, and Flat Structure Breaks All at Once

Before a company has found real product-market fit, a founder's job is building a great product. Past roughly twenty-five employees, the job changes, and it stays changed for the rest of the company's life: the founder's primary job becomes building a great *company* — the systems, structure, and culture that let the product keep getting built well without the founder personally touching every decision.

A specific trap makes this transition harder than it needs to be: a completely flat structure, with everyone reporting to the founder, works remarkably well up to a point — and then it doesn't, suddenly, rather than gradually. What worked fine at twenty employees becomes unworkable at thirty, seemingly overnight, because the failure is one of coordination overhead compounding past a threshold, not a gradual decline anyone notices coming. The fix isn't complicated: every employee needs exactly one manager, and every manager needs to know who their direct reports are. Companies that try to look

innovative by avoiding any real structure, or that overcomplicate it with matrixed, multiple-reporting-line arrangements to make people feel important, both make the same underlying mistake in opposite directions.



Why It's True: Four Ways Founders Fail as Managers, and What Actually Needs Building

Four common failure modes, in order of how often Altman has seen them

Being afraid to hire senior people. Early on, this instinct is correct — aptitude and belief in the mission matter more than experience for a company's first hires. Once a company is scaling, the same instinct becomes a liability: an experienced executive who's run a team like this one before can take over an entire function and make it work, and nearly every founder who finally hires one says the same thing afterward — they wish they'd done it sooner.

"Hero mode." A manager, often running support or a similarly ticket-driven function, responds to growing demand by working harder personally rather than hiring more people — leading by extreme, unsustainable example instead of accepting the short-term pain of building out a team properly. The fix requires a real, uncomfortable tradeoff: falling behind for a few weeks while hiring, rather than compensating personally forever.

Bad delegation. The common, wrong version looks like this: "go research this and come back with the tradeoffs, and I'll decide." The better version: "you're smart, that's why I hired you — here's context, here's what I think, but you decide, and tell me what you chose." The first approach doesn't scale and doesn't build trust. The second does, even though it feels riskier in the moment.

No personal organization system. Building product doesn't require much personal organization. Managing a growing team does — tracking what each person is working on, what needs following up, and what you told them last time you spoke — and founders who skip building this system for themselves find it catching up with them exactly when the company can least afford the confusion.

Write down the how and the why, or someone else will write it for you

Chapter 6 already covered writing down a company's core *values*. This chapter adds a related, more operational discipline: writing down *how* the company actually does things — how code gets shipped, how customer support gets handled, how decisions get made — the practices that used to spread naturally over shared lunches when the company was small enough for everyone to just absorb them by proximity. Skip this, and the company doesn't end up with no culture — it ends up with an *accidental* one, transmitted randomly by whichever manager or peer happens to onboard each new hire. Write it down, and a founder gets to be the one who actually sets the standard, rather than discovering years later that it drifted somewhere they never chose.

HR is not the TV-sitcom version, and it's worth taking seriously

Most founders resist formal HR because of a specific, inaccurate mental picture — office politics, box-checking, bureaucracy for its own sake. Done properly, HR is closer to the opposite: a clear structure, a real path for career growth, and frequent, honest performance feedback that actually speeds a company up rather than slowing it down. Concretely, this means: **compensation bands** instead of ad hoc, individually negotiated pay that inevitably becomes visible and inevitably feels unfair once it does; **continuing equity grants**, not just generous grants to the first ten hires (Chapter 5) — a healthy, growing company should expect to distribute something like 3 to 5% of its equity to employees every year, indefinitely, and needs a real system (not a shared spreadsheet) to track it once mistakes in that tracking can cost real money. Several vesting structures beyond the standard four-year schedule (Chapter 15) are worth considering as a company matures: longer, six-year grants for people expected to stay a long time; "pyramid" vesting that back-weights a grant so more of it vests in year four than year one, as a retention lever; and continuous forward vesting, where an employee's unvested grant is automatically topped back up every year to the same number of shares, so no one is ever staring at a nearly-fully-vested grant and quietly weighing whether to leave. Whichever structure a company picks, it needs a real option-management system, not a shared spreadsheet, since Sam Altman has personally seen tracking mistakes at this stage cost employees or companies tens of millions of dollars; **a formal hiring process**, including a full-time recruiter once the company can actually absorb hires at volume — brought on too early, before there's real product-market fit, this tends to produce hiring for its own sake rather than hiring for a real need; **announcing every offer internally before it goes**

out, since someone already at the company frequently knows something useful about a candidate that the hiring manager doesn't; and active attention to **diversity**, especially on early engineering teams, where a homogeneous first fifteen or twenty hires tends to calcify into a monoculture that's difficult and slow to correct later. One dimension of that diversity worth naming directly: immigrant founders are a large and growing share of the startup ecosystem, not a marginal case — in one recent Y Combinator batch, Sam Altman noted that 41% of founders funded were born outside the United States, across roughly thirty different countries. New compliance requirements — sexual harassment and diversity training among them — also kick in at the 50-employee mark in many jurisdictions, worth knowing about before it becomes an unpleasant surprise.

One more HR-adjacent responsibility easy to neglect: actively watching for **burnout** across the team, not just in yourself. What worked as an all-consuming sprint for the company's first year or two becomes unsustainable as a permanent operating mode, and a team that burns out all at once, rather than staggered, can be close to company-ending — worth deliberately building in real rest, not just tolerating it when someone happens to take it.

Productivity degrades on its own unless you actively fight it

Left alone, a growing company's productivity doesn't just fail to increase with headcount — it can decline, because the number of possible communication pairs between employees grows roughly with the *square* of the total headcount, not linearly with it. The single most effective countermeasure is **alignment**: making sure everyone in the company can name the same handful of current priorities, which is exactly the test that opened this chapter. Most founders wildly overestimate how well their own team already does this, because a goal stated once, months ago, doesn't function as a live priority — it has to be repeated far more often than feels necessary, reinforced in every all-hands, every one-on-one, every planning document, to actually stay top of mind across a growing team.

Process itself should serve the product, not become an end unto itself — a useful, lightweight forcing function some companies use is a simple commitment to ship something, however small, every single day, which keeps the whole organization oriented toward delivery rather than internal process for its own sake. Underneath that, a basic operating rhythm matters more than most founders expect: a weekly meeting with direct

reports, a monthly all-hands reviewing results and the roadmap, quarterly planning tied to the company's yearly goals, and — genuinely underused, per Altman's own account — periodic offsites that pull the company's best people out of daily operations specifically to think about what's not getting done that should be.

The unglamorous mechanics that become urgent around month 18

A handful of operational tasks tend to get ignored during a company's earliest phase, reasonably, since they're not writing code or talking to users — and then become urgent once things are actually working, typically somewhere around month 18. Proper accounting and annual audits should start around this point, alongside a systematic effort to collect and organize every legal document the company has ever signed (Chapter 15 already covered why this matters, and why it's much easier to do this before a high-stakes fundraising or acquisition than during one). Provisional patents are worth filing within roughly eleven months of first publicly discussing a product — patent law typically gives a twelve-month window after public disclosure, and a provisional filing is inexpensive and simply holds a company's place in line while it decides whether a full patent is worth pursuing. Trademarks, in the U.S. and in major international markets, are worth locking down around the same time, along with any domains a company hasn't already claimed. A dedicated FP&A (financial planning and analysis) hire, building a real financial model of the business, tends to get hired far later than it should — some of the most useful models built for fast-growing companies run to well over a thousand lines of detail. Altman points to Roelof Botha, PayPal's own CFO, as the standard-setting example: the top sheet alone of the financial model Botha built for the company ran to roughly 1,500 lines, a level of granular detail that let PayPal understand and tune levers in the business that most companies never even discover they have. That level of insight is available much earlier than most founders assume to look for it, and most companies don't hire someone capable of building it until they're hundreds of employees in — worth doing earlier than that. The same logic applies to hiring a dedicated, full-time fundraiser once a company is past its Series B — a specialist focused entirely on the next round tends to produce a meaningfully better outcome, at lower cost, than treating each future raise as an ad hoc process layered on top of everything else a founder is already doing.

FF stock — a special class of founder stock, structured so founders can sell some of it in a later financing round without distorting the valuation of the company's regular common stock — is worth knowing about, mostly so a founder recognizes it as a trap early on

rather than a perk. Setting it up right at company formation, when the company is still worth very little, became such a recognizable pattern (popularized heavily enough by Founders Fund that the "FF" name stuck) that investors learned to read a founder pushing for it at the seed stage as a bad signal — a sign of focus on personal payout before the company had proven anything, which correlated with those companies failing more often. Most founders don't actually want to sell any stock until the company is worth close to a billion dollars regardless; the safe time to set FF stock up is later, once the business is already working, typically around the Series B, so it can be sold two to four years down the road without ever having sent the wrong signal early on.

Tax structuring is worth a founder's attention around this same stage too, even though it's a genuinely unglamorous topic. One concrete example Altman cites: assigning a company's IP to a corporate entity in Ireland, which then licenses it back to the U.S. parent, can eliminate corporate tax on the licensed IP — a structure that has to be set up relatively early, since it stops being available once a company is large and public, and one that can become a real competitive disadvantage against competitors who did set it up early, once everyone involved is a large public company and it's too late to change.

Founder psychology gets worse, not better, and that's the actual reason companies die

A specific, honest warning worth stating plainly: the emotional experience of running a company does not get easier as it grows and succeeds. The highs get better. The lows get worse, and the swings between them tend to widen rather than narrow. Alongside that, success changes how the outside world treats a founder in a way nobody fully prepares for: an early-stage founder is generally an underdog people root for; a visibly successful one increasingly becomes a target for online criticism and, eventually, skeptical press coverage — a transition worth making peace with in advance rather than being blindsided by later.

Two specific, avoidable ways founders lose their footing once things start working: first, losing focus — accepting invitations to advise other companies, speak at conferences, and generally enjoy the trappings of success that follow a company doing well, at the exact moment sustained focus on the actual business matters as much as ever. Second, getting distracted by acquisition conversations — informal interest from potential acquirers is genuinely flattering, and engaging with it too early or too often is a

recognizable company-killer: it's a real distraction, it's demoralizing when nothing comes of it, and if a real offer does arrive, a team that's already mentally moved toward selling tends to undervalue itself and accept less than the company was actually worth. The practical rule: never start a real acquisition conversation unless you're genuinely willing to sell at whatever number is actually on the table, not the number you're hoping to hear.

The single most important claim in this section, worth stating directly: when startups fail, the actual proximate cause, more often than competition or a bad market or a technical failure, is that the founders themselves gave up. Managing your own psychology well enough to keep going — not blindly, but honestly — is one of the highest-leverage skills a founder can build, precisely because it's the thing most likely to determine whether the company gets the chance to succeed at all.

Marketing, PR, and business development at scale

Chapter 12 already covered why press mostly doesn't matter early on. Once a company is working, that changes, and two specific responsibilities shift back onto the founder personally rather than being fully delegated: defining the company's actual key messaging (never fully outsourced to a marketing hire or a PR firm), and maintaining direct, personal relationships with a small number of journalists, for exactly the reasons Chapter 12 already laid out about referrals and trust. Business development, similarly, only becomes relevant once a real product exists — Chapter 16's persistence and warm-introduction lessons apply directly here too — and Altman's own compressed version of what actually makes deals happen adds two further points worth naming: a genuine personal connection with the other side of any significant deal, rather than treating them as a purely transactional counterparty; and real **competitive dynamics** — having, or credibly appearing to have, another option on the table, since that alternative is usually what actually moves a negotiation, more than any argument made in the room. And, simply: ask directly for what you actually want in a deal, even when it feels aggressive to say it plainly. Most of the time, nobody laughs a founder out of the room for asking — they just don't get what they never asked for.



Common Mistakes

- **Keeping a fully flat structure past the point where it works**, or replacing it with an overcomplicated, multi-manager structure designed to make people feel important rather than to actually clarify who reports to whom.
- **Staying in "hero mode" instead of hiring**, absorbing unsustainable personal strain rather than accepting a short-term backlog while a real team gets built.
- **Assuming a goal stated once, months ago, is still a live priority** — Altman's own "ask ten random employees" test reveals how rarely this assumption holds up.
- **Treating HR as bureaucratic overhead rather than genuine infrastructure** — real compensation bands, ongoing equity grants, and a formal hiring process all speed a company up, not slow it down.
- **Engaging seriously with acquisition conversations without being genuinely willing to sell at the actual number on the table.**
- **Assuming running a company gets emotionally easier as it succeeds.** It doesn't — the highs improve, but the lows get worse, and founders who expect otherwise are caught off guard by exactly the swings that end up determining whether they keep going.



Tradeoffs and Edge Cases

Does every company really need all of this scaling infrastructure by month 18? The timing is a rough guide, not a strict deadline — the underlying principle is that these are the categories of work worth having on your radar *before* they become urgent, not that every company needs to execute on all of them at exactly eighteen months regardless of its actual size or trajectory. A company still well short of product-market fit at that point has bigger, more immediate problems; a company scaling unusually fast might need several of these earlier.

Isn't "founders give up" an oversimplified explanation for startup failure? It's worth taking seriously rather than dismissing, precisely because it's an uncomfortable claim: most founders, and most outside observers, prefer to attribute failure to external causes (competition, market timing, funding) because those explanations don't require examining a founder's own choices. Altman's claim isn't that external factors don't matter — it's that,

holding those factors constant, the proximate decision to stop trying is where most failures actually complete themselves, which is a claim about where to place attention (a founder's own psychology and sustained willingness to continue), not a claim that outside factors are irrelevant.

How does this chapter's advice on structure relate to Chapter 19's editor metaphor?

Directly — this chapter covers *when* and *why* a company needs real management structure in the first place; Chapter 19 covers *how* to actually operate once that structure exists. Read together, they describe a single continuous transition: from a founder who does everything personally, to a founder who edits, delegates, and sets the standard for a company that no longer needs them in every room.



Applying the Test

Run Altman's own test on your own company, or a hypothetical one: if you asked ten random employees what the top three priorities are right now, would they give you the same three, in roughly the same order? If you're not confident they would, that's not a hypothetical problem — it's the specific, checkable symptom of a misalignment that's already costing you productivity, whether or not it's visible yet.



Chapter Summary

- A founder's job shifts from building a great product to building a great company around twenty-five employees — flat structure works until it suddenly doesn't.
- The four common failure modes as founders become managers: fear of hiring senior people, "hero mode," bad delegation, and a missing personal organization system.
- Write down not just your values (Chapter 6) but your actual practices — how things get done — or the company will develop an accidental culture instead of a chosen one.
- Real HR — compensation bands, continuing equity grants, a formal hiring process, attention to diversity — speeds a company up rather than slowing it down.

- Productivity degrades on its own as headcount grows; alignment around a small, constantly-repeated set of priorities is the main countermeasure.
- A cluster of unglamorous mechanical tasks (accounting, patents, trademarks, financial modeling, a dedicated fundraiser) become urgent around month 18 — worth having on your radar well before then.
- Founder psychology gets harder, not easier, as a company succeeds, and the actual proximate cause of most startup failures is founders giving up — managing your own psychology is one of the highest-leverage skills available.
- Marketing, PR, and business development all shift back toward direct founder involvement once a company is working, rather than being fully delegated away.



Exercises

1. **Recall:** Explain why flat organizational structure is described as failing "all at once" rather than gradually, and what the minimum viable fix actually requires.
2. **Apply:** Run Altman's alignment test on a real team you're part of (or imagine running it): would ten random people name the same three priorities? If you're unsure, what would you need to do to find out honestly?
3. **Apply:** Pick one of the "month 18" mechanical tasks (patents, FP&A, a dedicated fundraiser) and describe what getting ahead of it, rather than reacting to it late, would concretely look like for your own company.
4. **Confront a counterexample:** This chapter claims founders giving up is the actual proximate cause of most startup failures, more than competition or market timing. Can you think of a real, well-documented startup failure that seems to have been caused primarily by external factors rather than founder psychology? Does it genuinely contradict this chapter's claim, or does a closer look complicate that reading?



Further Reading

- Sam Altman's original CS183B lecture (Lecture 20) covers additional detail on compensation structuring and equity vesting schedules beyond what this chapter had room to include.
- Chapter 6's discussion of writing down company values is the direct companion to this chapter's advice on writing down company practices — read together, they cover both halves of what needs to be documented as a company scales.
- The Epilogue picks up where this chapter leaves off, covering the full multi-year arc of what building a company actually feels like from the inside.



Epilogue — The Long Road

One of Airbnb's founders once drew a picture on a business card for another founder just starting out. Sam Altman saw it, thought it captured the entire process better than anything he'd tried to explain in words, and kept a photo of it. It's a simple drawing: two gears, one labeled with the product you're building, one labeled with the people who might want it. Between the gears, a line — the gap you're trying to close by getting out and actually meeting your users, over and over, until the gears finally mesh.

Next to the gears, a graph. You launch, usually with some press, and get a spike of attention. The spike falls — not slows down, falls — toward nothing. For most companies, it keeps falling until it actually dips below the starting line: a real, felt sense that the whole thing might be dying, not just struggling. Then a small recovery. Then a long, flat stretch that founders who've lived through it call the trough of sorrow — not because nothing is happening, but because nothing that's happening looks like the growth everyone imagined when they started. A few false-hope wiggles, moments that look like the turn and aren't. And then, if you're one of the companies that makes it through — finally, finally, finally, finally — real growth.

For Airbnb, from the day that graph starts to the day it turns upward, the gap was about a thousand days. Not a thousand days to build the product. A thousand days from launch to the point where the company's own founders could look at the numbers and know, for certain, that it was actually going to work.

This book has spent twenty-one chapters describing what to do at nearly every point along that graph — how to find the idea, how to choose the people, how to build the product, how to get the growth curve to bend, how to raise the money, how to run the company once it's no longer just you. What none of those chapters can fully convey, on their own, is the shape of the whole thing: how long it actually takes, how it feels to be in the trough with no proof yet that you're not just failing slowly, and what the road demands of you that no single chapter's advice can prepare you for in isolation.

The Road Doesn't Always End Where You Hoped, and There's a Right Way to Walk Away

Most startups, including many built by founders who did nearly everything in this book correctly, don't make it through the graph. That's not a failure of the advice — it's an honest description of the base rate, one this book has tried not to hide behind motivational language anywhere in the preceding twenty-one chapters. Because failure is the more common outcome, it's worth closing with what a good version of it actually looks like, since almost every founder will eventually face some version of this decision, even the ones who go on to build something that works the second or third time.

The single most important piece of that decision: tell your investors as soon as you genuinely know the company isn't going to work, rather than waiting and hoping. No serious investor is shocked that a given company failed — they expect most of their investments to, and they've already priced that into how they operate. What earns lasting goodwill, and what genuinely matters for a founder's future, is being honest early rather than letting people find out late. The same honesty applies to employees: they should hear it from you directly, with enough notice and enough of a severance cushion that they're not left in a sudden, unexpected cash crisis, and with real help finding what comes next, rather than discovering the company is over because the door was locked one morning with no warning.

None of this makes failure feel good. It's meant to make clear that there's a version of even the worst outcome this book covers that a founder can walk away from with their reputation, their relationships, and their own sense of having done right by the people who trusted them — intact. That's not a small thing, and it's worth planning for deliberately rather than only thinking about it after it's already too late to do well.

What Kept Showing Up, Across Twenty Different Speakers

One of the more striking things about assembling this book wasn't any single piece of advice — it was how often completely independent speakers, teaching different subjects to different audiences over the years this course ran, arrived at the identical underlying structure without ever citing each other.

Three separate lecturers — Sam Altman on ideas, Peter Thiel on monopoly theory, and Aaron Levie on enterprise software — each described, independently, the same concentric pattern:

start in a market so small it looks unattractive to almost everyone else, dominate it completely, and only then expand outward in stages. None of them were quoting each other. All three arrived at the same shape because it's apparently just true, robust enough to show up in three unrelated tellings.

The same thing happened with user research. Paul Graham's argument that expertise in your own users matters more than expertise in startup mechanics, Emmett Shear's insistence that you ask about existing behavior rather than proposed features, and Hosain Rahman's account of why Jawbone never asked customers directly whether they'd pay for a wireless speaker — three different industries, the same underlying discipline, discovered separately rather than copied.

A softer version of the same pattern connects Reid Hoffman's account of the "paradoxes" a founder has to navigate — flexible vs. persistent, belief vs. fear, data vs. vision — to Ben Horowitz's account of seeing every management decision through the whole company's eyes, and to Keith Rabois's account of when to delegate completely versus when to decide something yourself. These are three genuinely different frameworks, built for three different problems, and none of the three lecturers drew this connection themselves — but laid side by side, they share a family resemblance worth naming: each one is really a tool for judgment under a situation with no fixed right answer, rather than a rule that always points the same direction. Different vocabulary, a related kind of underlying call, showing up across otherwise unconnected lectures: the actual skill this entire book is trying to teach is knowing, in the moment, which side of a real tension the situation in front of you calls for.

If there's a single thread that ties the whole book together, it's this: nearly every piece of advice in these twenty-one chapters is really the same handful of underlying judgments, worn by different speakers into the specific shape of whatever problem they happened to be solving. Learning to recognize the underlying judgment, rather than memorizing each speaker's specific version of it, is what turns this book from a collection of anecdotes into something closer to a genuine skill.

Back to the Test This Book Opened With

Chapter 1 argued that the only good reason to do any of this is a specific, checkable feeling: you can't not do it, and the world — not just you — needs it built. Everything in the twenty chapters since then has been useful only in service of that one condition being true. The tactics for choosing a cofounder, building an MVP, raising a seed round, and scaling past twenty-five employees are all, in the end, just the mechanics of carrying out a decision that has to come from

somewhere else entirely — a problem you were already living inside, before you ever opened a book like this one to learn what to do about it.

If that's not true for you yet — if you're reading this book while still looking for the idea, still waiting for the compulsion Chapter 1 described — that's not a reason to force the rest of this process before it's ready. Go back to Paul Graham's advice from Chapter 2: learn about things that genuinely interest you, work with people you respect, and let the idea find you, rather than the reverse.

And if it is true — if there's already a problem you can't stop thinking about, and people who need it solved as much as you do — then the thousand days on that graph are waiting for you the same way they waited for every founder in this book. Long, mostly invisible from the outside, and, if you get enough of the twenty-one chapters before this one right, eventually worth it.



Exercises

1. **Recall:** Describe the shape of the Airbnb graph — spike, fall below the starting line, trough of sorrow, false-hope wiggles, real growth — and explain why founders in the trough usually can't yet tell which part of that shape they're in.
2. **Apply:** If you're currently in (or can imagine being in) something like the trough of sorrow, name one concrete signal, from any chapter in this book, you'd actually check to tell honest stagnation apart from a genuinely dying idea.
3. **Apply:** Write down, in one paragraph, whether the "you can't not do it" test from Chapter 1 is currently true for you. If it isn't yet, name one specific thing from Chapter 2's advice you could do this month to let an idea find you, rather than forcing one.
4. **Confront a counterexample:** This epilogue argues that telling investors and employees the truth early, when a company is failing, is the right way to walk away. Can you think of a situation where early honesty about likely failure could plausibly backfire — for the company, or for the people being told? How would you weigh that risk against the case this epilogue makes?



Further Reading

- Sam Altman's own account of the Airbnb business-card drawing, and of the "thousand days" it took the company to turn the corner, is covered in his CS183B lecture.
- Paul Graham's essay "How to Get Startup Ideas" (Chapter 2) is the direct source of the "let the idea find you" advice this epilogue closes on.
- Chapter 1 is worth rereading immediately after this epilogue — the "you can't not do it" test it opens with is the same test this book closes on.



Appendix A — A Full Cap Table Walkthrough

Chapter 15 introduced the mechanics of a valuation cap and gave a taste of how dilution compounds across rounds. This appendix walks through one complete, simplified example from founding to a Series A, with real numbers at every step, so you can see exactly where each shareholder's percentage comes from — not just the concept, but the arithmetic underneath it.

A caveat worth stating up front: real cap tables are messier than this. They involve multiple SAFEs at different terms, seed extensions, warrants, and negotiated side-letters. This walkthrough uses one clean, illustrative scenario specifically so the mechanism is visible. Don't treat the specific percentages below as typical — treat the *method* as transferable.

Step 1: Founding

Two cofounders incorporate and issue themselves 8,000,000 shares of common stock, split evenly.

Shareholder	Shares	Ownership
Founder A	4,000,000	50%
Founder B	4,000,000	50%
Total	8,000,000	100%

Both founders' shares vest over four years with a one-year cliff, per Chapter 15.

Step 2: The Seed Round (a post-money SAFE)

The company raises \$2,000,000 on a SAFE with a **\$10,000,000 post-money valuation cap**. This detail matters: a *post-money* cap (the standard since Y Combinator updated its template, mentioned in Chapter 15) means the cap already accounts for the SAFE money itself. That makes

the investor's eventual ownership percentage simple to calculate directly, with no circular math required:

$$\text{Investor's locked-in ownership} = \text{Investment} \div \text{Post-money cap} = \$2,000,000 \div \$10,000,000 = \mathbf{20\%}$$

Nothing converts into actual shares yet — a SAFE holder isn't a shareholder until a future priced round triggers conversion (Chapter 15). But the 20% figure is now locked in, and it will get diluted by future rounds exactly the same way the founders' ownership will.

The company's ownership picture right after the SAFE closes (on a fully diluted, as-if-converted basis):

Shareholder	Ownership
Founder A	40%
Founder B	40%
Seed SAFE investor (locked in, not yet issued shares)	20%
Total	100%

Step 3: The Series A

A year later, the company raises a Series A: \$5,000,000 from a new investor, at a **\$20,000,000 pre-money valuation** (meaning \$25,000,000 post-money). At the same time, the board creates a new **15% option pool** for future employees — carved out of the existing (pre-money) ownership, not out of the new investor's stake, which is the standard structure.

The new investor's share is simple: $\$5,000,000 \div \$25,000,000 = \mathbf{20\% \text{ of the post-Series-A company.}}$

Everyone who held equity *before* this round — the two founders and the converting seed SAFE — now splits what's left: $100\% - 20\% \text{ (new investor)} - 15\% \text{ (option pool)} = \mathbf{65\%}$, divided among them in the same proportion they held before this round (40% / 40% / 20%).

Shareholder	Pre-Series-A ownership	Share of the remaining 65%	Final post-Series-A ownership
Founder A	40%	$40\% \times 65\%$	26%
Founder B	40%	$40\% \times 65\%$	26%
Seed SAFE investor (now converted to real shares)	20%	$20\% \times 65\%$	13%
New Series A investor	—	—	20%
Option pool (for future employees)	—	—	15%
Total			100%

What Actually Happened, in Plain Terms

The two founders started at 50% each. After one seed round and one Series A — with a brand-new option pool created along the way — they're each down to 26%, a combined 52%. That's a completely normal, healthy outcome for a company that's successfully raised two rounds; it's not a sign anything went wrong. What it illustrates is exactly the point Chapter 13 raised: think in advance about the ownership level at which you'd personally start to feel demotivated, because two more rounds after this one (a Series B, a Series C) will each dilute the founders further using the identical mechanism.

Notice, too, where the dilution actually came from at each step: the seed SAFE diluted everyone equally once it converted, but the *option pool* at the Series A came specifically out of the founders' and the seed investor's pre-existing stakes, not out of the new investor's contribution — which is why option pool size is a real, negotiable term worth understanding, not an afterthought to accept whatever a term sheet proposes by default.

Try It Yourself

Using this same method, model your own company's projected cap table:

1. Start with your actual or planned founder share count and split.

2. Add a seed round: pick a realistic raise amount and post-money cap for your stage and market, and calculate the resulting locked-in percentage.
3. Add a Series A: pick a realistic new-money amount and pre-money valuation, decide on an option pool size, and work through the same three-step division this appendix used.
4. Look at your own final founder percentage. If it's lower than you expected, that's worth knowing now — before you sign anything — rather than after.



Appendix B — Reading a Term Sheet Clause by Clause

A term sheet is a short document — usually two to five pages — that lays out the terms an investor is proposing for a priced round (Chapter 15 covers the difference between a priced round and an unpriced SAFE or note). It isn't the final legal agreement, but everything in the final agreement gets negotiated here first. This appendix walks through the clauses you'll actually see, in the order they usually appear, so that reading one for the first time doesn't feel like reading a foreign language.

Valuation (pre-money and post-money). Pre-money valuation is what the company is considered worth *before* the new investment. Post-money is pre-money plus the new money being raised. If a term sheet proposes a \$20 million pre-money valuation on a \$5 million raise, the post-money valuation is \$25 million, and the new investor is buying 5/25, or 20%, of the company. Every other percentage in the term sheet is anchored to this one number.

Liquidation preference. This determines who gets paid first, and how much, if the company is ever sold or liquidated — before any remaining proceeds get split by ownership percentage. The standard, investor-friendly-but-normal version is a **1x non-participating preference**: in an exit, preferred shareholders get the larger of either (a) their original investment back, or (b) their pro-rata share of the proceeds as if they'd converted to common stock — whichever is worth more to them. Watch for two variations that shift the balance meaningfully toward investors: a preference **higher than 1x** (meaning investors get back more than they put in before anyone else sees a dollar), and a **participating** preference (meaning investors get their preference *and then also* participate in the remaining proceeds pro-rata, effectively double-dipping). Both are red flags worth pushing back on if they appear in an early-stage term sheet.

Option pool. Appendix A showed exactly how this affects a cap table. The clause to watch for is *when* the pool gets created relative to the valuation — a pool created "pre-money" (before the new investor's money, diluting existing shareholders only) is standard; be alert to language that would size the pool in a way that dilutes only the founders and not the incoming investor.

Board composition. This clause specifies who sits on the board after the round closes — commonly some mix of founders, investors, and independent members. Chapter 13 already covered why this matters less in practice than founders fear, as long as trust is high, but it's still worth reading carefully: a board composition that gives investors control of the majority of seats this early is unusual and worth understanding exactly why, if it appears.

Protective provisions. A list of specific actions the company cannot take without investor approval — commonly things like selling the company, raising future rounds on certain terms, taking on debt, or changing the size of the board. These are normal and expected. What's worth checking is whether the list is a reasonable, standard set of major decisions, or an unusually long list that would require investor sign-off on routine operational decisions.

Anti-dilution provisions. These protect an investor if the company later raises money at a *lower* valuation than this round (a "down round"). The standard, founder-friendlier version is **broad-based weighted average** — a formula that adjusts the investor's effective price modestly, accounting for how large the down round is relative to the company's overall size. The much harsher version, **full ratchet**, would reprice the investor's entire stake to match the new, lower price regardless of the down round's size — a term that can be devastating to founder and employee ownership if it's ever triggered, and one worth resisting in early-stage negotiations.

Pro-rata rights. Covered in Chapter 15: the right for this investor to invest again in future rounds to maintain their percentage ownership. Standard and generally reasonable, though it does mean founders bear more cumulative dilution as investors exercise it round after round.

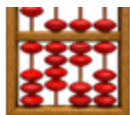
Drag-along and tag-along rights. A drag-along clause means that if shareholders holding a specified majority agree to sell the company, remaining shareholders can be required ("dragged along") to sell too, on the same terms — this prevents a small minority from blocking an exit the rest of the company wants. A tag-along clause is the protective mirror image for smaller shareholders: if a majority holder sells their shares, minority holders get the right to sell alongside them on the same terms, rather than being left behind as a minority partner to whoever bought in.

No-shop / exclusivity period. Once you sign a term sheet, this clause commits you to not soliciting or negotiating with other investors for a specified window — commonly 30 to 60 days — while the investor completes due diligence and finalizes the legal documents. This is standard, but signing a term sheet is a real commitment, even though it isn't the final binding agreement; don't sign one you're not seriously prepared to close.

Founder vesting / re-vesting. Occasionally, an investor's term sheet will ask founders to restart their vesting schedule (or vest a portion of previously-vested shares again) as part of the round, specifically to ensure founders stay motivated and committed for the new round's investment horizon. This is worth understanding clearly before agreeing to it, since it directly affects what happens to a founder's equity if they leave the company after this round closes.

The One Habit Worth Building

Every clause in this appendix answers the same underlying question from a different angle: *what happens if things go well, and what happens if they don't*. Read every term sheet clause twice — once assuming the company is a huge success, and once assuming it struggles or gets sold for less than everyone hoped. A clause that looks harmless in the success scenario can look very different in the second one, and that's exactly the scenario a term sheet's protective language is usually written for.



Appendix C — Unit Economics Formula Sheet

A quick-reference sheet for the formulas introduced across this book, gathered in one place. Each entry names the chapter where the concept was first explained in full — flip back there for the reasoning and worked examples behind the formula.

Growth and Retention (Chapters 10–11)

Retention rate — the percentage of a cohort still active after a given time period. Plot it against time since acquisition to get a retention curve; an asymptotic (flattening) curve signals real product-market fit.

North Star metric — not a formula, but a discipline: the single number (monthly active users, nights booked, messages sent) the whole company optimizes toward.

Viral coefficient (K-factor)

$K = \text{payload (invites sent per user)} \times \text{conversion rate (of those invited who sign up)} \times \text{frequency (how often the invite cycle repeats)}$

K above 1 means a user base grows on its own from virality alone; below 1, it eventually flatlines without other growth channels.

CAC (Customer Acquisition Cost)

$CAC = \text{Cost per click} \div \text{Conversion rate}$

CLV / LTV (Customer Lifetime Value) — the total net revenue a customer generates over the time they remain a customer, typically calculated over a fixed window (a "12-month CLV," for instance).

Unit economics profit per customer

$$\text{Profit} = \text{CLV} - \text{CAC}$$

A positive number means the channel is worth scaling; a negative one means you're paying more to acquire customers than they're worth.

Payback period — how long it takes a customer's revenue to repay the CAC spent to acquire them. A 3-month payback is a conservative, safe benchmark; 12 months is riskier but sometimes reasonable for a well-capitalized company; beyond 12 months moves into territory where a company is betting its cash position on an unproven prediction.

Financial Statements and Company Health (Chapter 15)

Burn rate — net cash spent per month (expenses minus revenue).

Runway

$$\text{Runway (in months)} = \text{Cash in the bank} \div \text{Monthly burn rate}$$

Every founder should be able to state this number from memory.

P&L (Profit and Loss statement) — revenue minus expenses over a period of time.

Balance sheet — a snapshot, at one moment, of assets (what the company owns) minus liabilities (what it owes), equal to equity (what's left over).

Cash flow statement — actual cash moving in and out, distinct from the P&L's "profit," since revenue is counted when earned, not necessarily when the cash arrives.

Fundraising Mechanics (Chapters 13–15, Appendices A–B)

SAFE / note valuation cap conversion (post-money style)

$$\text{Investor's locked-in ownership} = \text{Investment} \div \text{Post-money valuation cap}$$

Pre-money vs. post-money valuation

Post-money valuation = Pre-money valuation + New money raised

New investor's ownership = New money raised ÷ Post-money valuation

Vesting schedule (standard 4-year, 1-year cliff) — 0% vested before month 12; 25% at month 12; the remaining 75% vests monthly over the next 36 months.

A Reminder Before You Use Any of These

Every formula on this page is only as useful as the segment-level detail behind it. Chapter 11 makes this point directly about CAC and CLV: a single blended average across your entire customer base can look healthy while hiding a badly unprofitable segment sitting right next to a highly profitable one. Calculate these numbers by customer segment, by channel, and by cohort — never just once, for the whole company, and call it done.





Appendix D — The Full Pitch Roleplay

Transcript

Chapter 14 condensed the lessons from a live mock-pitch exercise run in the original course, led by YC partner Dalton Caldwell with a founder (referred to as Qasar) role-playing both a weak pitch and a strong one back to back. The full, unedited exchange is reproduced here, exactly as it happened, for readers who want to study the complete back-and-forth rather than the chapter's summary of its lessons.

The First Pitch (Weak)

Dalton: Qasar, I understand you're coming to pitch me today. What can you tell me about what you do?

Qasar: We're building a communication platform that will allow businesses and consumers to collaborate on one single platform rather than in the fractured state that they're in right now.

Dalton: I don't follow.

Qasar: Think about WhatsApp or Snapchat. Those are for consumers. We want to do that for businesses. What that means is we want to enable consumers to talk to businesses. That's the goal of our business, or what our startup is.

Dalton: Who uses this product? What does the product do?

Qasar: It's for consumers and businesses. A messaging product that allows consumers to send —

Dalton: Why would a consumer want to use your product?

Qasar: Because they want to message a business.

Dalton: What can you tell me about the market and the opportunity? What's the size of this company?

Qasar: Messaging companies are really big, obviously. WhatsApp sold for 19 billion dollars. Snapchat is really growing very quickly as well. We think the opportunity is very big.

Dalton: Can you tell me a little bit about your traction, your numbers? Have you given this to people yet?

Qasar: We don't want to open the kimono and go into all the details here. At a high level, we definitely have thousands of users in the Bay Area. Hundreds of businesses.

Dalton: Can you tell me who some of those businesses are?

Qasar: There's ones that you've been to. We don't really want to get too much into the details because we're still early — we're trying to stay stealth.

Dalton: Okay, well, can you tell me about what you've learned so far? What insights have you had from customers?

Qasar: Yeah, the consumers are sending messages to these businesses. And we think that's great. And these businesses are responding to the messages, and we think that's — I don't think it's obvious that would happen.

Dalton: So can you tell me about what your business model is and how —

Qasar: Yeah, so we charge businesses like a monthly rate. We haven't precisely figured out what that is. Right now we're free for the few hundred companies we're in right now, but we're looking to probably do a monthly —

Dalton: How much do you think a business would be willing to pay?

Qasar: We think certainly ten to fifteen thousand dollars a month...

Dalton: Okay. So, can you tell me a little bit about your team and who you have working on this?

Qasar: Yeah, we have five founders. Technically I am the only one who's full-time right now. We're raising money so we can get the rest of the team on board.

Dalton: Can any of the founders program?

Qasar: Yeah. One of them has a Bio PhD but he's really picked up coding. I'm a Python developer — I learned Python the hard way.

Dalton: Look at the time. Well, it's been really great meeting you. Please keep me in the loop. This sounds fantastic.

Qasar: I will send you an update.

Dalton: Just keep me in the loop as this progresses.

Qasar: I'll send you an update. [Breaking character] Great, that was awful.

The Second Pitch (Strong)

Dalton: Qasar, I understand you have a company. Can you tell me a little bit about what you guys do?

Qasar: Yes, we're a messaging product — that's kind of vague. What we allow you to do is essentially message a location. When you walk into a Crate and Barrel, you can send the manager a message like, "Hey, there's puke in the hallway." Or if you're in the airport: "I can't find this specific gate — where's the terminal for Virgin?" Or at Target: "What aisle is the shampoo in?"

Dalton: So is this a mobile app?

Qasar: On the consumer side we have an iOS and Android app, but getting consumers to download apps is obviously very difficult.

Dalton: I don't usually download an app just to send a message to Crate and Barrel.

Qasar: Most businesses have a call to action that says "text the owner directly." We tested a bunch of copy, and what works best is small print saying the messages are anonymous — that lowers the barrier to entry. The most counterintuitive thing we've learned in the launch we've had — three hundred fifty locations in the Bay Area, about three months in, roughly 11% weekly growth in participating businesses — is around whether people would actually send messages at all. We weren't sure that would work.

Dalton: Do people send messages?

Qasar: They do.

Dalton: What's the number one type of message people send?

Qasar: We originally thought this would be in-location feedback — that was the premise. What we found is more than half the messages aren't about feedback at all. People ask things like — there was a kebab stand in San Jose where messages came in asking "are you hiring?" Which is strange, because you'd think they'd just ask the owner directly. But the person walking in doesn't know who the owner is, and we do, so texting through us is actually the easier path for them.

Dalton: Okay, so it's like a suggestion box — a way to message a business?

Qasar: That's what we initially thought it was. But what we actually discovered is that over half the messages are just things like: "When do you open? When do you close?" — because that's not on Google. "Are you catering? Do you have reservations tonight?" That kind of thing.

Dalton: Okay. In terms of your traction, tell me what you guys have right now.

Qasar: Three hundred fifty businesses, San Jose to San Francisco. We sold all of them ourselves — the three of us are all technical, but we did all the sales, because we learned a lot about how these businesses actually work that way. We come from a retail background. We originally built this for large enterprise players — Starbucks, Walmart — but we realized closing those contracts wasn't realistic on our runway, so we got the product into users' hands with small businesses instead, and that's when we discovered this messaging pattern.

Dalton: That sounds interesting. It sounds like you have customers. How can this be big, though?

Qasar: In terms of numbers, we see about one and a half messages on average per location per day. That might not sound like much, but compare it to a Yelp or Google review — a business might get five or seven of those in its entire lifetime. So this is a much higher volume of engagement than what they're used to, and it's private, not public. As for how we actually make money — frankly, we don't have a fully clear answer yet. There are two paths: the small-business side, or the large-customer side. Large customers, from our retail experience, pay three to four million dollars a year for feedback tools like this — think Sears, where we came from. Small businesses, in testing, are willing to pay around fifty dollars a month. I think this can be a large business, but there are two genuinely different paths to get there.

Dalton: I can see that. Can you tell me about your distribution strategy, and a little about the team?

Qasar: On distribution — what we learned selling to small businesses is that it's genuinely hard: LTV minus CAC just doesn't work out well in that segment on its own. So we have two paths. One is going up-market, back toward Starbucks or Walmart-scale customers. The other is partnering with consumer-facing companies — Yelp, Google, Facebook.

Dalton: Have you talked with them? Are they actually going to do it?

Qasar: We've talked to Google and Facebook, and we're meeting with Apple. The idea is that every time you search for a business, there should be a message button — we want consumers in the habit of knowing they can text any business directly. That gets us broad distribution. Our real vision is to become the messaging infrastructure between consumers and businesses. If those partnerships don't materialize — if Google, Facebook, and Yelp don't want to give up that surface — we still have a real business simply selling this as a feedback tool to large players directly.

Dalton: Tell me a little about the team — we're running low on time.

Qasar: There are three of us, all technical. Mike and I started a company together before — it failed, so I don't know if that's a point in our favor or not — and Sonny was an engineer before this. We come from retail. We built everything ourselves, and we sold everything ourselves.

Dalton: Okay.

Qasar: We've already had a couple of conversations with your firm. We're raising five hundred thousand dollars on an \$8.5 million convertible note. Of that, two hundred fifty thousand is already committed by [named angel investors], and [a named seed fund] is willing to fill the rest of the round. We think you and your firm could bring real value given your retail experience — is this something you'd be interested in?

Dalton: Yeah, I think this is really interesting. I'd need to talk to a couple more people on my side, but I do think this could be pretty big.

Qasar: Since we've already had a couple of conversations, we're certainly willing to meet again — we're closing the round this Friday, so I'll follow up with you before then. We'd love to have you in the round.

Dalton: Okay, sounds good — I've got to run, but thanks for that.

Reading It Cold

Notice, reading the two side by side, that neither pitch differs in format, length, or enthusiasm. Every difference traces back to the seven components Chapter 14 named: a concrete one-sentence description instead of a vague one, real numbers instead of evasion, a genuine and specific insight instead of a generic claim, an honest answer on business model instead of a hedge, a real team story instead of a thin one, and — the thing the first pitch never got to at all — an actual, fully specified ask.



Glossary

Terms are listed alphabetically. Each entry names the chapter where the term is first introduced and explained in full.

83(b) election — an IRS form that must be filed within 30 days of receiving restricted stock, locking in a founder's tax basis while the company's value is still low. (Ch. 15)

Accredited investor — an investor who meets specific income or net worth thresholds, presumed sophisticated enough to bear the risk of an early-stage investment. (Ch. 15)

ACRU (Activated Confirmed Registered User) — eBay's internal metric for a user who didn't just register but actually bid on or listed an item; contrasted with a raw Confirmed Registered User (CRU). (Ch. 10)

Ammunition — in Keith Rabois's framework, people who do excellent work but need direction, as opposed to a "barrel" who can drive an idea from concept to shipped reality on their own. (Ch. 19)

Anti-dilution provision — a term sheet clause protecting an investor if the company later raises money at a lower valuation. See broad-based weighted average and full ratchet. (Appendix B)

Balance sheet — a snapshot, at one moment, of a company's assets minus liabilities, equal to equity. (Ch. 15)

Barrel — see Ammunition; the person who can drive an idea all the way to shipped reality. (Ch. 19)

Board seat — a position on a company's board of directors; Chapter 15 recommends declining requests for one unless the investor adds genuine, ongoing strategic value.

Burn rate — the net amount of cash a company spends per month, beyond what it brings in. (Ch. 15)

CAC (Customer Acquisition Cost) — the cost to acquire one paying customer, commonly calculated as cost per click divided by conversion rate. (Ch. 11)

Cap table — the record of who owns what percentage of a company, and how that changes across financing rounds. (Ch. 15; full worked example in Appendix A)

Champion — a customer who actively advocates for a product to others, as distinguished from a merely satisfied customer. (Ch. 9)

Churn — the rate at which existing customers stop using a product or stop paying for it. (Ch. 8, Ch. 10)

CLV / LTV (Customer Lifetime Value) — the total net revenue a customer generates over a defined period as a customer. (Ch. 11)

Cofounder vesting — see Vesting; applies to founders exactly as it does to employees. (Ch. 4, Ch. 15)

Cohort — a group of users who joined or signed up during the same period, tracked together to see how their behavior (especially retention) changes over time. (Ch. 7, Ch. 10)

Concentric expansion — the strategy of dominating a small market completely, then expanding outward into progressively larger, adjacent markets. Introduced independently by three separate lectures in this book (Chapters 2, 3, and 16). (Ch. 2)

Contractor vs. employee — a legal distinction with different tax treatment (1099 vs. W-2) and different obligations (workers' compensation, work-hour control). (Ch. 15)

Convertible note — a debt instrument that converts into equity at a future priced round, an alternative to a SAFE. (Ch. 15)

Drag-along / tag-along rights — term sheet clauses governing what happens to minority shareholders when a majority decides to sell the company. (Appendix B)

Elephants and rabbits — Christoph Janz's framework for categorizing a business by customer count and price point — a few customers at a high price ("elephants") versus many customers at a low price ("rabbits"), with a spectrum of combinations in between — each requiring a genuinely different sales motion. (Ch. 16)

FF stock — a special class of founder stock structured so founders can sell some of it in a later financing round without distorting the common stock valuation. Setting it up at company formation reads as a bad signal to investors; the safe time is later, once the business is already working. (Ch. 21)

Four Horsemen — John Gottman's four relationship-ending behaviors (criticism, contempt, defensiveness, stonewalling), applied in this book to customer support. Stonewalling — simply not responding — is identified as the most damaging to a startup. (Ch. 8)

Full ratchet — the harshest form of anti-dilution provision, repricing an investor's entire stake to match a lower down-round price regardless of the round's size. (Appendix B)

GMV (Gross Merchandise Volume) — the total dollar value of goods or services transacted through a marketplace, as distinct from the marketplace's own revenue (its cut of that volume); eBay's GMV ran at roughly 10 times its reported revenue. (Ch. 10)

Growth accounting (new / resurrected / churned) — a framework for breaking down user growth into new signups, users who return after 30+ days away, and users who leave — useful because resurrected and churned users often dominate the numbers once a product has been around for a few years. (Ch. 10)

Honesty curve — Adora Cheung's observation that feedback gets more honest the less personally close the source is to you (strangers more honest than friends, friends more honest than family) — and that charging money for a product makes strangers' feedback even more honest still, since real money is now on the line. (Ch. 7)

Investment thesis — a written, specific hypothesis for why a company will work, including what the founder believes they know that skeptics don't; tracked over time by whether confidence in it is rising or falling. (Ch. 18)

K-factor — see Viral coefficient.

Knowledge gap — Jared Spool's concept: the distance between what a user already knows and what they'd need to know to use a product well; closed either by increasing user knowledge or decreasing what the product requires. (Ch. 8)

Last mover advantage — Peter Thiel's argument that durability, not speed to market, determines whether a company captures most of its eventual value; the goal is to be the last significant company in a category, not the first. (Ch. 3)

Liquidation preference — a term sheet clause determining who gets paid first, and how much, in an acquisition or liquidation, before remaining proceeds are split by ownership percentage. (Appendix B)

Magic moment — the specific instant a new user genuinely understands what a product is for (seeing a friend's photo on Facebook; finding the right listing on Airbnb). (Ch. 10)

Moat — a term popularized by Warren Buffett for whatever makes a business hard for a competitor to attack; in this book's terms, the characteristics that make a monopoly durable (proprietary technology, network effects, economies of scale, branding). (Ch. 3)

Monopoly (business sense) — a business with no serious competitor doing what it does, stable enough to plan years ahead and capture real profit; contrasted with perfect competition. (Ch. 3)

MVP (Minimum Viable Product) — the smallest version of a product that still genuinely solves the core problem; the emphasis belongs on "viable," not "minimum." (Ch. 7)

NPS (Net Promoter Score) — a simple 0–10 rating of how likely a user is to recommend a product to a friend, used as a fast leading indicator of retention before a full retention curve has had time to develop. (Ch. 7)

North Star metric — the single number a company chooses to align every team's decisions around (monthly active users, nights booked, messages sent), rather than running a separate, siloed growth team. (Ch. 10)

Onion theory of risk — Andy Rachleff's framework, via Marc Andreessen: a startup carries many layers of risk (team, product, technical, launch, market, revenue, cost of sales, viral growth), and each funding round should be sized to remove a specific layer, not simply to raise as much as possible. (Ch. 13)

Option pool — shares set aside for future employees, typically created or expanded at a priced financing round; where it's carved from (pre-money vs. post-money) affects who bears the dilution. (Ch. 15; Appendix A)

P&L (Profit and Loss statement) — revenue minus expenses over a period of time. (Ch. 15)

Payback period — how long it takes a customer's revenue to repay the cost of acquiring them; three months is a conservative benchmark, twelve months is a riskier upper bound. (Ch. 11)

Payload, conversion, frequency — Sean Parker's three factors determining virality: how many people a viral action reaches, what fraction convert, and how often it happens. (Ch. 11)

Perfect competition — a market with many similar sellers and no meaningful pricing power, where, per Peter Thiel, almost nobody makes real money; contrasted with monopoly. (Ch. 3)

Post-money / pre-money valuation — pre-money is what a company is considered worth before new investment; post-money is pre-money plus the new money raised. (Ch. 15; Appendix A, B)

Priced round — a financing round (typically Series A or later) that sets an actual valuation for the company, as opposed to an unpriced SAFE or convertible note. (Ch. 15)

Pro-rata rights — an investor's right to invest again in future rounds to maintain their existing ownership percentage. (Ch. 15; Appendix B)

Protective provisions — a list of major company actions that require investor approval, written into financing terms. (Appendix B)

Retention curve — a plot of what percentage of a user cohort remains active over time since acquisition; an asymptotic (flattening) curve signals product-market fit, a curve decaying to zero signals its absence. (Ch. 7, Ch. 10)

Runway — how many months of operation remain at the current burn rate before a company runs out of cash (cash in bank ÷ monthly burn). (Ch. 15)

SAFE (Simple Agreement for Future Equity) — an instrument, alongside the convertible note, that lets an investor provide cash now in exchange for equity later, once a priced round sets a valuation. (Ch. 15; Appendix A)

Support-Driven Development (SDD) — Kevin Hale's practice of having everyone on a team, including engineers and founders, personally handle customer support, to keep the product feedback loop intact. (Ch. 8)

TAM (Total Addressable Market) — the ceiling on how large a market could eventually become; useful only alongside the market's growth rate, not on its own. (Ch. 2)

Task-relevant maturity — Andy Grove's concept: how experienced a specific person is at a specific task right now, which should determine how much a manager delegates versus guides, rather than applying one uniform management style to everyone. (Ch. 19)

Term sheet — a short document laying out the proposed terms of a priced financing round, negotiated before the final legal agreements. (Ch. 15; full walkthrough in Appendix B)

Unique insight — the specific thing a founder understands that skeptics or competitors don't, required for a contrarian idea to also be a correct one. (Ch. 18)

Valuation cap — the upper limit on the valuation used to convert a SAFE or note into equity, rewarding early investors with a better effective price once a priced round sets a higher valuation. (Ch. 15; Appendix A)

Vesting — the process by which equity is earned over time rather than owned immediately; the Silicon Valley standard is four years with a one-year cliff. (Ch. 4, Ch. 15)

Viral coefficient (K-factor) — roughly, how many new users each existing user brings in; above 1 compounds into exponential growth, below 1 eventually flatlines. (Ch. 11, Ch. 17)

Why now — the discipline, credited to Sequoia, of identifying a specific, dated, external reason an idea works today and didn't work two years ago. (Ch. 2)

X and Y (value creation and capture) — Peter Thiel's framing: a company's value depends on how much value it creates (X) and what percentage of that value it captures (Y) — two independent variables, both required for a company to be genuinely valuable. (Ch. 3)



Bibliography

Primary Source

Stanford University, CS183B: How to Start a Startup (Fall 2014). Twenty lectures delivered by founders, investors, and operators, taught by Sam Altman with guest lectures from Dustin Moskovitz, Paul Graham, Adora Cheung, Peter Thiel, Alex Schultz, Kevin Hale, Stanley Tang, Walker Williams, Justin Kan, Marc Andreessen, Ron Conway, Parker Conrad, Alfred Lin, Brian Chesky, Patrick Collison, John Collison, Ben Silbermann, Aaron Levie, Reid Hoffman, Keith Rabois, Ben Horowitz, Emmett Shear, Hosain Rahman, Kirsty Nathoo, Carolynn Levy, Tyler Bosmeny, Michael Seibel, and Dalton Caldwell. This book is a synthesis of all twenty lectures, reorganized and expanded; every chapter's "Further Reading" section notes which lecture or lectures it draws from.

Books

Fried, Jason, and David Heinemeier Hansson. *It Doesn't Have to Be Crazy at Work*. Referenced in Chapter 1 as the fullest case for the "calm company" alternative to hyper-growth culture.

Gottman, John. Referenced in Chapter 8; Gottman's research on marriage and conflict prediction is the basis for this book's customer-support framework (the "Four Horsemen," and the money/kids/sex/time categories of conflict).

Grove, Andy. *High Output Management* (1982). Referenced in Chapter 19 as the source of task-relevant maturity and the "measure output, not motion" framing of management.

Horowitz, Ben. Referenced in Chapter 20; Horowitz's own account of the Toussaint Louverture case draws on C.L.R. James's historical work on the Haitian Revolution.

Lencioni, Patrick. *The Five Dysfunctions of a Team*. Referenced in Chapter 6 as the source of the trust-conflict-commitment-accountability-results framework.

Thiel, Peter, with Blake Masters. *Zero to One*. Referenced in Chapters 2 and 3 as the fuller development of the monopoly-theory argument covered from Thiel's original lecture.

Treacy, Michael, and Fred Wiersema. *The Discipline of Market Leaders*. Referenced in Chapter 8 as the source of the best-price / best-product / best-overall-solution framework.

Walsh, Bill. *The Score Takes Care of Itself*. Referenced in Chapter 19 for the 49ers turnaround and the argument that operational details, consistently executed, matter more than any single strategic insight.

Essays and Articles

Andreessen, Marc. "Why Software Is Eating the World." Referenced in Chapter 2 as the fuller argument behind the "why now" discussion.

Graham, Paul. "How to Get Startup Ideas." Referenced in Chapter 2 as the source of the "live in the future" and "don't consciously search for ideas" arguments.

Graham, Paul. "Do Things That Don't Scale." Referenced in Chapter 9 as the origin of that chapter's title and additional examples.

Holiday, Ryan. *Trust Me, I'm Lying*. Referenced in Chapter 12 as a deliberately cynical account of how stories spread online, useful for context though not this book's recommended method.

Janz, Christoph. "Five Ways to Build a \$100 Million Business." Referenced in Chapter 16 as the source of the elephants-to-rabbits customer-count/price-point framework.

Kincaid, Jason. Overview of pitching press, referenced in Chapter 12 for additional tactical depth on media relations.

The Economist. "Entrepreneurs Anonymous" (2014). Referenced in Chapter 1 as the source Moskowitz cites on founder depression as a documented phenomenon.

Other Referenced Sources

Altman, Sam. Blog post on stock option exercise windows, referenced and critiqued directly in Chapter 20.

Buffett, Warren. Shareholder letters, referenced in Chapter 3 as the origin of the "moat" terminology as applied to durable competitive advantage.

Rachleff, Andy. Original writing on the "onion theory of risk," referenced in Chapter 13 via Marc Andreessen.

Y Combinator. Standard startup legal document library (incorporation, financing, and employment templates), referenced in Chapter 15 and throughout as the practical starting point for the mechanics that chapter covers.





Index

This book has no fixed page numbers — it's organized by chapter instead. Entries below point to the chapter (or appendix) where a topic, person, or company is discussed. Defined terms also appear in the Glossary; this index is for finding *where* something is discussed, not what it means.

People

Altman, Sam — Ch. 1, 2, 4, 5, 13, 17, 21, Epilogue
Andreessen, Marc — Ch. 2, 13, 16
Bosmeny, Tyler — Ch. 14, 16
Buffett, Warren — Ch. 3
Caldwell, Dalton — Ch. 14, Appendix D
Chesky, Brian — Ch. 1, 6, 9
Cheung, Adora — Ch. 7, 9, 10, 11
Collison, John — Ch. 4, 5
Collison, Patrick — Ch. 4, 5, 6, 10
Conrad, Parker — Ch. 13
Conway, Ron — Ch. 13, 14
English, Paul — Ch. 8
Gebbia, Joe — Ch. 2, 6, 8
Gottman, John — Ch. 8
Graham, Paul — Ch. 1, 2, 4, 7, 9
Grove, Andy — Ch. 19, 21
Hale, Kevin — Ch. 8
Hoffman, Reid — Ch. 4, 18
Horowitz, Ben — Ch. 1, 20
Kan, Justin — Ch. 9, 12
Levie, Aaron — Ch. 16
Levy,Carolynn — Ch. 15
Libin, Phil — Ch. 1
Lin, Alfred — Ch. 6
Louverture, Toussaint — Ch. 20
Moskovitz, Dustin — Ch. 1
Nathoo, Kirsty — Ch. 15
Perkins, Melanie — Ch. 1
Rabois, Keith — Ch. 5, 19
Rahman, Hosain — Hardware Interlude
Rosenstein, Justin — Ch. 1
Schultz, Alex — Ch. 10, 11, 17
Seibel, Michael — Ch. 14
Shear, Emmett — Ch. 7
Silbermann, Ben — Ch. 5, 6
Spool, Jared — Ch. 8
Tang, Stanley — Ch. 9
Thiel, Peter — Ch. 2, 3, 19
Walsh, Bill — Ch. 19
Williams, Walker — Ch. 9
Zuckerberg, Mark — Ch. 1, 2, 4

Companies

Airbnb — Ch. 1, 2, 3, 5, 6, 9, 13
Amazon — Ch. 3, 16
Asana — Ch. 1
Box — Ch. 16
Canva — Ch. 1
Clever — Ch. 14, 16
DoorDash — Ch. 9
Dropbox — Ch. 1, 2
eBay — Ch. 3, 10, 11
Facebook — Ch. 1, 2, 3, 4, 10, 11, 17
GitLab — Ch. 4
Google — Ch. 2, 3, 13, 14, 18
Groupon — Ch. 4, 18
Gusto (ZenPayroll) — Ch. 16
Homejoy — Ch. 7, 9, 10, 11
Hotmail — Ch. 11
Jawbone — Hardware Interlude
Kayak — Ch. 8
LinkedIn — Ch. 18, 20
PayPal — Ch. 3, 13, 18, 19, 20
Pinterest — Ch. 5, 6
Square — Ch. 8, 19
Stripe — Ch. 2, 6, 19
Teespring — Ch. 9
Twitch (Justin.tv) — Ch. 7, 12
Wufoo — Ch. 8
Y Combinator — throughout; formation and mechanics in Ch. 15
Zapier — Ch. 4
Zappos — Ch. 6
Zenefits — Ch. 13, 16
ZenPayroll — see Gusto

Concepts and Topics

Business development — Ch. 21 Business model, stating plainly — Ch. 14 Cap table — Ch. 15; Appendix A Cofounder equity split — Ch. 4, 15 Competitive dynamics (negotiation) — Ch. 21 Culture, defining — Ch. 6 Culture, at scale — Ch. 19, 21 Diversity (hiring) — Ch. 21 Enterprise software, "why now" for — Ch. 16 Equity, employee — Ch. 5, 21 Execution, focus and intensity — Ch. 17 Firing — Ch. 5, 20 Founder psychology — Ch. 1, 18, 21, Epilogue Fundraising mechanics — Ch. 13, 14, 15 Growth channels — Ch. 11 Hardware development — Hardware Interlude Hiring, first ten employees — Ch. 5 Incorporation — Ch. 15 International expansion — Ch. 11 Legal mechanics — Ch. 15; Appendix B Management structure — Ch. 19, 21 Monopoly theory — Ch. 3 MVP — Ch. 7 Onion theory of risk — Ch. 13 Paradoxes (founder judgment) — Ch. 18 Pitching — Ch. 14; Appendix D Press and PR — Ch. 12, 21 Pricing (elephants/rabbits) — Ch. 16 Product-market fit — Ch. 7, 10 Public relations firms — Ch. 12 Retention — Ch. 7, 10 Sales (early-stage) — Ch. 16 Scaling the company — Ch. 21 Term sheets — Appendix B Unit economics (CAC, CLV) — Ch. 11; Appendix C Vesting — Ch. 4, 15; Appendix A Virality — Ch. 11 Why now (idea timing) — Ch. 2